A FINAL-YEAR PROJECT REPORT ON

ScamShield

Pre-Transaction Scam Detection and Proportional Intervention
for UPI-Style Payments using a Hybrid Rules + Machine-Learning Engine

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering / Electronics and Communication Engineering

Submitted by

Ansh	23BCS016
Harshit	23BCS054
Vijender	23BCS132
Sourabh	23BEC055

Under the guidance of

Dr. Anushree Kini

Indian Institute of Information Technology Dharwad

Dharwad, Karnataka – 580009

Academic Year 2026–27

CERTIFICATE

This is to certify that the project report entitled **"ScamShield: Pre-Transaction Scam Detection and Proportional Intervention for UPI-Style Payments using a Hybrid Rules + Machine-Learning Engine"** is a bona fide record of the work carried out by

Name	Roll Number
Ansh	23BCS016
Harshit	23BCS054
Vijender	23BCS132
Sourabh	23BEC055

in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology at the Indian Institute of Information Technology Dharwad during the academic year 2026–27, under my supervision and guidance. To the best of my knowledge, the matter embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Dr. Anushree Kini
Project Guide
IIIT Dharwad

Date: _____

Head of the Department
Department of Computer Science and Engineering
IIIT Dharwad

Date: _____

Internal Examiner
Name and signature

External Examiner
Name and signature

DECLARATION

We hereby declare that the project report entitled **"ScamShield: Pre-Transaction Scam Detection and Proportional Intervention for UPI-Style Payments using a Hybrid Rules + Machine-Learning Engine"**, submitted to the Indian Institute of Information Technology Dharwad in partial fulfilment of the requirements for the degree of Bachelor of Technology, is a record of original work carried out by us under the guidance of Dr. Anushree Kini.

The work reported here has not been submitted, in part or in full, to any other university or institute for the award of any degree or diploma. Wherever we have used material from other sources — publications, government data, software libraries or online documentation — it has been duly acknowledged and cited in the References. All users, payments, balances and fraud cases described in this report are **synthetic**; no real customer data, bank system or payment network was used.

Ansh (23BCS016)	Harshit (23BCS054)
Vijender (23BCS132)	Sourabh (23BEC055)

Place: Dharwad
Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Dr. Anushree Kini**, for her guidance, encouragement and critical feedback throughout this project. Her insistence that every result be measured honestly and every claim be defensible shaped the way we evaluated our system.

We thank the Director and the faculty of the Indian Institute of Information Technology Dharwad, and in particular the Department of Computer Science and Engineering, for providing the academic environment and computing facilities that made this work possible.

We are grateful to our classmates, friends and family members who discussed their own experiences with payment scams and fraudulent calls with us; those conversations convinced us that the problem is real and personal for many households.

Finally, we acknowledge the open-source communities behind Next.js, Express, FastAPI, scikit-learn, PostgreSQL, Redis, BullMQ, Prisma and Playwright, on whose work this project is built.

Ansh · Harshit · Vijender · Sourabh

ABSTRACT

The Unified Payments Interface (UPI) processed more than 24,000 crore transactions in FY2025-26, and its speed and irreversibility have made it a preferred channel for fraud. Most UPI fraud is not a technical break-in: the victim is persuaded — by a fake bank or KYC agent, a fake police officer, a "job offer" or a remote-access app — to authorise the payment personally. Once the money has moved, recovery is slow and uncertain. Existing defences are largely after the fact (complaint helplines, account freezing) or operate inside banks without explaining anything to the payer at the moment of decision.

This project presents **ScamShield**, a pre-transaction scam-detection and intervention system for a UPI-style wallet. Every payment is scored *before* it is authorised. A feature builder computes 20 behavioural features from Redis and PostgreSQL (amount relative to the payer's 30-day baseline, payee age, distinct senders to the payee in 24 hours, velocity, new device, failed attempts and others). Eleven configurable rules and a calibrated gradient-boosting model (scikit-learn HistGradientBoosting with isotonic calibration) produce two scores that are blended into a 0–100 risk score with LOW, MEDIUM and HIGH bands and plain-language reasons. The application intervenes proportionately: nothing for LOW; a warning with three "scam-context" questions for MEDIUM; and a blocking screen with a ten-second pause and mandatory re-typing of the recipient's UPI ID for HIGH. Analyst-confirmed scam accounts are blocked for all users. Analysts review alerts live over Server-Sent Events, and their labels feed back into payee reputation and model retraining.

The system is built as a containerised monorepo (Next.js, Express, FastAPI, PostgreSQL, Redis, BullMQ). Because real fraud data is not available to students, we built a seeded synthetic data generator with four user personas, realistic "hard negatives" and four scam scenarios, and evaluated on a time-based holdout of 45,387 payments (1.72 % scams). At the false-positive rate of the rules-only baseline (3.7 %), the hybrid engine catches **95.6 %** of scams versus **73.3 %** for rules alone; at the MEDIUM threshold it achieves 79.1 % precision and 93.2 % recall at a 0.43 % false-positive rate. We report honestly that the ML-only model ranks payments slightly better (PR-AUC 0.946 versus 0.933), and that two data leaks discovered during development were fixed rather than reported as gains. The 95th-percentile latency of the scoring endpoint was about 110 ms at 20 requests per second, against a 300 ms budget; 341 automated tests pass. The report also documents the product case — user research, market analysis, legal compliance under the DPDP Act 2023 and RBI directions, and a launch plan — while being explicit that the results come from synthetic data and do not represent real-world fraud rates.

Keywords: UPI, authorised push payment fraud, scam detection, gradient boosting, probability calibration, explainable risk scoring, security warnings, DPDP Act.

TABLE OF CONTENTS

Certificate ..... **ii**

Declaration ..... **iii**

Acknowledgement ..... **iv**

Abstract ..... **v**

List of Figures ..... **ix**

List of Tables ..... **xi**

List of Abbreviations ..... **xiii**

Chapter 1 Introduction **1**

1.1 Background and Context 1

1.2 Motivation 1

1.3 Objectives 2

1.4 Scope of the Project 3

1.5 Organisation of the Report 3

Chapter 2 Problem Definition **4**

2.1 Problem Statement 4

2.2 Why This Is a Real Problem 4

2.3 Root Causes 6

2.4 Who Is Affected and How 7

Chapter 3 User Research **9**

3.1 Research Approach 9

3.2 User Personas 10

3.3 Survey and Interview Questionnaire 11

3.4 Summary of Responses and Key Insights 11

3.5 User Journey Map 13

Chapter 4 Market and Competitive Analysis **14**

4.1 Market Size and Opportunity 14

4.2 Existing Competitors and Alternatives 15

4.3 Feature Comparison 17

4.4 SWOT Analysis 18

4.5 Unique Value Proposition 18

Chapter 5 Proposed Solution **20**

5.1 Solution Overview 20

5.2 How the Solution Addresses the Problem 21

5.3 Key Features 22

5.4 Business Model and Monetisation 23

5.5 Go-to-Market Strategy 23

Chapter 6 Requirements Specification **25**

6.1 Users and Roles 25
6.2 Functional Requirements 25
6.3 Non-Functional Requirements 26
6.4 Hardware and Software Requirements 27
6.5 Constraints and Assumptions 28

Chapter 7 Policies, Legal and Compliance 29
7.1 Privacy and Data Protection 29
7.2 Terms of Service and User Consent 30
7.3 Security Policies 31
7.4 Accessibility and Ethical Considerations 32
7.5 Domain-Specific Regulation 32

Chapter 8 System Design and Technical Details 34
8.1 System Architecture 34
8.2 Technology Stack and Justification 35
8.3 Data Flow Diagrams 36
8.4 UML Diagrams 38
8.5 Database Schema 41
8.6 APIs and Integrations 42
8.7 Algorithms and Business Rules 43
8.8 Synthetic Data Generation 45

Chapter 9 Implementation 47
9.1 Module-by-Module Description 47
9.2 Development Methodology 48
9.3 Team Roles and Responsibilities 49
9.4 Screenshots of the Working Product 50

Chapter 10 Testing and Results 58
10.1 Testing Strategy 58
10.2 Test Cases 58
10.3 Model Evaluation Results 60
10.4 Performance Results 64
10.5 User Feedback and Acceptance Testing 64

Chapter 11 Challenges and Limitations 65
11.1 Challenges Faced and How We Solved Them 65
11.2 Current Limitations 66

Chapter 12 Future Scope and Roadmap 67
12.1 Planned Improvements and Features 67
12.2 Scaling Plan 67
12.3 Product Roadmap 68

Chapter 13 Product Launch Documentation 69
13.1 Product Requirements Document (Summary) 69

13.2 Lean Canvas 69
13.3 Cost Estimation and Budget 70
13.4 Risk Analysis and Mitigation 71
13.5 Key Metrics and KPIs After Launch 72

Chapter 14 Conclusion 73

References 75

Appendix A Questionnaire 80
A.1 Section A — About you and your payments 80
A.2 Section B — Exposure to scams 80
A.3 Section C — Losses and recovery 80
A.4 Section D — Warnings and friction 81
A.5 Section E — Trust and privacy 81
A.6 Interview guide (semi-structured, 10–15 minutes) 81

Appendix B Survey Data 82

Appendix C Additional Diagrams and Tables 83

Appendix D Code Excerpts 85
D.1 Decision function (packages/shared/src/blend.ts) 85
D.2 Two example rules (packages/shared/src/rules.ts) 85
D.3 Occlusion explanation (services/ml/scamshield_ml/explain.py) 86
D.4 Atomic confirm (conceptual SQL inside one transaction) 86

Appendix E User Manual 87
E.1 Installation and start-up 87
E.2 Demo accounts 87
E.3 For payers 87
E.4 For analysts 88
E.5 For administrators 88
E.6 Troubleshooting 88

LIST OF FIGURES

2.1 Reported cyber financial-fraud losses, 2023–24, and "digital arrest" cases, 2022–24 (sources as in Table 2.1) 5
2.2 Root causes of authorised push payment fraud on UPI 6
3.1 Illustrative emotional journey of Persona 1 through a fake-KYC scam, without and with ScamShield (1 = distressed, 5 = at ease) 13
4.1 TAM, SAM and SOM for ScamShield (annual, ₹ crore, log scale) 15
5.1 ScamShield end-to-end flow from payment initiation to analyst feedback 21
8.1 System architecture of ScamShield 34
8.2 Level-0 data flow diagram (context diagram) 36
8.3 Level-1 data flow diagram 37
8.4 Use case diagram 38
8.5 Sequence diagram of the two-phase payment 39
8.6 Class diagram of the risk-decision components 40
8.7 State diagram of a transaction 40
8.8 Entity–relationship diagram (configuration tables omitted) 41
9.1 Project timeline (Gantt chart) 49
9.2 Landing page with one-click demo accounts 50
9.3 Wallet dashboard: balance, 30-day spending and recent transactions 51
9.4 MEDIUM risk: score, reasons and the three scam-context questions 51
9.5 HIGH risk: blocking screen with "Cancel and report this account" as the primary action 52
9.6 HIGH-risk screen on a 390 px mobile viewport in dark mode 53
9.7 Analyst console: live alert queue (left) and alert detail with score breakdown and review form (right) 54
9.8 Scam Simulator: a mule-collection run escalating from LOW to MEDIUM as distinct senders accumulate 54
9.9 Rules and thresholds editor with what-if preview on labelled holdout data 55
9.10 Model and evaluation page: active model, headline comparison and side-by-side metrics 56
9.11 System health: service status, circuit breaker, decisions by band and latency 57
10.1 Precision–recall curves on the holdout 61
10.2 Hybrid threshold sweep: precision, recall and false-positive rate against score threshold 61
10.3 Share of scams caught at the MEDIUM threshold, by scenario 62
10.4 Share of genuine payments wrongly flagged at the MEDIUM threshold, by legitimate scenario 62
10.5 Calibration of the ML model on the holdout 63
10.6 Permutation feature importance of the ML model (top 12 features) 63

10.7 Latency of POST /payments/check under load (1,200 requests at 20 req/s, client-side) 64
12.1 Product roadmap 68
13.1 Lean Canvas for ScamShield 70
C.1 Circuit-breaker state machine for the ML call 83
C.2 Asynchronous pipeline: alerts, reputation and the feedback loop 84
C.3 Deployment diagram (Docker Compose) 84

LIST OF TABLES

1.1	Scope of the project	3
2.1	Indicators of the scale of payment fraud in India	4
2.2	Stakeholders affected by UPI scams	7
3.1	Structure of the validation questionnaire	11
3.2	Template for reporting primary survey results (to be filled with collected data)	12
3.3	Research insights and the design decisions they led to	12
3.4	Journey stages: current experience versus experience with ScamShield	13
4.1	TAM, SAM and SOM estimates (assumptions in the last column)	14
4.2	Existing solutions and alternatives	15
4.3	Feature comparison: ScamShield versus alternatives (✓ yes, ◐ partial, ✗ no, ? not public)	17
5.1	Mapping root causes (Section 2.3) to ScamShield mechanisms	21
5.2	Core features implemented in the prototype	22
5.3	Revenue streams and pricing assumptions	23
6.1	User classes and their privileges	25
6.2	Functional requirements	25
6.3	Non-functional requirements with measurable targets and achieved status	26
6.4	Hardware requirements	27
6.5	Software requirements	28
7.1	Data-protection and cyber-security law relevant to ScamShield	29
7.2	Security controls: implemented in the prototype versus required for production	31
7.3	Ethical risks and how the design responds	32
7.4	Payments-sector regulation relevant to a ScamShield deployment	32
8.1	Containers and their responsibilities	35
8.2	Technology choices and justification	35
8.3	Database tables	41
8.4	Principal API endpoints	42
8.5	Default rules (weights and parameters are editable at runtime)	44
8.6	Synthetic population: user personas	46
8.7	Synthetic fraud scenarios (≈ 2.4 % of payments overall)	46
9.1	Repository structure	47
9.2	Development phases and deliverables	49
9.3	Team roles and responsibilities	50
10.1	Test layers, tools and results	58
10.2	Representative test cases	58
10.3	Holdout results at the MEDIUM threshold (score ≥ 35)	60
10.4	Holdout results at the HIGH threshold (score ≥ 70)	60
11.1	Development challenges and their resolution	65
12.1	Planned improvements	67

12.2 Scaling stages 68
13.1 PRD summary for ScamShield v1.0 (pilot release) 69
13.2 Estimated first-year budget for a pilot (₹) 70
13.3 Risk register (likelihood and impact: L = low, M = medium, H = high) 71
13.4 Post-launch KPIs and targets for the pilot 72
B.1 Coding sheet for survey responses 82
C.1 Redis keys, types, time-to-live and purpose 83
E.1 Demo accounts (password Demo@1234; one-click buttons on the landing page) 87

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
APP fraud	Authorised Push Payment fraud
CERT-In	Indian Computer Emergency Response Team
CFCFRMS	Citizen Financial Cyber Fraud Reporting and Management System
DFD	Data Flow Diagram
DPDP Act	Digital Personal Data Protection Act, 2023
ER	Entity–Relationship
FN / FP / TN / TP	False Negative / False Positive / True Negative / True Positive
FPR	False-Positive Rate
FRI	Financial Fraud Risk Indicator (Department of Telecommunications)
FRM	Fraud Risk Management
GBM	Gradient-Boosting Machine
I4C	Indian Cyber Crime Coordination Centre
IST	Indian Standard Time
JWT	JSON Web Token
KPI	Key Performance Indicator
KYC	Know Your Customer
ML	Machine Learning
NCRP	National Cyber Crime Reporting Portal
NPCI	National Payments Corporation of India
OTP	One-Time Password
PR-AUC	Area Under the Precision–Recall Curve
PRD	Product Requirements Document
PSP / TPAP	Payment Service Provider / Third-Party Application Provider
RBAC	Role-Based Access Control
RBI	Reserve Bank of India
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SAM / SOM / TAM	Serviceable Available / Serviceable Obtainable / Total Addressable Market
SRS	Software Requirements Specification
SSE	Server-Sent Events
UAT	User Acceptance Testing
UPI	Unified Payments Interface

UVP	Unique Value Proposition

CHAPTER 1

Introduction

1.1 Background and Context

India has become the world's largest real-time payments market. The Unified Payments Interface (UPI), operated by the National Payments Corporation of India (NPCI), lets any user move money from one bank account to another in seconds using only a virtual payment address such as `name@bank`. According to figures placed before Parliament, UPI processed about 18,587 crore transactions in FY2024-25 and about 24,162 crore transactions worth ₹314.23 lakh crore in FY2025-26 [1, 2]; NPCI publishes the monthly figures [3]. About 55.49 crore users had been onboarded by June 2026 [4], and more than 700 banks are live on the network [5]. For a street vendor, a student or a pensioner, a UPI payment is now as ordinary as handing over cash.

The properties that make UPI successful — instant settlement, no card or account details, payments to anyone, irrevocability once authorised — also make it attractive to fraudsters. Parliamentary replies show roughly 13.42 lakh UPI fraud cases involving ₹1,087 crore in FY2023-24 and 12.64 lakh cases involving ₹981 crore in FY2024-25 [6]. Across all channels, the Ministry of Home Affairs reported that citizens lost ₹22,845.73 crore to cyber financial fraud in calendar year 2024, up from ₹7,465.18 crore in 2023 [7].

A defining feature of this fraud is that, in most cases, nothing is technically "hacked". The victim personally opens their payment app, enters the amount and their UPI PIN, and sends the money — because someone posing as a bank official, a KYC desk, a courier, a police officer or a prospective buyer has convinced them to do so. In the international literature this is called *authorised push payment* (APP) fraud. Because the victim authorises the payment, controls built to stop unauthorised access — PINs, OTPs, device binding — do not stop it.

1.2 Motivation

We chose this problem for three reasons.

1. **It is close to home.** Almost every member of our team and our families has received a fake "KYC expiry", "electricity disconnection" or "job offer" message. Several of our relatives have been targeted, and some have lost money. Survey data agree: in a 2025 LocalCircles survey, one in five UPI-using families reported experiencing UPI fraud since 2022 [8].
2. **The moment of payment is the last, best chance.** Once money leaves the victim's account it is typically moved through a chain of mule accounts within minutes.

Government fraud-freezing systems have saved significant sums — more than ₹11,158 crore by June 2026 [9] — but that is a fraction of reported losses. A check that runs *before* authorisation can stop the loss rather than chase it.

3. **It combines the subjects we have studied.** The problem needs data engineering (behavioural features computed in milliseconds), machine learning under severe class imbalance, distributed-systems design (idempotency, consistency, graceful degradation) and human-centred design (warnings people actually heed). It is a good test of an engineering education.

Why we believe it will work. Three pieces of evidence give us confidence. First, scams leave behavioural fingerprints that differ from a user's normal behaviour: a first-time payee, an amount many times the user's usual, a recipient account that is only days old or that is suddenly being paid by many unrelated people. Fraud-detection research has used such signals for decades [10, 11]. Second, the context that transaction data cannot see — "someone is on the phone telling me to pay" — can be asked directly, in a few seconds, of the person about to pay. Third, our prototype, evaluated on a time-based holdout of synthetic data that includes deliberately confusing legitimate behaviour, catches 95.6 % of scams at the false-positive rate where a rules-only system catches 73.3 % (Chapter 10). These results are from synthetic data and are not a claim about real-world performance; they show that the method is sound and worth piloting with real data.

1.3 Objectives

The objectives of the project were to:

1. Score every payment for scam risk **before** it is authorised, returning a 0–100 score, a LOW / MEDIUM / HIGH band and plain-language reasons, within a 95th-percentile latency budget of 300 ms.
2. Combine transparent, configurable **rules** with a **calibrated machine-learning** model, and show on a fair, time-based evaluation whether the hybrid beats rules alone in recall at a matched false-positive rate — and report the result honestly either way.
3. Design **proportional interventions**: no friction for low-risk payments, an informative warning plus scam-context questions for medium risk, and deliberate friction (pause and verify) for high risk.
4. Provide an **analyst console** with live alerts, full explanations and a feedback loop (labels → payee reputation, hard blocks, retraining).
5. Build the system to **degrade gracefully**: if the ML service is slow or down, payments are still checked with rules and never held up.
6. Document the work as a **product**: user research, market and competitive analysis, requirements, legal and compliance considerations, and a launch plan.

1.4 Scope of the Project

Table 1.1: Scope of the project

In scope	Out of scope
A working UPI-style wallet (registration, balance, send money, history) with synthetic money	Integration with real banks, NPCI or any real payment network
Pre-transaction risk scoring: 20 features, 11 rules, calibrated ML, blending, overrides and reasons	Card-not-present, net-banking, loan or investment fraud
Proportional interventions for LOW / MEDIUM / HIGH in the payer's UI	Telecom-side controls (call/SMS blocking)
Analyst console, admin console (rules, thresholds, model, simulator, system health)	Case management, chargebacks, legal reporting workflows
Synthetic data generator, training, calibration, evaluation harness	Evaluation on real labelled fraud data (not available to us)
Product documentation: research, market, compliance, launch plan	Production certification (penetration test, RBI/PCI audit)

1.5 Organisation of the Report

Chapter 2 defines the problem with data and analyses its root causes. Chapter 3 describes user research: personas, the questionnaire and the user journey. Chapter 4 analyses the market and competitors. Chapter 5 presents the proposed solution and business model. Chapter 6 specifies the requirements. Chapter 7 covers policies, law and compliance. Chapter 8 details the system design, and Chapter 9 the implementation. Chapter 10 reports testing and results. Chapter 11 discusses challenges and limitations, Chapter 12 the future roadmap, and Chapter 13 the product-launch documentation. Chapter 14 concludes. The appendices contain the full questionnaire, the survey-data template, additional diagrams, code excerpts and a user manual.

CHAPTER 2

Problem Definition

2.1 Problem Statement

Problem statement. Users of instant, irrevocable payment systems such as UPI are being manipulated into authorising payments to fraudsters. Existing safeguards verify *who* is paying (PIN, OTP, device) but not *whether the payment makes sense*, and they act mostly after the money has gone. There is no widely available mechanism that, at the moment of payment, (a) estimates how likely this specific payment is to be a scam, (b) explains why in plain language, and (c) adds friction in proportion to the risk — without slowing down the overwhelming majority of genuine payments.

The problem has three parts. The **detection** problem is to estimate scam risk from behavioural signals within a few hundred milliseconds, with extremely imbalanced data (scams are a tiny fraction of payments) and adversaries who adapt. The **intervention** problem is to change the victim's behaviour at the critical moment, even when a scammer is coaching them on the phone, without training all users to ignore warnings. The **feedback** problem is to turn every confirmed scam into protection for the next potential victim — for example by blocking a mule account for all users once an analyst confirms it.

2.2 Why This Is a Real Problem

Table 2.1 summarises the scale of the problem using the most recent figures available to us. Figures from parliamentary replies are cited through the news reports that published them; we name the base of each figure because different publications use different scopes.

Table 2.1: Indicators of the scale of payment fraud in India

Indicator	Period	Value	Source
UPI transactions processed	FY2025-26	≈ 24,162 crore (₹314.23 lakh crore)	[1, 2]
UPI fraud cases / amount	FY2023-24	≈ 13.42 lakh / ₹1,087 crore	[6]
UPI fraud cases / amount	FY2024-25	≈ 12.64 lakh / ₹981 crore	[6]
UPI fraud amount (part year, provisional)	FY2025-26 to Dec 2025	₹805 crore	[12]
Cyber financial-fraud losses reported (all channels)	2023	₹7,465.18 crore (24.43 lakh incidents)	[7]
Cyber financial-fraud losses reported (all channels)	2024	₹22,845.73 crore (36.37 lakh incidents)	[7]
“Digital arrest” scam cases / amount	2024	1,23,672 / ₹1,935.51 crore	[13]

Amount saved through CFCFRMS (cumulative)	to Jun 2026	> ₹11,158 crore	[9]
UPI-using families reporting fraud since 2022	2025 survey	≈ 1 in 5	[8]
UPI fraud victims who did not file a complaint	2025 survey	≈ 51 %	[14]

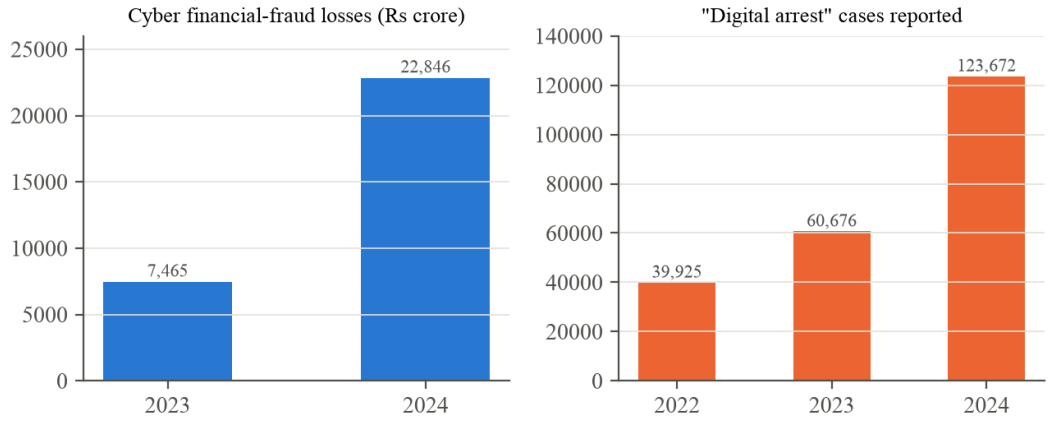


Figure 2.1: Reported cyber financial-fraud losses, 2023–24, and "digital arrest" cases, 2022–24 (sources as in Table 2.1)

Five observations follow from these numbers. First, the absolute loss is large and growing: reported cyber financial-fraud losses roughly tripled between 2023 and 2024. Second, the volume of UPI fraud cases (more than a million a year) is too large for manual handling; any defence must be automated. Third, the number of UPI fraud cases fell slightly between FY24 and FY25 while UPI volume kept growing, which suggests that controls do work — the problem is tractable. Fourth, this is high-volume, low-value retail fraud: in RBI's data for FY2024-25, card and internet frauds made up about two-thirds of bank fraud *cases* but only about 7 % of the *amount* involved [15] — many small losses to ordinary people, which are rarely worth a long investigation and therefore have to be prevented rather than pursued. Fifth, the true extent is under-reported: about half of victims surveyed never file a complaint [14].

Behind the statistics are typical stories, several of which we heard from people we know:

- **Fake KYC / bank call.** "Your account will be blocked today unless you complete KYC." The victim pays a "verification fee" or is guided to a "refund" that is actually a payment.
- **Digital arrest.** A caller posing as police or customs claims a parcel in the victim's name contained drugs and demands a "security deposit" to avoid arrest, keeping the victim on a video call for hours.
- **Job and task scams.** A "part-time job" requires a small registration fee, then larger "deposits" to unlock earnings. The collecting account is a mule paid by dozens of victims in a day.

- **Remote-access drain.** The victim is persuaded to install a screen-sharing app "for support"; the fraudster then makes several rapid payments of similar amounts.
- **Account takeover.** Stolen credentials are used from a new device, often at night, after several failed attempts, to send most of the balance to a new payee.

2.3 Root Causes

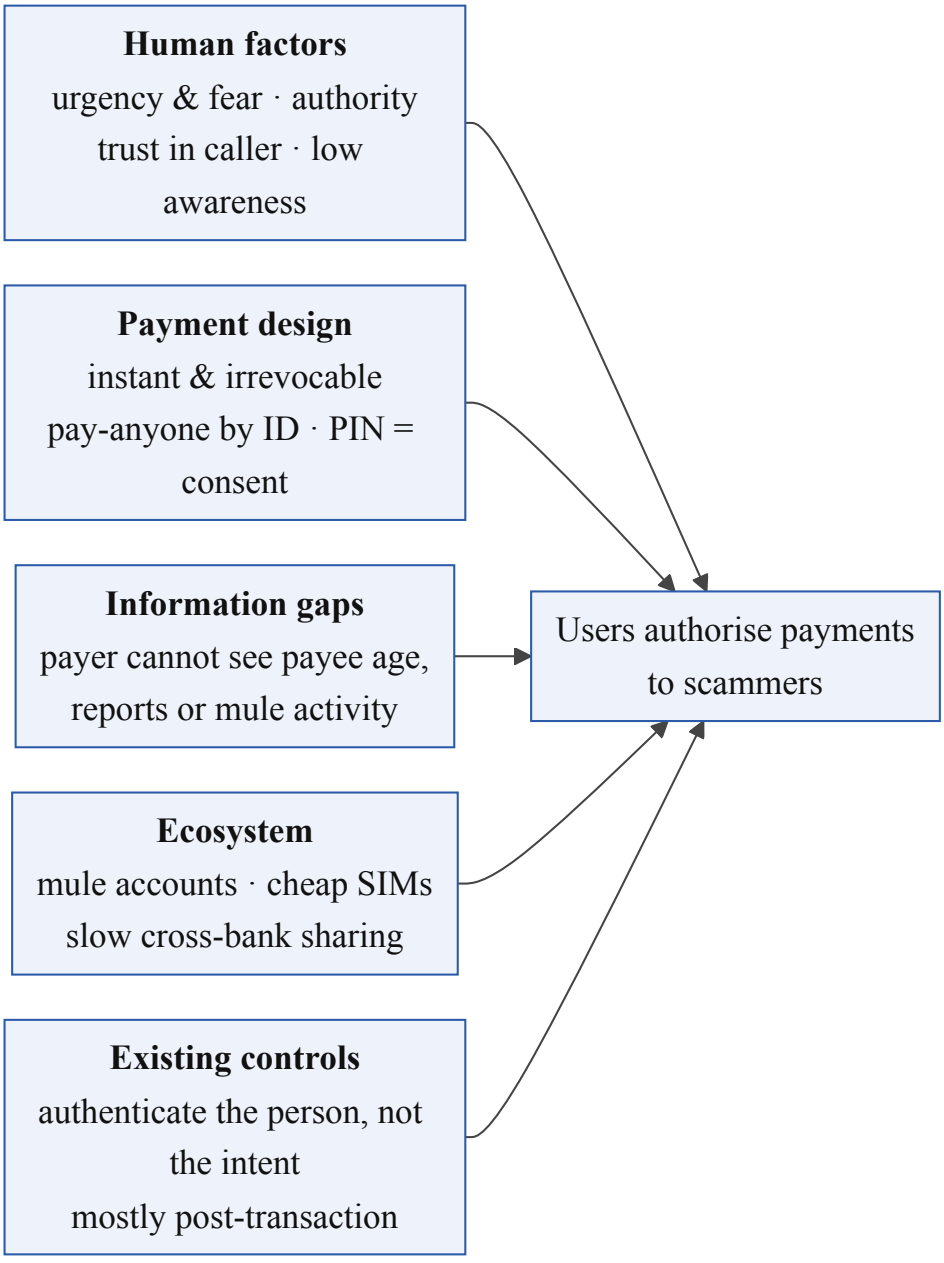


Figure 2.2: Root causes of authorised push payment fraud on UPI

2.3.1 Why the problem exists

1. **Social engineering exploits predictable psychology.** Scams rely on authority, urgency, fear, reciprocity and distraction [16]. A frightened person on a call does not stop to check

a payee's details.

2. **Authentication is not intent verification.** A UPI PIN proves that the account holder is present; it says nothing about whether they understand where the money is going. In APP fraud the right person enters the right PIN.
3. **The payer lacks information the system has.** The platform knows that a payee account is two days old, has been reported three times or received money from 15 strangers today. The payer sees only a name.
4. **Irreversibility and speed.** Settlement is instant, and funds are layered through mule accounts within minutes, so post-transaction recovery is hard.
5. **Warnings are generic and frequent.** Blanket messages ("never share your OTP") are shown to everyone, all the time, and people habituate to them [17, 18].

2.3.2 Why it has not been solved yet

- **Data is siloed and sensitive.** Fraud labels sit inside individual banks and PSPs; sharing them is constrained by privacy law and competition. Research on real payment fraud data is therefore scarce outside large institutions.
- **False positives are expensive.** With 800 million payments a day, even a 1 % false-alarm rate would interrupt millions of genuine payments. Banks therefore tune systems conservatively, and many risky payments pass.
- **Latency constraints.** Real-time payment rails leave very little time for scoring; complex models must answer in milliseconds.
- **Explainability.** Black-box scores are hard to justify to customers, regulators and analysts; rule-only systems are explainable but miss novel patterns.
- **Adversaries adapt.** Scam scripts change monthly. Static rules decay, and models trained on last year's data drift [19].

2.4 Who Is Affected and How

Table 2.2: Stakeholders affected by UPI scams

Stakeholder	How they are affected
Senior citizens	Targeted by "bank", "pension" and "digital arrest" scams; often lose savings; low familiarity with app warnings.
Students and young earners	Targeted by job, task and investment scams; small repeated payments; peer embarrassment discourages reporting.
Small merchants	Fake buyer "refund" and QR-code scams; their legitimate collection patterns (many payers) can be mistaken for mule activity by naive systems.
First-time digital users	Less able to judge legitimacy of requests; rely on caller instructions.
Banks and payment apps	Customer-support load, reputational damage, regulatory scrutiny, and (in some jurisdictions) reimbursement liability [20].

Law enforcement	Very high complaint volumes, cross-state mule networks, slow recovery.
Society	Loss of trust in digital payments, which slows financial inclusion.

CHAPTER 3

User Research

User research had two purposes: to confirm that the problem matters to real people, and to make design decisions — which signals to use, how much friction is acceptable, how warnings should be worded — on the basis of evidence rather than our own assumptions. This chapter presents the personas we designed for, the questionnaire used to validate the problem, the evidence gathered, and the user journey.

3.1 Research Approach

We combined three sources of evidence:

1. **Secondary research** — government data, regulator publications and large published surveys (Chapter 2), in particular the 2025 LocalCircles survey of more than 32,000 respondents across 365 districts [8].
2. **Primary research** — a structured questionnaire (Appendix A) for UPI users, and short semi-structured conversations with family members, classmates and local shopkeepers about scam attempts they had faced.
3. **Literature on warning design** — field studies showing that people habituate to frequent, generic security warnings and respond better to rare, specific and varied ones [17, 21, 18].

The personas below are *composite* personas in the sense of Cooper [22]: each combines traits observed across several people and published reports, and none describes a real individual. The same four user types drive the synthetic data generator (Section 8.8), so the system is designed and evaluated for the same people.

3.2 User Personas

Persona 1 — Kamala Rao, 67, retired schoolteacher (Hubballi)	
Background	Lives with her husband; son works in Bengaluru and set up her UPI app. Receives a monthly pension. Makes about three payments a day (vegetables, medicines, electricity), typically ₹100–₹800, during the day.
Goals	Pay bills independently without bothering her son; keep her savings safe.
Pain points	Trusts callers who sound official; fears her account or pension will be "blocked"; small screen text; does not know what a "collect request" is.
Behaviour	Follows instructions step by step; panics under time pressure; rarely reads long messages.
Scam risk	Fake KYC / bank / pension calls; "digital arrest".
What she needs	A clear, calm "stop" at the right moment, one sentence that explains why, and a big, obvious safe option.

Persona 2 — Rohan Kulkarni, 20, engineering student (Dharwad)	
Background	Monthly allowance from parents; about seven small payments a day (canteen, auto, recharges, ₹50–₹300), often late at night.
Goals	Fast, frictionless payments; earn some money part-time.
Pain points	Annoyed by any delay; sees "work-from-home / task" offers on social media; embarrassed to report losses to parents.
Behaviour	Tech-savvy but over-confident; ignores generic warnings; pays new people often (friends, hostel contacts).
Scam risk	Job/task scams paying a mule account; investment "groups".
What he needs	No friction for everyday payments, and a specific warning ("this account is collecting from many people today") when it matters.

Persona 3 — Imran Shaikh, 38, kirana shop owner (Belagavi)	
Background	Receives about 25 payments a day; pays suppliers in batches of three to five within minutes; amounts ₹500–₹20,000.
Goals	Keep the queue moving; pay suppliers on time; avoid fake "I sent too much, please refund" tricks.
Pain points	Cannot tolerate false alarms on routine supplier payments; often uses a second phone at the counter.
Behaviour	High volume, repetitive, time-pressured; legitimate patterns that look like velocity and new devices.
Scam risk	Fake buyer refunds, QR-code swaps, account takeover.
What he needs	A system that learns his normal behaviour, so batches and festival rush do not trigger warnings.

Persona 4 — Priya Nair, 29, fraud-operations analyst at a mid-size bank (B2B user)	
Background	Reviews flagged transactions in a vendor FRM tool; part of a small team handling a large queue.
Goals	Review the riskiest cases first; make defensible decisions quickly; stop mule accounts early.
Pain points	Alert fatigue; opaque scores without reasons; no way to see that ten victims paid the same new account today; rule changes need IT tickets.
Behaviour	Works from a queue; wants keyboard-fast review, full context on one screen, and a feedback loop that visibly improves the system.
What she needs	Live alerts with score breakdown, reasons, model contributions and payee history; one-click "confirm fraud" that blocks the payee for everyone.

3.3 Survey and Interview Questionnaire

The questionnaire (reproduced in full in Appendix A) has 20 questions in five sections, designed to take under five minutes on a phone. Table 3.1 shows its structure and the hypothesis each section tests.

Table 3.1: Structure of the validation questionnaire

Section	Questions	Hypothesis tested
A. Demographics & usage	Q1–Q5	Our personas cover the main user types; most respondents use UPI daily.
B. Exposure to scams	Q6–Q10	H1: Most respondents (or their families) have received a scam attempt; scams are mainly social engineering, not hacking.
C. Losses and recovery	Q11–Q13	H2: Victims rarely recover money and often do not report.
D. Attitudes to warnings and friction	Q14–Q18	H3: Users accept a short delay on a risky payment if it is rare and explained; H4: three yes/no questions are acceptable.
E. Trust and privacy	Q19–Q20	H5: Users accept behavioural analysis of their payments if the purpose is limited to fraud prevention.

3.4 Summary of Responses and Key Insights

3.4.1 Evidence from published surveys

The large published surveys strongly support H1 and H2. In the 2025 LocalCircles survey, about 20 % of UPI-using families reported at least one fraud incident since 2022; among victims, about half reported that their PIN or settings were compromised and about 40 % lost money after clicking a payment link [8]. About 51 % of victims never filed a complaint [14]. Government data show "digital arrest" cases — a pure social-engineering scam — tripling from 39,925 in 2022 to 1,23,672 in 2024 [13].

3.4.2 Primary survey results

TEAM TO COMPLETE — The team's own questionnaire responses are not included in the project repository, so they have not been reported here. Insert the actual number of respondents (n), the collection period and channel (e.g. Google Form shared in college and family groups), and the percentage answering each option, using the template in Appendix B. Do not estimate or round up figures that were not collected.

Table 3.2: Template for reporting primary survey results (to be filled with collected data)

Key question	% of respondents	Supports hypothesis?
Q6 — Received at least one scam call/message in the last 12 months	__ %	H1
Q8 — Self or family member lost money to a UPI scam	__ %	H1
Q12 — Money recovered (of those who lost money)	__ %	H2
Q15 — Would accept a 10-second pause on a payment flagged as high risk	__ %	H3
Q17 — Would answer three yes/no questions before a risky payment	__ %	H4
Q19 — Comfortable with payment-pattern analysis for fraud prevention only	__ %	H5

3.4.3 Key insights that shaped the design

Table 3.3: Research insights and the design decisions they led to

Insight	Design decision in ScamShield
Scams are mostly persuasion of a legitimate user; the PIN is entered by the real owner.	Score the *payment*, not just the login; add context questions that capture what transaction data cannot (a caller applying pressure, a request for an OTP or app install).
People habituate to frequent, generic warnings [17, 18].	No warning at all for LOW risk (the vast majority); warnings are specific (“this account is two days old”) and limited to the top three reasons.
Time pressure is the scammer's main tool [16].	HIGH risk imposes a ten-second pause and requires the UPI ID to be retyped; the safe action (“Cancel and report”) is the most prominent button.
A coached victim may lie to the app.	Questionnaire answers can only *raise* the score, never lower it.
Merchants and festival shoppers have unusual but legitimate behaviour.	Per-user baselines (30-day mean, spread, maximum, usual hour, known payees and devices); hard-negative scenarios in the training data.
Victims rarely report, and reports arrive late.	One-tap “Cancel and report” at the moment of the warning; reports raise the payee's risk for every user.

Analysts need context, not just a score.	Analyst view shows rule points, ML score and top contributing features, all reasons, and the payee's history; FRAUD verdict blocks the payee platform-wide.

3.5 User Journey Map

Figure 3.1 maps the emotional journey of Persona 1 (Kamala) through a fake KYC scam today, and Table 3.4 contrasts each stage with the journey when ScamShield is in place.

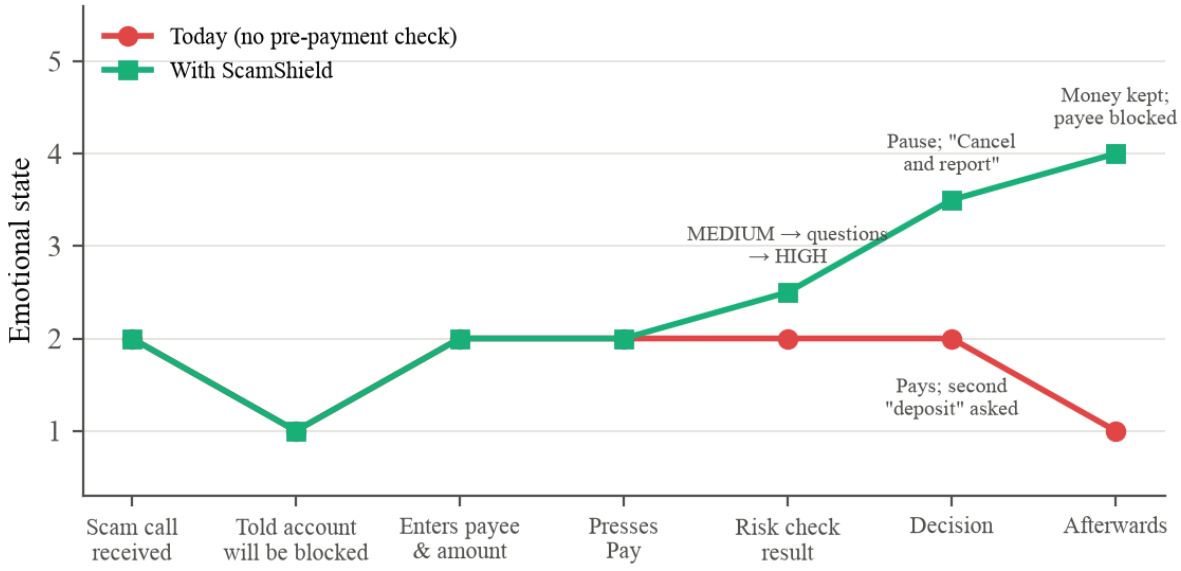


Figure 3.1: Illustrative emotional journey of Persona 1 through a fake-KYC scam, without and with ScamShield (1 = distressed, 5 = at ease)

Table 3.4: Journey stages: current experience versus experience with ScamShield

Stage	Today	With ScamShield
Contact	Unsolicited call or message; no signal from the payment app.	Unchanged (telecom layer is out of scope).
Pressure	Caller creates urgency and fear.	Unchanged — but the app is designed to interrupt it at the next step.
Recipient entry	Sees only a name.	Recipient card shows “first payment to this person” and “on ScamShield for 2 days”.
Payment check	None beyond PIN.	Risk check in ≈ 10–100 ms. MEDIUM: warning with reasons and three questions. Answering “someone called asking me to pay urgently” raises it to HIGH.
Decision	Pays immediately.	HIGH: blocking screen, ten-second pause, retype the UPI ID; “Cancel and report this account” is the primary button.
Aftermath	Loss; slow complaint; scammer reuses the account on others.	Report raises the payee's risk for everyone; analyst confirms fraud; all future payments to the account are blocked.

CHAPTER 4

Market and Competitive Analysis

4.1 Market Size and Opportunity

ScamShield is positioned as a business-to-business (B2B) risk engine for institutions that originate UPI payments — banks, small finance and cooperative banks, and third-party application providers (TPAPs) — with the consumer-facing intervention screens supplied as a software development kit (SDK). We estimate the market in two ways: top-down from published market studies, and bottom-up from UPI volumes. Every assumption is stated so that it can be challenged.

4.1.1 Top-down: fraud detection and prevention in India

Published estimates of the Indian fraud detection and prevention (FDP) market vary widely. MarketsandMarkets estimates about US$ 990 million in 2026, growing at 22.4 % a year to about US$ 2.7 billion by 2031 [23]; IMARC estimates US$ 1.4 billion in 2024, growing at about 20 % a year to US$ 8.9 billion by 2033 [24]. The two studies use different scopes and base years, so we take the lower figure as a conservative TAM. Both agree on a growth rate of 20–22 % a year.

4.1.2 Bottom-up: pre-transaction scam scoring for UPI

UPI processed about 24,162 crore (≈ 241.6 billion) transactions in FY2025-26 [1]. If every UPI payment initiated by a customer were scored by a pre-transaction scam engine priced at an assumed **2 paise per scored payment**, the annual spend would be about $241.6 \times 10^9 \times$ ₹0.02 ≈ ₹**483 crore**. This is the serviceable market for the specific capability we offer.

Table 4.1: TAM, SAM and SOM estimates (assumptions in the last column)

Level	Definition	Estimate (per year)	Basis and assumptions
TAM	All fraud detection and prevention spending in India	≈ US$ 990 million (≈ ₹8,400 crore)	MarketsandMarkets 2026 estimate [23]; ₹85 per US$ assumed. Higher published estimates exist [24].
SAM	Pre-transaction scam-risk scoring of customer-initiated UPI payments	≈ ₹483 crore	241.6 bn UPI transactions (FY26) × assumed ₹0.02 per scored payment.
SOM	Share obtainable within five years of launch	≈ ₹4.8 crore	1 % of UPI volume (≈ 2.4 bn scored payments a year), from 5–10 cooperative / small finance banks and smaller TPAPs, at ₹0.02 each.

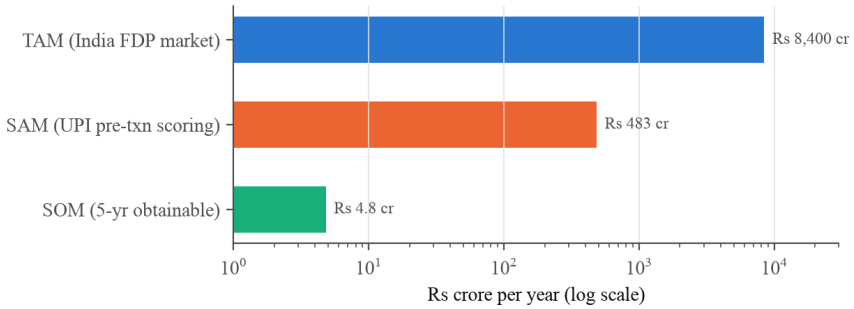


Figure 4.1: TAM, SAM and SOM for ScamShield (annual, ₹ crore, log scale)

The SOM is deliberately modest. Large banks and the largest payment apps build fraud capability in-house or buy from established vendors; our realistic first customers are smaller institutions with real-time UPI exposure but without a data-science team. Regulatory momentum works in our favour: RBI's 2025 authentication directions require risk-based checks in addition to two-factor authentication from April 2026 [25], and RBI's Innovation Hub has shown with MuleHunter.AI that banks will adopt AI-based mule detection [26, 27].

4.2 Existing Competitors and Alternatives

Table 4.2: Existing solutions and alternatives

Solution	Type	What it does	Gap relative to our problem
Bank fraud-risk-management platforms (e.g. FICO Falcon, Clari5, SAS, Feedzai)	Commercial B2B	Real-time transaction monitoring and case management inside banks, driven by rules and models.	Expensive, aimed at large banks; typically decline or hold silently — the payer is not given an explanation or asked about context.
MuleHunter.AI (RBI Innovation Hub)	Regulator-backed AI	Identifies likely mule accounts at the receiving bank; used by about two dozen banks [26, 27].	Focuses on the receiving side; does not intervene at the payer's moment of decision.
DoT Financial Fraud Risk Indicator (FRI)	Government	Rates mobile numbers as medium, high or very high fraud risk using NCRP, Chakshu and institution data; some payment apps use it to block payments [28].	One signal (the phone number) — complementary input, not a full risk engine.
Sanchar Saathi / Chakshu	Government	Citizens report suspected fraud communications; numbers and handsets are disconnected or blacklisted [29].	Telecom layer; acts on reports after the fact.
1930 helpline / CFCFRMS (I4C)	Government	Victims report fraud; funds in the chain are frozen [9].	Post-transaction; depends on fast reporting.

In-app warnings in major UPI apps	Built-in	Apps show warnings for some suspicious payees or links; details are proprietary.	Not available to smaller banks/TPAPs; approach not published, so effectiveness cannot be assessed.
Caller-identification apps (e.g. Truecaller)	Consumer app	Labels spam and suspected fraud calls.	Does not see the payment; easily bypassed with fresh numbers or messaging apps.
UK Confirmation of Payee and APP reimbursement (international)	Regulatory model	Name-matching check before payment; mandatory reimbursement of APP fraud, split between sending and receiving firms, since October 2024 [20].	Reference for policy; reimbursement creates strong incentives for pre-transaction controls.

4.3 Feature Comparison

Table 4.3: Feature comparison: ScamShield versus alternatives (✓ yes, ◐ partial, ✗ no, ? not public)

Capability	ScamShield	Bank FRM platform	MuleHunter.AI	DoT FRI	In-app warnings	1930 / CFCFRMS
Acts before the money moves	✓	✓	◐	✓	✓	✗
Explains the risk to the payer in plain language	✓	✗	✗	◐	◐	✗
Proportional friction (none / warn / pause-and-verify)	✓	◐	✗	◐	?	✗
Asks the payer for scam context	✓	✗	✗	✗	?	✗
Per-user behavioural baseline	✓	✓	?	✗	?	✗
Cross-payer mule signal	✓	◐	✓	◐	?	◐
Analyst console with live alerts and feedback loop	✓	✓	?	✗	✗	◐
Runtime-editable rules with what-if preview	✓	✓	?	✗	✗	✗
Graceful degradation if the model is down	✓	?	?	—	?	—
Affordable for small banks / TPAPs	✓	✗	◐	✓	✗	✓
Validated on real fraud data	✗	✓	✓	✓	?	—

The comparison is deliberately candid: the established systems have one decisive advantage that we lack — validation on real fraud data. Our differentiation lies in the payer-facing half of the problem.

4.4 SWOT Analysis

Strengths

- Hybrid engine: explainable rules + calibrated ML, with hard overrides
- Payer-facing, proportional intervention and context questions
- Low latency (p95 ≈ 110 ms) and graceful degradation
- Honest, reproducible evaluation harness and simulator
- Runtime-tunable rules and thresholds with what-if preview

Weaknesses

- Trained and evaluated only on synthetic data
- No integration with NPCI or real bank systems yet
- Questionnaire relies on the user's honesty
- Single-instance deployment tested only to ≈ 20 requests/s
- Student team without sales or compliance track record

Opportunities

- RBI's 2025 risk-based authentication directions [25]
- Fast-growing FDP market (≈ 20 % a year) [23, 24]
- Thousands of cooperative and small banks lacking in-house data science
- Integration of public signals (DoT FRI, Suspect Registry)
- Multilingual warnings for first-time digital users

Threats

- Large UPI apps and established FRM vendors adding similar features
- Scammers adapting scripts to coach victims through warnings
- Regulatory and data-protection obligations raise the cost of entry
- False positives damaging user trust and institution adoption
- Long B2B sales and security-audit cycles in banking

4.5 Unique Value Proposition

For banks and payment apps whose customers are being talked into paying scammers, **ScamShield** is a pre-transaction scam-risk engine that **stops the scam before the money moves** — explaining the risk to the payer in one sentence, asking the questions a fraud analyst would ask, and adding friction only when it matters. **Unlike** conventional fraud-monitoring platforms that silently decline or hold payments, ScamShield turns the payer into the last line of defence, and turns every confirmed scam into protection for every other user.

Four things make the proposition distinctive:

1. **Intervention, not just detection.** The output is a designed user experience (none / warn and ask / pause and verify), not only a score.
2. **Explainable by construction.** Every decision carries reason codes rendered as plain sentences for users and technical detail for analysts.
3. **Measured honestly.** The evaluation harness compares rules, ML and hybrid on a time-based holdout and publishes the trade-offs, including where the hybrid is worse.

4. **Built for smaller institutions.** Container-based, runs on modest hardware, and lets operations staff tune rules without code changes.

CHAPTER 5

Proposed Solution

5.1 Solution Overview

ScamShield inserts a risk check between “the user presses Pay” and “the money moves”. A payment is a two-phase operation: **check**, which scores the payment and never moves money, and **confirm**, which moves the money only if the client presents a valid, short-lived check token and has satisfied any intervention the risk band requires. Figure 5.1 shows the end-to-end flow.

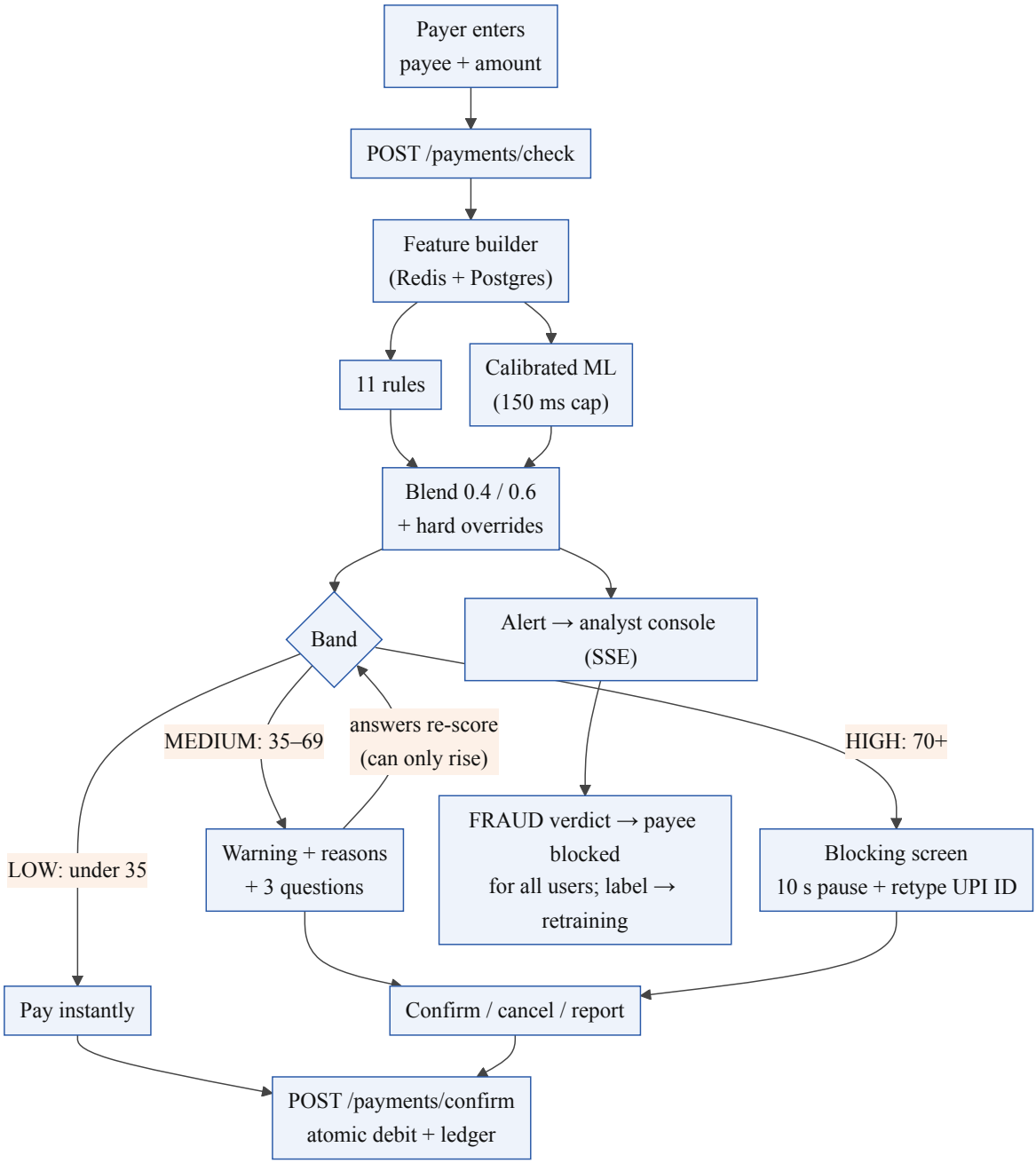


Figure 5.1: ScamShield end-to-end flow from payment initiation to analyst feedback

5.2 How the Solution Addresses the Problem

Table 5.1: Mapping root causes (Section 2.3) to ScamShield mechanisms

Root cause	ScamShield mechanism
Authentication is not intent verification	Every payment is scored on behavioural features, regardless of correct PIN/credentials.
The payer lacks information the platform has	Recipient card shows first-payment status and account age; reasons expose payee reports, mule pattern and amount anomaly.
Time pressure and fear	HIGH risk enforces a ten-second pause and retyping of the UPI ID; “Cancel and report” is the primary action.

Transaction data cannot see the phone call	Three scam-context questions; “asked for an OTP or to install an app” forces HIGH.
Habituation to generic warnings	No friction for LOW; at most three specific reasons; warnings tied to concrete facts.
Mule accounts reused across victims	Distinct-senders-in-24-hours signal; reports and analyst verdicts propagate to all users; confirmed scam accounts are blocked outright.
Static rules decay; models drift	Runtime-editable rules with what-if preview; analyst labels feed retraining; evaluation harness rerun on every training.

5.3 Key Features

5.3.1 Core features (implemented)

Table 5.2: Core features implemented in the prototype

#	Feature	Description
1	Hybrid risk engine	20 features, 11 weighted rules, calibrated gradient-boosting model, blend (default 0.4 rules / 0.6 ML), bands LOW < 35 ≤ MEDIUM < 70 ≤ HIGH.
2	Hard overrides	Analyst-confirmed scam payee → score ≥ 95, blocked. OTP/app-install request → at least HIGH.
3	Plain-language reasons	Reason catalogue (keyed by locale) renders top three reasons for users and full technical detail for analysts.
4	Proportional interventions	LOW: none. MEDIUM: warning, reasons, questionnaire, “Verify recipient / Cancel / Continue anyway”. HIGH: blocking modal, 10 s cooldown, retype UPI ID.
5	Scam-context questionnaire	Three yes/no questions become features; the payment is re-scored; the score can only rise.
6	Per-user baselines	30-day mean, spread, maximum, usual hour (circular mean), known payees and devices; fraud-labelled payments excluded.
7	Payee reputation and reporting	Reports (once per user), distinct senders, account age and analyst verdicts combine into a reputation score.
8	Analyst console	Live alert queue over Server-Sent Events; score breakdown; reasons; ML contributions; payee history; FRAUD / SAFE verdicts with notes.
9	Admin console	Rules and thresholds editor with what-if preview; model and evaluation page; retraining; system health.
10	Scam simulator	Runs normal, impersonation, mule, velocity-drain and account-takeover scenarios through the real pipeline.
11	Resilience	150 ms ML timeout, Redis-backed circuit breaker, rules-only “degraded” mode, idempotent check, atomic and double-spend-safe confirm.
12	Evaluation harness	Rules vs ML vs hybrid on a time-based holdout: PR-AUC, precision, recall, FPR, confusion matrices, calibration, threshold sweep, per-scenario results.

5.3.2 Future features

- Graph-based mule-ring detection over the payee–sender network; device fingerprinting.
- Integration of external risk signals: DoT FRI for the payee's phone number, I4C Suspect Registry.
- Multilingual reasons and voice prompts (the catalogue is already keyed by locale).
- "Trusted contact" confirmation for senior citizens on HIGH-risk payments.
- Model registry with shadow and canary deployment; drift monitoring.
- Federated learning across institutions to share patterns without sharing customer data.

5.4 Business Model and Monetisation

ScamShield is offered as a B2B service to payment-originating institutions. The end user never pays.

Table 5.3: Revenue streams and pricing assumptions

Stream	Model	Indicative price (assumption)
Risk-scoring API	Pay per scored payment, with volume tiers	₹0.02 per check, falling to ₹0.01 above 100 million checks a month
Platform subscription	Analyst and admin consoles, simulator, evaluation reports	₹50,000–₹2,00,000 per month by seat count
Intervention SDK	Licensed UI components for Android/iOS/web wallets	Included with subscription
Implementation and tuning	One-time onboarding, data mapping, model calibration on the client's data	₹5–15 lakh per client
On-premise deployment	Annual licence for institutions with strict data-localisation needs	Custom

The value to a customer is reduced fraud losses and support costs, protection from reputational damage, and evidence for regulators of risk-based controls. Where reimbursement liability exists (as in the UK since 2024 [20]), each prevented scam has a direct monetary value to the institution.

5.5 Go-to-Market Strategy

1. **Phase 1 — Pilot (months 0–6).** Partner with one cooperative or small finance bank, or a university/campus payment app. Run ScamShield in *shadow mode* (scoring without intervening) to measure precision and recall on real data, then enable interventions for a consenting user cohort.

2. **Phase 2 — Early adopters (months 6–18).** Use pilot results as a case study; target cooperative banks through their federations and technology service providers, and smaller TPAPs. Offer the risk-scoring API with a free tier up to a monthly volume.
3. **Phase 3 — Scale (months 18–36).** Seek inclusion in sandbox programmes (RBI Regulatory Sandbox, RBI Innovation Hub); integrate with core banking and UPI switch vendors as a plug-in; add regional languages.

Channels: direct sales to bank risk teams, partnerships with core-banking and UPI-switch vendors, fintech conferences and hackathons, and publication of our evaluation methodology to build credibility. **Positioning:** "the explainable, payer-facing layer that complements your existing fraud monitoring."

CHAPTER 6

Requirements Specification

This chapter specifies the software requirements following the structure recommended by IEEE Std 830 [30]. Requirements are identified so that they can be traced to design elements (Chapter 8) and test cases (Chapter 10). Priorities use the MoSCoW scheme (M = must, S = should, C = could).

6.1 Users and Roles

Table 6.1: User classes and their privileges

Role	Description	Main functions
User (payer)	Wallet holder sending money	Register, log in, view balance and history, send money, answer questions, cancel and report, view My Safety
Analyst	Fraud-operations staff	Everything a user can do, plus live alert queue, alert detail, FRAUD / SAFE verdicts
Admin	Risk manager / system owner	Everything an analyst can do, plus rules and thresholds, what-if, model retraining, evaluation, simulator, system health

6.2 Functional Requirements

Table 6.2: Functional requirements

ID	Requirement	Priority
Accounts and wallet		
FR-1	The system shall allow a person to register with name, e-mail and a password of at least 8 characters, assign a UPI ID and credit synthetic starting balance (₹25,000).	M
FR-2	The system shall authenticate users and maintain a session using an httpOnly cookie, and shall count failed logins as a risk signal.	M
FR-3	The system shall show balance, 30-day spending by day, transaction history (filterable by status and risk band) and number of warnings received.	M
FR-4	The system shall let a user look up a payee and see display name, age on the platform, report count and whether the user has paid them before.	S
Risk check and payment		
FR-5	On a payment request, the system shall compute the 20 defined features from the user's and payee's state without moving any money.	M
FR-6	The system shall evaluate all enabled rules, obtain a calibrated ML probability, blend them into a 0–100 score and assign a band using configurable thresholds.	M

FR-7	The system shall apply hard overrides: confirmed scam payee → score ≥ 95 and blocked; OTP/app-install request → at least HIGH.	M
FR-8	The system shall return reason codes, a one-line summary and the top three plain-language reasons with every decision.	M
FR-9	The check endpoint shall be idempotent with respect to an `Idempotency-Key` header.	M
FR-10	The system shall issue a signed check token (10 minutes) binding user, transaction, amount and payee; confirm shall require it.	M
FR-11	For LOW risk the payment shall proceed with no additional step.	M
FR-12	For MEDIUM risk the UI shall show the warning, reasons and three yes/no questions; answers shall re-score the payment and may only raise the score.	M
FR-13	For HIGH risk the system shall enforce a 10-second cooldown and require the payee UPI ID to be retyped exactly before confirm succeeds.	M
FR-14	Confirm shall debit the payer, write ledger entries, credit a platform payee and mark the transaction completed atomically, and shall prevent double spending.	M
FR-15	The user shall be able to cancel a pending payment and optionally report the payee; each user may report a payee once.	M
Analyst and admin		
FR-16	MEDIUM and HIGH decisions shall create alerts that appear in the analyst console in real time.	M
FR-17	The analyst shall see features, all reasons with technical detail, ML top contributing features and payee history, and record a FRAUD or SAFE verdict with notes.	M
FR-18	A FRAUD verdict shall label the transaction, mark the payee as confirmed fraud and update its reputation.	M
FR-19	The admin shall enable/disable rules and edit their weights and parameters, blend weights and band thresholds at runtime, with a what-if preview on labelled holdout data.	S
FR-20	The admin shall trigger retraining using synthetic data plus analyst labels and view the evaluation of the active model.	S
FR-21	The admin shall run scam scenarios (normal, impersonation, mule, velocity, takeover) through the real pipeline.	S
FR-22	The admin shall view system health: service status, decisions by band, latency percentiles, ML outcomes and breaker state, queue depths.	S
FR-23	Security-relevant administrative actions (verdicts, rule changes, retraining, simulator runs) shall be written to an audit log.	M

6.3 Non-Functional Requirements

Table 6.3: Non-functional requirements with measurable targets and achieved status

ID	Category	Requirement	Status (Chapter 10)
NFR-1	Performance	p95 latency of `POST /payments/check` < 300 ms end-to-end on the reference machine.	Met: ≈ 110 ms at 20 req/s
NFR-2	Performance	ML scoring call capped at 150 ms; ML service p95 < 50 ms for a single request.	Met (tested)

NFR-3	Availability	If the ML service is slow or down, payments shall still be scored with rules only and flagged as degraded.	Met (tested)
NFR-4	Consistency	No negative balances; exactly one of several concurrent confirms succeeds.	Met (tested, 5 concurrent)
NFR-5	Security	Passwords hashed (bcrypt, cost 10); role checks server-side on every route; input validated on every request; rate limits on auth (20/min/IP) and check (120/min/user).	Met
NFR-6	Security	Check tokens signed (HS256) and short-lived; tampered or foreign tokens rejected.	Met (tested)
NFR-7	Privacy	Device identifiers stored only as hashes; no card or bank data; synthetic data only.	Met
NFR-8	Scalability	API and ML services stateless so they can scale horizontally; shared state in Redis/PostgreSQL.	Designed; not load-tested beyond one instance
NFR-9	Usability	No added step for LOW payments; at most three reasons shown; HIGH screen cannot be dismissed accidentally; dark mode and mobile layout.	Met (screenshots, e2e)
NFR-10	Explainability	Every decision carries reason codes; analysts see per-feature model contributions.	Met
NFR-11	Reproducibility	Data generation, training and evaluation are deterministic for a given seed.	Met (seed 42)
NFR-12	Maintainability	Feature and rule definitions shared between Node and Python via a JSON spec with parity tests.	Met (300 parity cases)
NFR-13	Portability	Whole stack starts with one command on any machine with Docker.	Met (`make demo`)
NFR-14	Observability	Structured logs with request IDs; health and readiness endpoints; metrics page.	Met

6.4 Hardware and Software Requirements

Table 6.4: Hardware requirements

Environment	Minimum	Used for development and testing
Development / demo machine	4-core 64-bit CPU, 8 GB RAM, 10 GB free disk	Apple-silicon laptop, 10 cores, Docker Desktop
Client	Any modern browser (desktop or mobile)	Chromium, Safari; 390 px mobile viewport
Pilot server (projected)	2 × 4 vCPU / 16 GB VMs + managed PostgreSQL and Redis	—

Table 6.5: Software requirements

Component	Version
Operating system	macOS, Linux or Windows with WSL2
Docker Engine / Compose	Docker Desktop with Compose v2
Node.js	≥ 20 (TypeScript 5.6)
Python	3.11+ with FastAPI 0.115, scikit-learn 1.6, pandas 2.2, NumPy 2.2
Web	Next.js 15, React 19, Tailwind CSS 4, TanStack Query 5, Recharts 2
API	Express, Prisma 6, ioredis, BullMQ, zod, pino
Data stores	PostgreSQL 16, Redis 7
Testing	Vitest, supertest, pytest, Playwright, autocannon

6.5 Constraints and Assumptions

6.5.1 Constraints

- **No real payment network or data.** Students cannot connect to NPCI or obtain bank fraud data, so all users, money and fraud cases are synthetic.
- **Time.** The core build had to be completed within a one-week sprint, which favoured simple, well-understood components over breadth.
- **Latency.** Scoring must fit inside a real-time payment flow; this rules out heavy models and slow explanation methods at request time.
- **Hardware.** Everything must run on a single laptop for the demonstration.
- **Regulation.** A production deployment would be subject to RBI, NPCI and DPDP Act obligations (Chapter 7) which a student prototype cannot meet in full.

6.5.2 Assumptions

- All hour-of-day features use Indian Standard Time; the maximum payment is ₹1,00,000.
- The payer's device sends a stable random identifier; the server stores only its hash.
- `payee_age_days` is time since the payee was first seen on the platform (0 if never).
- `failed_attempts_15min` counts failed logins plus failed recipient-retype checks.
- Questionnaire answers are voluntary and may be untruthful; they can therefore only increase risk.
- Analyst verdicts are authoritative for blocking a payee; user reports alone never block.

CHAPTER 7

Policies, Legal and Compliance

The prototype uses only synthetic data, so no law is engaged by the demonstration itself. A real deployment, however, would process personal and financial data of identifiable people and make decisions that affect their payments. This chapter sets out the policies and controls a launch would require. It reflects our reading of the relevant laws and regulations and is not legal advice; any launch would require review by qualified counsel and the partner institution's compliance team.

7.1 Privacy and Data Protection

7.1.1 Applicable law in India

Table 7.1: Data-protection and cyber-security law relevant to ScamShield

Instrument	Key obligations relevant to ScamShield
Digital Personal Data Protection Act, 2023 [31]	Personal data may be processed only for a lawful purpose, on the basis of consent following a clear notice or for a specified legitimate use; obligations of the *Data Fiduciary* include purpose limitation, accuracy, reasonable security safeguards, breach notification to the Data Protection Board and to affected *Data Principals*, erasure when the purpose is served, and a grievance mechanism. Penalties for failing to take reasonable security safeguards can reach ₹250 crore.
DPDP Rules, 2025 [32]	Notified in November 2025 with a staggered roll-out: most substantive obligations (notice, security, breach intimation, retention, rights) apply about 18 months after notification, i.e. around May 2027. A product launching in 2027 must comply from day one.
Information Technology Act, 2000 [33] and SPDI Rules, 2011 [34]	Section 43A (compensation for negligence in protecting sensitive personal data, including financial information) and the SPDI Rules remain relevant until the DPDP regime fully applies; Section 72A penalises disclosure of personal information in breach of contract. Sections 66C and 66D (identity theft; cheating by personation) describe the offences ScamShield helps prevent.
CERT-In Directions, 28 April 2022 [35]	Report specified cyber incidents to CERT-In within 6 hours; maintain ICT system logs for a rolling 180 days within India; synchronise clocks to national time servers.
RBI, Storage of Payment System Data, 2018 [36]	Payment-system data must be stored only in India — the platform must be hosted in Indian regions.
GDPR (if offered outside India) [37]	Lawful basis, data-protection impact assessment, rights not to be subject to solely automated decisions with significant effects (Art. 22) — relevant because HIGH-risk friction is automated.

7.1.2 Privacy policy (summary of the proposed policy)

1. **What we collect.** Account details (name, e-mail, UPI ID); payment details (amount, payee, time); a hashed device identifier; login and verification outcomes; answers to the scam-context questions; reports filed. We do **not** collect contacts, SMS, call logs, location, card numbers or bank credentials.
2. **Why.** Solely to detect and prevent payment fraud and scams, to operate the wallet, and to meet legal obligations. Data is not used for advertising, credit scoring or sale to third parties.
3. **Legal basis.** Consent obtained through a clear, itemised notice at onboarding (available in English and the user's language), and legitimate uses permitted by law where applicable.
4. **Retention.** Raw behavioural features for 12 months (needed to retrain models and handle disputes); aggregated baselines for 30 days; audit logs and security logs as required by CERT-In and DPDP rules; deletion or anonymisation thereafter.
5. **Sharing.** With the partner bank or PSP that holds the user's account; with law enforcement when legally required; confirmed-fraud payee identifiers may be shared with industry registries.
6. **User rights.** Access a summary of data held, correction, erasure (subject to legal retention), grievance redressal within the statutory period, and nomination. Users can see every warning they received and every report they filed on the *My Safety* page.
7. **Security.** Encryption in transit and at rest, access control, audit logging, breach-response plan (Section 7.3).
8. **Contact.** A named Grievance Officer / Data Protection contact.

7.2 Terms of Service and User Consent

- **Nature of the service.** ScamShield provides risk *warnings*; it lowers but cannot eliminate the risk of fraud, and does not guarantee that every scam will be caught or that every flagged payment is a scam.
- **User autonomy.** For MEDIUM risk the user may continue after reading the warning. For HIGH risk the user may continue after the pause and verification. Only payments to analyst-confirmed scam accounts are blocked outright; such blocks can be appealed through the grievance channel.
- **Questionnaire.** Answering is voluntary; answers are used only to assess the current payment and to improve detection.
- **Reports.** Users must report in good faith; malicious reporting may lead to suspension. Reports alone never block a payee.
- **Consent flow.** Consent is collected with an unbundled notice (separate from the wallet terms), can be withdrawn at any time, and withdrawal is as easy as giving consent. Since

fraud screening is integral to the payment service, withdrawal may limit the ability to make payments through the partner app — this is disclosed up front.

- **Liability and disputes.** Governed by the partner institution's customer-protection policy and applicable RBI directions; jurisdiction of Indian courts.

7.3 Security Policies

Table 7.2: Security controls: implemented in the prototype versus required for production

Area	Implemented in prototype	Required before production
Authentication	bcrypt-hashed passwords (cost 10); session JWT (HS256, 8 h) in httpOnly, SameSite=Lax cookie; generic error messages; constant-time-ish login for unknown e-mails; failed-attempt counting	Delegate to the partner bank's authentication; MFA for analysts and admins; SSO; secure flag over HTTPS only
Authorisation	RBAC (USER < ANALYST < ADMIN) enforced server-side on every route	Four-eyes approval for rule/threshold changes; least-privilege service accounts
Payment integrity	Signed, short-lived check token binding user, transaction, amount and payee; conditional updates prevent double spending and negative balances	HSM-backed keys; key rotation
Input and API	zod validation on every body, query and parameter; 100 kB body limit; helmet headers; CORS restricted to the web origin; rate limits in Redis	Web application firewall; API gateway; OWASP ASVS Level 2 verification [38]
Encryption	Not applicable on localhost	TLS 1.2+ everywhere; encryption at rest for PostgreSQL, Redis persistence and backups
Data storage	Device IDs hashed; no card/bank data; synthetic data only; secrets in `.env`, not in the repository	Secrets manager; India-region hosting; field-level encryption of personal data; data-retention jobs
Logging and audit	Structured logs with request IDs, secrets and cookies redacted; audit log for verdicts, rule changes, retraining, simulator runs	Tamper-evident log storage, 180-day retention in India, SIEM integration
Incident response	—	Documented plan: CERT-In report within 6 hours; DPDP Board and data-principal notification; post-incident review
Assurance	341 automated tests including token tampering, RBAC and concurrency	Independent penetration test; RBI/partner security audit

7.4 Accessibility and Ethical Considerations

7.4.1 Accessibility

ScamShield's users include senior citizens and first-time digital users. The interface targets WCAG 2.2 Level AA [39], in line with the spirit of the Rights of Persons with Disabilities Act, 2016 [40]: risk is never conveyed by colour alone (each band has an icon and text label), the warning screens use short sentences, buttons are large and clearly labelled, and the layout works at 390 px width and in dark mode. Future work includes screen-reader testing, regional languages and voice read-out of HIGH warnings.

7.4.2 Ethics

Table 7.3: Ethical risks and how the design responds

Concern	Design response
False positives burden innocent users	Thresholds chosen on measured false-positive rates (0.43 % at MEDIUM on the holdout); LOW payments see nothing; hard-negative scenarios in training; per-scenario false-alarm reporting (e.g. festival spending, merchant batches).
Bias against groups (e.g. seniors, new users)	No demographic attributes are used as features; behaviour is compared to the user's own baseline. In production, false-positive rates should be monitored by user segment.
Paternalism vs autonomy	Friction is proportional and explained; the user may proceed in all cases except confirmed scam accounts.
Dark patterns	Friction is used only to slow down a likely scam, never to drive commercial outcomes; the safe option is prominent but every option is visible.
Automated decisions	Every decision has human-readable reasons; analysts review MEDIUM/HIGH alerts; payee blocks require a human verdict.
Malicious reporting	One report per user per payee; reports raise risk but never block; analyst review is authoritative.
Honest communication of results	We report that results come from synthetic data, that ML-only ranks slightly better than hybrid, and the data leaks we found and fixed.

7.5 Domain-Specific Regulation

Table 7.4: Payments-sector regulation relevant to a ScamShield deployment

Regulation	Relevance
RBI Master Direction on Digital Payment Security Controls, 2021 [41]	Requires regulated entities to have governance, authentication and fraud-risk-management controls for digital payments, including real-time transaction monitoring — ScamShield is one such control.
RBI Master Directions on Fraud Risk Management, 2024 [42]	Board-approved fraud-risk policy, early-warning signals, red-flagging and reporting. ScamShield's alerts and audit trail can feed the partner's early-warning framework.

RBI Authentication Mechanisms for Digital Payment Transactions Directions, 2025 [25]	From 1 April 2026, in addition to two-factor authentication, issuers are expected to apply risk-based checks — directly supporting the case for pre-transaction risk scoring.
NPCI UPI operating guidelines and API usage limits (2025) [43]	Any integration must respect NPCI's API rate limits and operating procedures; scoring must run within the PSP's own systems, not by polling NPCI.
Outsourcing of financial services (RBI)	A bank using ScamShield as a vendor service must meet RBI's outsourcing and IT-outsourcing expectations: audit rights, data location, business continuity.
DoT Financial Fraud Risk Indicator [28]	An external signal that a deployment could consume (risk of the payee's mobile number), subject to the terms of access.

CHAPTER 8

System Design and Technical Details

8.1 System Architecture

ScamShield is a service-oriented system of six containers orchestrated with Docker Compose [44] (Figure 8.1). The design separates the **hot path** — everything that must happen before the payer sees a decision — from **asynchronous work** that must not slow the payer down (alerts, baselines, reputation, metrics, retraining).

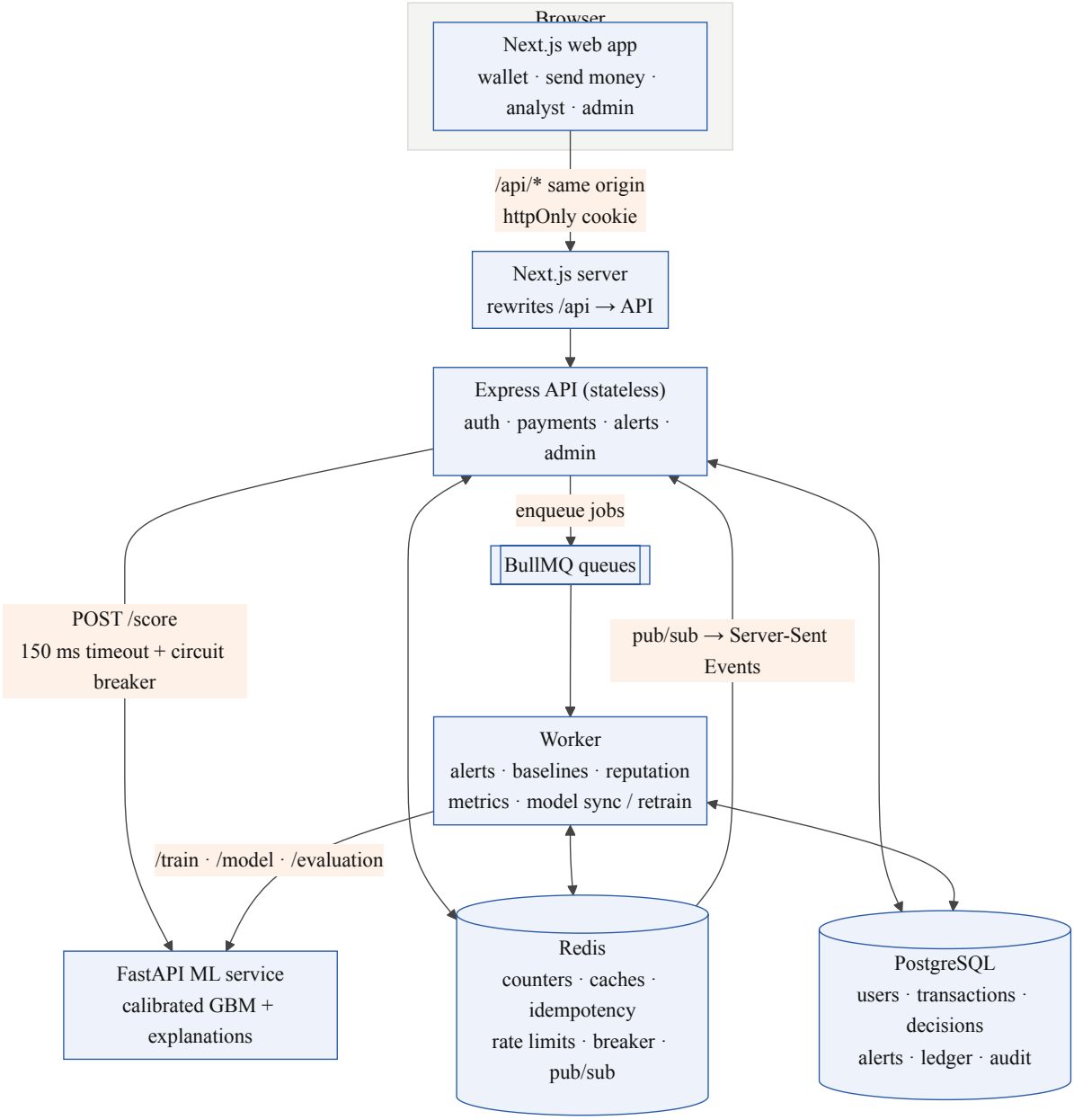


Figure 8.1: System architecture of ScamShield

Table 8.1: Containers and their responsibilities

Container	Responsibility
web	Next.js App Router UI; proxies `/api/*` to the API so that the session cookie stays same-origin.
api	Stateless Express API: authentication, payment check/confirm, alerts, admin; runs database migrations and an idempotent seed on start.
worker	Same image as the API running BullMQ consumers: alerts, baseline refresh, payee reputation, metrics roll-up, model sync and retraining.
ml	FastAPI service; on first boot generates synthetic data, trains, calibrates and evaluates the model, then serves `/score`.
postgres	System of record (PostgreSQL 16).
redis	Hot state: sliding-window counters, caches, idempotency records, rate limits, circuit breaker, job queues, pub/sub (Redis 7).

8.2 Technology Stack and Justification

Table 8.2: Technology choices and justification

Layer	Choice	Why we chose it (and what we rejected)
Front end	Next.js 15, React 19, Tailwind CSS 4, Radix UI, TanStack Query, Recharts [45]	Server-side rewrites keep cookies same-origin (no token in JavaScript); accessible Radix primitives for modal dialogs that cannot be dismissed accidentally; Recharts for the evaluation charts. A plain SPA would have needed CORS cookies.
API	Node.js + Express + TypeScript [46]	The rules, features and reason catalogue are shared TypeScript used by both the web and the API, so one language end-to-end; Express is minimal and well understood for a viva.
Validation	zod	One schema gives runtime validation and TypeScript types; also generates OpenAPI docs.
ORM / DB	Prisma 6 + PostgreSQL 16 [47, 48]	ACID transactions and conditional updates for money movement; declarative schema and migrations. A document store was rejected because the ledger needs transactions.
Hot state	Redis 7 [49]	Sorted sets give O(log n) sliding windows for velocity and distinct-sender counts; atomic SET NX for idempotency and breaker probes; pub/sub for live alerts.
Queues	BullMQ [50]	Retries with back-off on Redis we already run; Kafka would be over-engineered at this scale (noted as future work).
ML service	Python, FastAPI, scikit-learn [51, 52]	scikit-learn's HistGradientBoosting is fast, handles unscaled features and needs no GPU; FastAPI gives typed request validation. XGBoost/LightGBM [53, 54] and SHAP [55] were left out to keep the container small; occlusion explanations are used instead.
Live updates	Server-Sent Events	One-way server → analyst stream over plain HTTP; simpler than WebSockets and works through the Next.js proxy.

Testing	Vitest, supertest, pytest, Playwright [56], autocannon	Integration tests against real PostgreSQL/Redis; browser end-to-end tests; HTTP load tests.
Deployment	Docker Compose	One command (`make demo`) starts the whole stack identically on every team member's machine.

8.3 Data Flow Diagrams

Figure 8.2 is the context diagram (Level 0): ScamShield as a single process with its external entities. Figure 8.3 decomposes it into its main processes and data stores (Level 1).

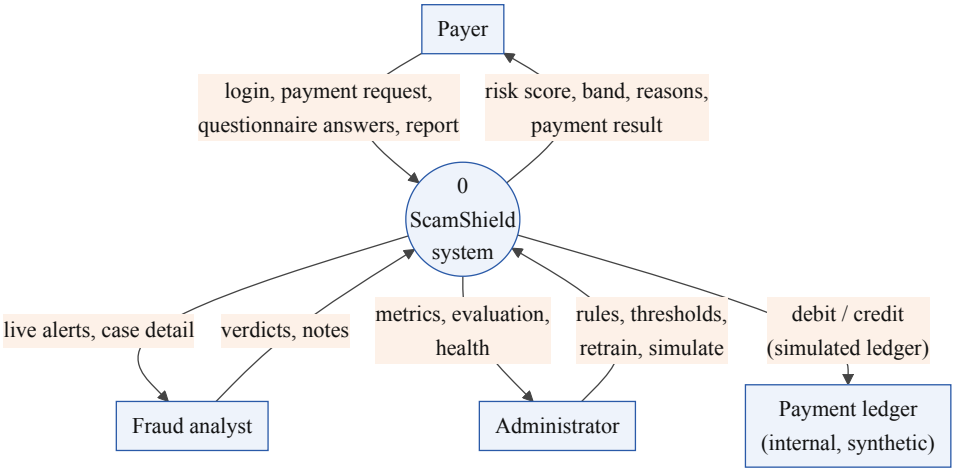


Figure 8.2: Level-0 data flow diagram (context diagram)

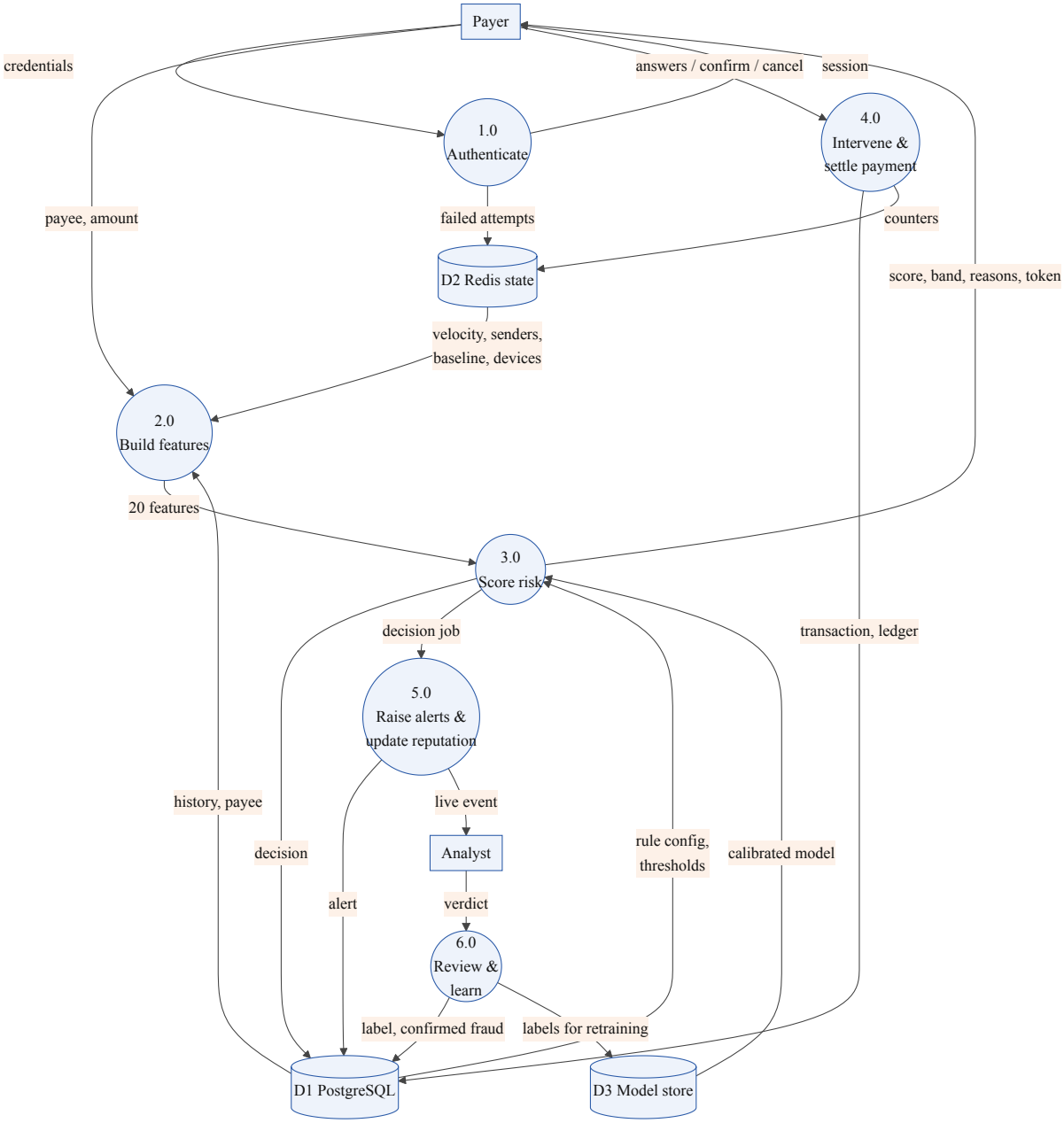


Figure 8.3: Level-1 data flow diagram

8.4 UML Diagrams

8.4.1 Use case diagram

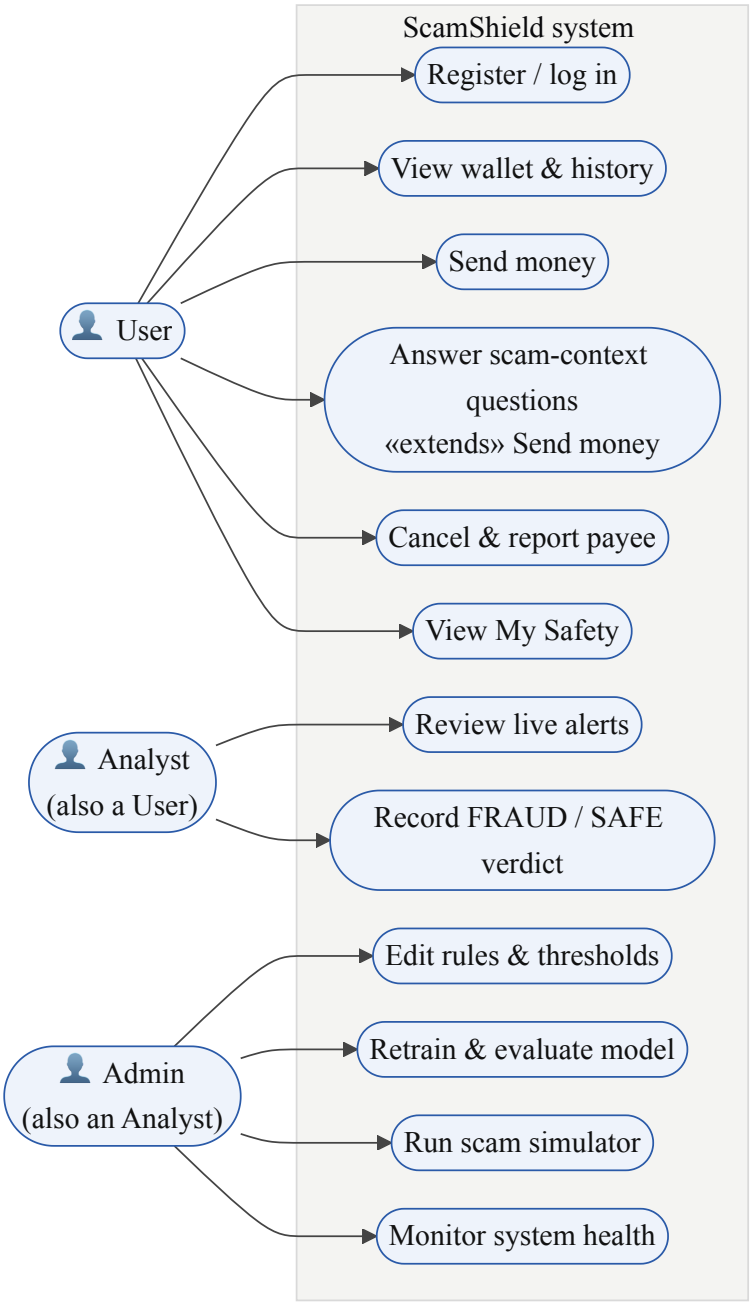


Figure 8.4: Use case diagram

8.4.2 Sequence diagram: checking and confirming a payment

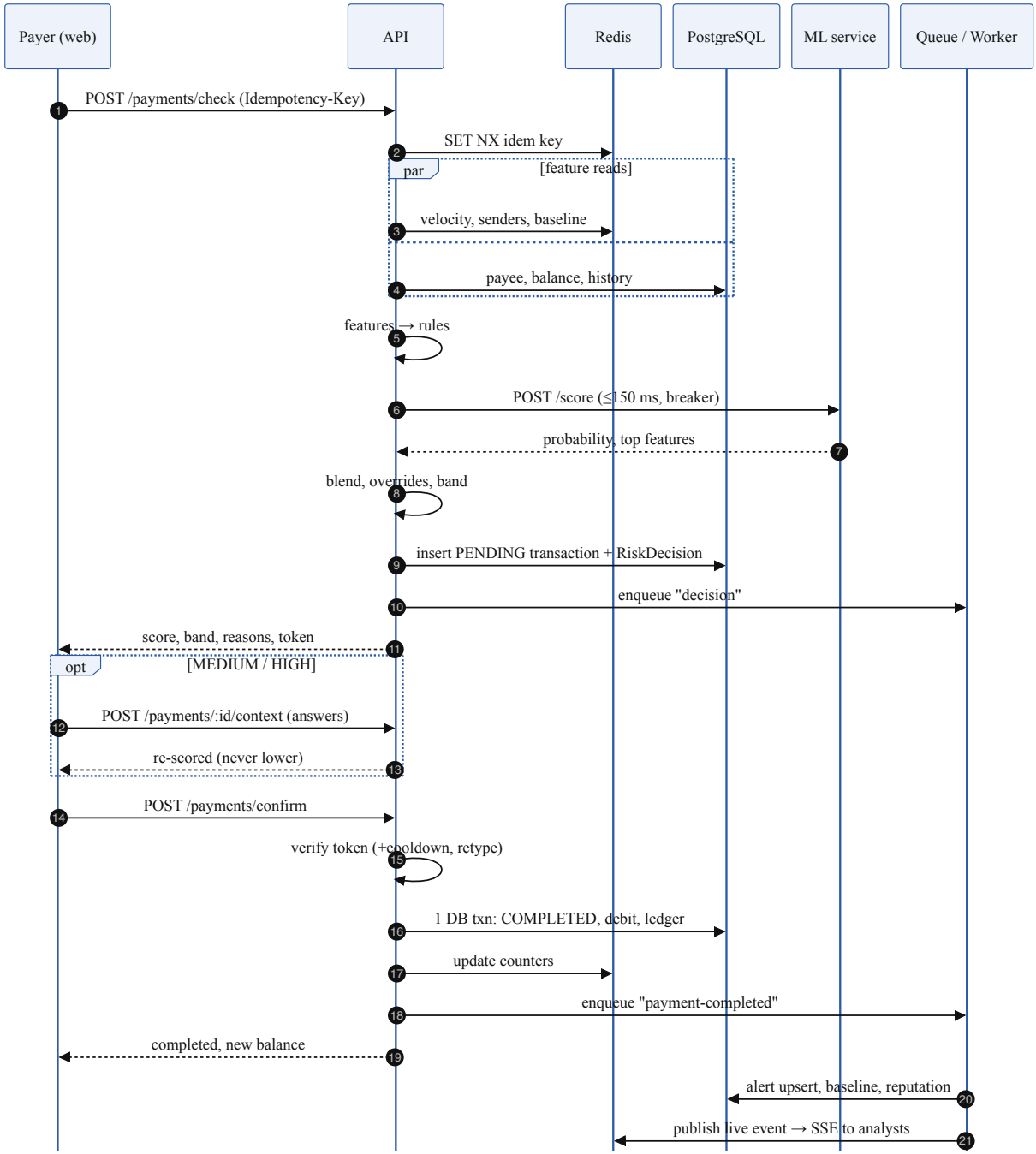


Figure 8.5: Sequence diagram of the two-phase payment

8.4.3 Class diagram (domain and services)

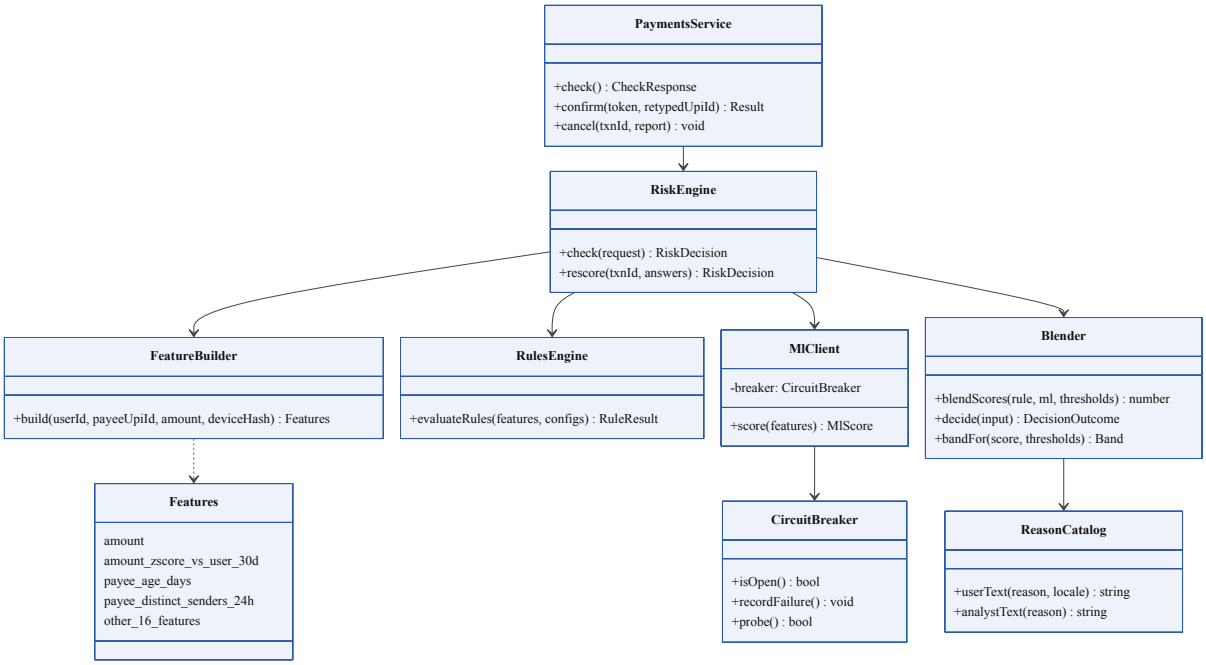


Figure 8.6: Class diagram of the risk-decision components

8.4.4 State diagram: transaction life cycle

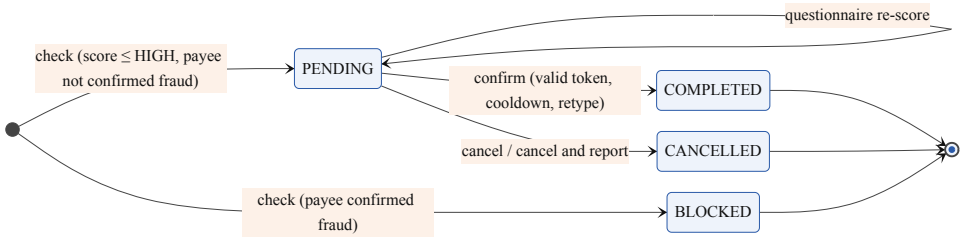


Figure 8.7: State diagram of a transaction

8.5 Database Schema

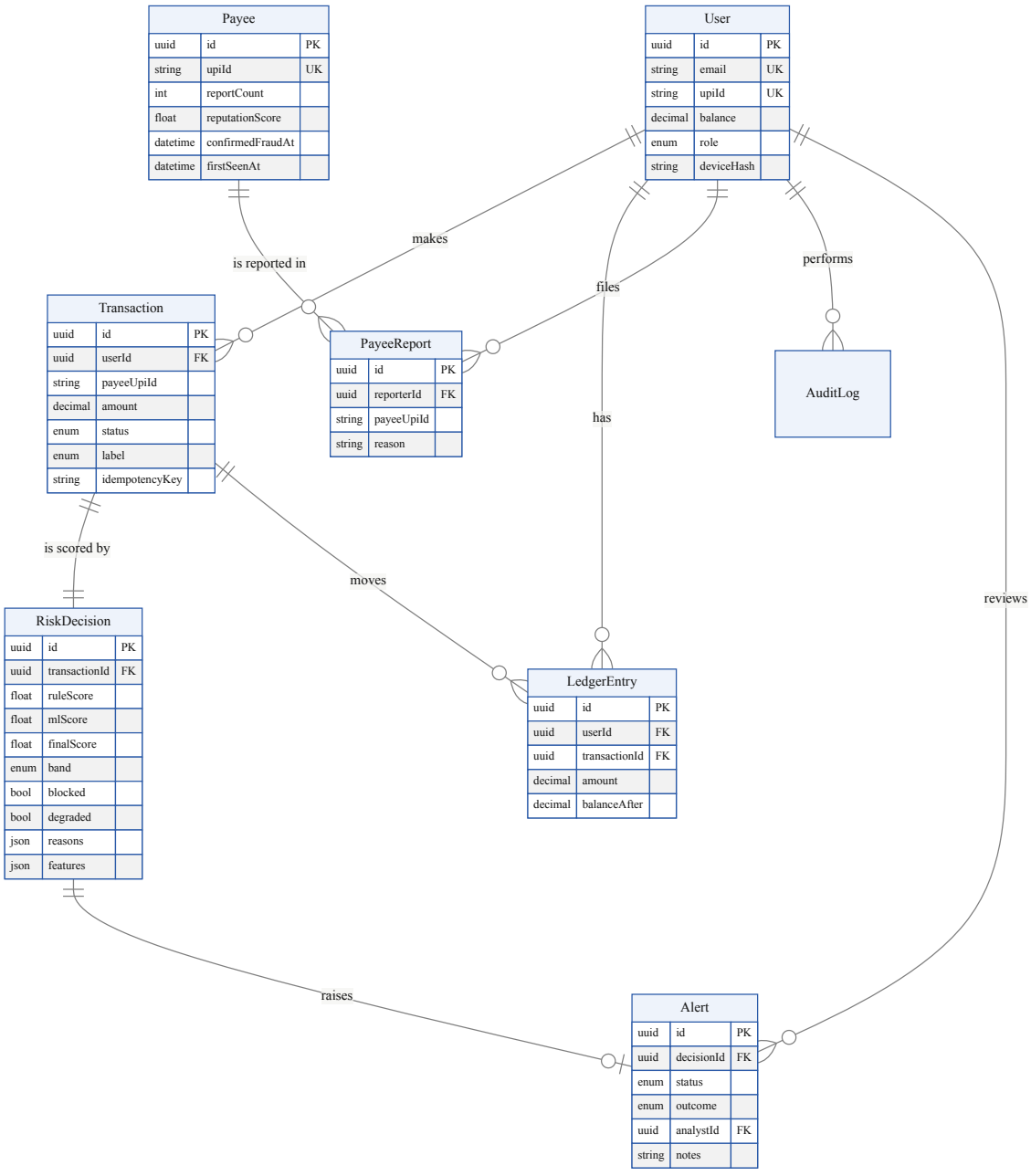


Figure 8.8: Entity–relationship diagram (configuration tables omitted)

Table 8.3: Database tables

Table	Purpose and key columns	Constraints / indexes
User	Account, role (USER/ANALYST/ADMIN), UPI ID, balance (Decimal 12,2), hashed registered device	unique email, unique upiId
Payee	Every UPI ID ever paid: first seen, report count, reputation 0–100, confirmedFraudAt	unique upiId

Transaction	Payment request: amount, payee, status (PENDING/BLOCKED/COMPLETED/CANCELLED), label (FRAUD/SAFE/UNKNOWN), device hash, IP	unique (userId, idempotencyKey); (userId, createdAt); (payeeUpiId, createdAt); (status)
RiskDecision	Rule, ML, final and pre-context scores; band; blocked; degraded; reasons, features, ML top features, context answers (JSON); model version; latency	unique transactionId; (band, createdAt)
Alert	Analyst work item for MEDIUM/HIGH decisions; status, outcome, notes	unique decisionId; (status, createdAt)
LedgerEntry	Double-entry record of each balance movement with resulting balance	(userId, createdAt)
PayeeReport	User report against a payee	unique (reporterId, payeeUpiId)
RuleConfig	Per rule: enabled, weight, parameters (JSON)	key = rule code
Setting	Runtime settings, e.g. blend weights and thresholds	key
ModelRun	Registered model versions with metrics; one active	unique version
AuditLog	Actor, action, entity, metadata for administrative actions	(createdAt)

In addition, Redis holds the hot state listed in Appendix C (Table C.1): sorted sets for 10-minute and 1-hour velocity windows and 24-hour distinct senders per payee, hashes for 30-day baselines, sets of known payees and devices, idempotency records (24 h), rate-limit counters and circuit-breaker keys.

8.6 APIs and Integrations

The API is documented with OpenAPI 3 and served interactively at `/docs`. Errors share one shape: `{"error": {"code", "message", "details"}}`.

Table 8.4: Principal API endpoints

Method	Path	Role	Purpose
POST	/api/auth/register, /login, /logout	public	Create wallet; sign in (rate-limited); sign out
GET	/api/wallet, /api/transactions, /api/safety	user	Balance and spending; history with risk band; warnings and reports
GET	/api/payees/lookup?upi=	user	Payee name, age, reports, confirmed-fraud flag, paid-before
POST	/api/payments/check	user	**Phase 1:** score a payment (idempotent); returns score, band, reasons, checkToken
POST	/api/payments/:id/context	user	Questionnaire answers → re-score (once)
POST	/api/payments/confirm	user	**Phase 2:** move money with checkToken (+ retyped UPI ID for HIGH)
POST	/api/payments/:id/cancel	user	Cancel, optionally report payee

GET	/api/alerts, /api/alerts/:id, /api/alerts/stream	analyst	Queue, detail, Server-Sent Events stream
POST	/api/alerts/:id/review	analyst	FRAUD / SAFE verdict with notes
GET/PUT	/api/admin/rules, /api/admin/thresholds	admin	Rule configuration; blend weights and bands
POST	/api/admin/rules/what-if	admin	Preview a draft configuration on labelled holdout sample
GET/POST	/api/admin/model, /model/retrain, /evaluation	admin	Active model; queue retraining; evaluation
POST	/api/admin/simulate/:scenario	admin	Run a scam scenario through the real pipeline
GET	/api/admin/metrics; /health; /ready	admin / public	Metrics; liveness; readiness
POST	ML: /score, /train, /reload	internal	Score 20 named features; retrain; reload model

Confirm status codes. 200 completed; 401 invalid or tampered check token; 403 payee is a confirmed scam account; 409 already processed (double tap or replay); 422 retyped UPI ID did not match (also counted as a failed attempt); 425 safety pause still running; 400 insufficient balance.

Integrations in a real deployment. ScamShield would sit inside a PSP's or bank's UPI app back end: the app calls `check` before invoking the UPI collect/pay flow, and `confirm` is replaced by the institution's own UPI pay call guarded by the check token. Optional external signals: DoT FRI for the payee's mobile number, the I4C Suspect Registry, and the bank's own FRM case system.

8.7 Algorithms and Business Rules

8.7.1 Feature computation

Twenty features are computed from a common `FeatureInput` by identical TypeScript and Python implementations driven by a shared JSON specification; 300 randomised parity cases guarantee that training and serving compute exactly the same values. Key definitions:

- **Amount z-score** against the user's 30-day completed payments: $z = (a - \mu) / \max(\sigma, 0.1\mu, 50)$, clamped to [−5, 20], and 0 with fewer than three past payments. The floor on σ prevents users with very regular payments from producing explosive z-scores.
- **Usual hour** is a circular mean: the mean angle of $2\pi h/24$ over past payments, so that 23:00 and 01:00 average to midnight rather than noon. The deviation is the circular distance in hours.
- **Velocity** counts completed payments in the previous 10 minutes and 1 hour from Redis sorted sets; **split payments** counts payments to the same payee in the last hour within 20 % of the amount.

- **Mule signal**: distinct other users who paid the payee in the last 24 hours; **payee age**: days since the payee was first seen.
- **Context**: three binary questionnaire features (0 until answered).

8.7.2 Rules

Table 8.5: Default rules (weights and parameters are editable at runtime)

Rule	Condition	Points
NEW_PAYEE	User has never completed a payment to this UPI ID	15
AMOUNT_SPIKE	z-score ≥ 3 or amount ≥ 2 × 30-day maximum	25
DRAIN_BALANCE	Amount ≥ 80 % of balance	20
VELOCITY_BURST	≥ 3 payments in 10 min or ≥ 6 in 1 h	25
SPLIT_PAYMENTS	≥ 2 similar amounts to the same payee in 1 h	20
ODD_HOUR	Night (23:00–05:59 IST) and ≥ 4 h from usual hour	10
MULE_PATTERN	≥ 5 distinct senders in 24 h and payee age ≤ 30 days	30
REPORTED_PAYEE	≥ 1 report (+5 per extra report)	25–40
NEW_DEVICE	Device never used for a completed payment by this user	15
REPEATED_FAILED_ATTEMPTS	≥ 3 failed logins / verifications in 15 min	20
CONTEXT_PRESSURE	Urged to pay (25) + asked for OTP / app (35) + unknown recipient (10)	0–70

$$RuleScore = min(100, \Sigma_{enabled\ rules}\ weight_r \times points_r)$$

8.7.3 Machine-learning model

The model is scikit-learn's `HistGradientBoostingClassifier` — a histogram-based gradient-boosted tree ensemble [57] — with 300 iterations, learning rate 0.08, 31 leaves per tree, L2 regularisation 1.0 and balanced class weights. Class weighting and boosting distort probabilities, so the raw scores are calibrated with **isotonic regression** [58, 59] fitted on a separate, later time slice. The calibrated probability × 100 is the ML score. A class-weighted logistic regression is trained as a sanity-check baseline.

Explanations by occlusion. For each decision the service builds a batch of 21 rows — the original feature vector and 20 copies in which one feature is reset to the value of a typical legitimate payment — and predicts them in one call. The drop in fraud probability for each feature is its contribution; the top three (if above 0.5 percentage points) are returned to analysts. This is model-agnostic and costs one batched prediction.

8.7.4 Blending, bands and overrides

$$FinalScore = clamp_{[0,100]}(\ w_R \cdot RuleScore + w_M \cdot MLScore\ ),\ w_R = 0.4,\ w_M = 0.6$$

- If the ML service is unavailable, FinalScore = RuleScore and the decision is marked *degraded*.
- Bands: LOW if FinalScore < 35; MEDIUM if 35 ≤ FinalScore < 70; HIGH if FinalScore ≥ 70.
- Override 1 — payee confirmed as fraud by an analyst: FinalScore ← max(FinalScore, 95); payment BLOCKED.
- Override 2 — user reports being asked for an OTP or to install an app: FinalScore ← max(FinalScore, 70).
- Questionnaire re-score: $\text{FinalScore}_{\text{new}} \leftarrow \max(\text{FinalScore}_{\text{new}}, \text{FinalScore}_{\text{before}})$ — answers can never lower the score.
- Reasons are ordered: overrides, then rule hits by points, then "matches known scam patterns" if MLScore ≥ 50. Users see the top three.

8.7.5 Resilience and consistency algorithms

- **Circuit breaker** [60]: failures of the ML call are counted in Redis over a 30-second window; at five failures the breaker opens for 15 seconds (ML skipped). Afterwards a single probe request, elected with an atomic `SET NX` lock, closes the breaker on success or re-opens it on failure. Because the state lives in Redis, all API instances share it.
- **Idempotency** [61]: the Idempotency-Key is claimed with `SET NX` for 24 hours; a retry with the same key and body replays the stored response, a different body returns 422, and a concurrent duplicate returns 409. A unique database index is the second line of defence.
- **Atomic confirm**: in one database transaction, `UPDATE transaction SET status='COMPLETED' WHERE id=? AND status='PENDING'` (exactly one concurrent confirm can win) and `UPDATE user SET balance = balance − amt WHERE id=? AND balance ≥ amt` (balance can never go negative), followed by ledger entries. Any failure rolls back everything.
- **Check token**: an HS256 JSON Web Token [62] valid for 10 minutes, binding user, transaction, amount and payee, so a client cannot skip or alter the risk check.

8.8 Synthetic Data Generation

Because real labelled fraud data is not available to students, we wrote a seeded generator (default seed 42) that simulates 150 users for 120 days (30-day burn-in, 90 days written out, ≈ 165,000 payments). Its design goal is *realism that makes detection hard*, including legitimate behaviour that looks suspicious.

Table 8.6: Synthetic population: user personas

Persona	Users	Payments/day	Typical amount	Active hours (IST)
Salaried	50	~9	₹260	7–10, 12–15, 18–23
Student	40	~7	₹120	10–24 and some after midnight
Shopkeeper	30	~25	₹900	8–22
Senior citizen	30	~3	₹400	7–19

Hard negatives include monthly rent of up to ₹25,000, two festival windows (one inside the holdout) with larger and more frequent payments and pop-up stalls collecting from many people, first purchases at new merchants (₹8,000–45,000), shopkeeper supplier batches, night-owl students, phone upgrades (new devices) and login typos. 1,500 merchants follow a Zipf popularity distribution, and 20 % open during the simulation.

Table 8.7: Synthetic fraud scenarios (≈ 2.4 % of payments overall)

Scenario	Story	Signals
Impersonation	Fake bank, KYC desk, police or buyer pressures the victim	New payee; amount 3–40× usual; evenings; follow-up “deposits”; pressure answers
Mule collection	One account collects a fixed fee from 10–28 victims in 6–36 hours	Many distinct senders; new or bought account
Velocity drain	Remote-access app empties the account	4–9 similar payments in minutes to 1–2 new payees; OTP/app answers
Account takeover	Stolen credentials	2–6 failed logins; new device; mostly at night; most of balance

Scam accounts are not all new: 30–40 % are 60–900 days old, as bought or rented accounts would be. Victims report the payee 10–50 % of the time, 1–48 hours later. Features are computed by replaying events in time order with no look-ahead, and fraud-labelled payments are excluded from baselines, exactly as in the live API. The data is split **by time**: first 60 % for training, next 15 % for calibration, last 25 % as the holdout.

CHAPTER 9

Implementation

The system is implemented as a single monorepo with npm workspaces for the TypeScript packages and a Python package for the ML service. The repository contains 134 TypeScript/TSX source and test files and 17 Python files. This chapter describes each module, the development process and the working product.

9.1 Module-by-Module Description

Table 9.1: Repository structure

Directory	Contents
`packages/shared`	Feature specification and computation, rule definitions and engine, blending and overrides, reason catalogue, zod schemas — used by the API, the web app and (via JSON spec) the ML service
`services/ml`	FastAPI scoring service; synthetic data generator; training, calibration and evaluation; occlusion explanations; parity fixture
`apps/api`	Express API and BullMQ worker; Prisma schema and migrations; seed data; services for features, risk, payments, alerts, admin, simulator, metrics
`apps/web`	Next.js App Router UI: landing, auth, wallet, send money, My Safety, analyst console, admin pages; Playwright end-to-end tests
`docs`, `scripts`, `infra`	Architecture, API, data and demo documentation; load-test and screenshot scripts; database initialisation

9.1.1 Shared risk core (packages/shared)

The heart of the system is a pure, side-effect-free library: `computeFeatures()` turns a `FeatureInput` into the 20 features; `evaluateRules()` applies the enabled rules with their weights; `decide()` blends rule and ML scores, applies overrides and orders reasons; and the reason catalogue renders each reason as a user sentence, a summary phrase and an analyst detail. Because the logic is pure, it is exhaustively unit-tested and identical in the API and the browser. Appendix D reproduces the decision function.

9.1.2 ML service (services/ml)

On first start the service runs the generator, trains the gradient-boosting model and a logistic-regression baseline, calibrates on the calibration slice, evaluates rules-only, ML-only and hybrid on the holdout, writes a versioned model directory (`model.joblib`, `metrics.json`, `evaluation.json`) and only then begins serving. `POST /score` validates that exactly the 20 named features are present, returns the calibrated probability, the 0–100 score, the model

version and the top contributing features. Retraining runs in the background with progress exposed at `/train/status`; analyst labels are added to the training data only, never to the holdout.

9.1.3 API (apps/api)

The **feature builder** gathers all inputs in one PostgreSQL round trip plus pipelined Redis reads executed in parallel, falling back to PostgreSQL to rebuild known-payee and device sets on a cache miss. The **risk engine** calls the shared core and the ML client (with timeout and circuit breaker), stores the decision with its latency and model version, and enqueues a job. The **payments service** implements idempotent check, questionnaire re-scoring, cancel-and-report, and the atomic confirm with check-token verification, cooldown and retype checks. **Middleware** provides session/bearer authentication, role checks and Redis rate limiting; every route validates input with zod and logs with a request ID.

9.1.4 Worker (apps/api/src/worker.ts)

Consumes two BullMQ queues: `risk-events` (create or escalate alerts for MEDIUM/HIGH and publish live events; refresh the payer's baseline after each completed payment; recompute payee reputation; 30-second metrics roll-up that doubles as the worker heartbeat) and `model` (sync the active model every 60 seconds; retrain on request). Jobs retry with exponential back-off; enqueue failures are logged and never block payments.

9.1.5 Web application (apps/web)

- **User:** landing page with one-click demo accounts; wallet dashboard (balance, 30-day spending chart, recent transactions); *Send money* with recipient card, risk meter, reasons, questionnaire and the MEDIUM/HIGH interventions; *My Safety* (warnings received, payees reported).
- **Analyst:** live alert queue with filters and a “Live” indicator (SSE); alert detail with score breakdown, reasons, ML top features, features table, payee panel and review form.
- **Admin:** Scam Simulator; Rules & Thresholds with what-if preview; Model & Evaluation (comparison table, PR curves, threshold sweep, calibration, feature importance, confusion matrices, per-scenario table, retraining); System Health.

9.2 Development Methodology

We followed an incremental, Agile-inspired process organised as seven phases (P0–P6), each ending with a working, tested increment and a commit. Before any coding, we fixed a written **definition of done** that included an honest-evaluation criterion: the hybrid must beat rules-only on recall at a matched false-positive rate, and if it did not, the report would say so. Phases were sized so that the riskiest parts — data, model and evaluation — came first.

Table 9.2: Development phases and deliverables

Phase	Goal	Deliverables
P0	Scaffold	Monorepo, Docker Compose (6 services), Prisma schema and migrations, health endpoints
P1	Data and model	Synthetic generator, training with calibration, evaluation harness, FastAPI scoring service, Node/Python parity tests
P2 + P3	API core and risk pipeline	Auth and RBAC, feature builder, rules + ML + blend, two-phase payments, idempotency, circuit breaker, worker jobs
P4 + P5	Web application	Wallet, send money with LOW/MEDIUM/HIGH interventions, My Safety, analyst console with SSE, simulator, rules what-if, model evaluation, system health
P6	Hardening and documentation	Worker-job tests, Playwright end-to-end tests, load test, screenshots, architecture/API/data/demo documentation

The core build (P0–P6) was completed in an intensive one-week sprint beginning 14 October 2026, followed by load testing, a data-leak investigation, demo rehearsal and the preparation of this report. Figure 9.1 shows the timeline.

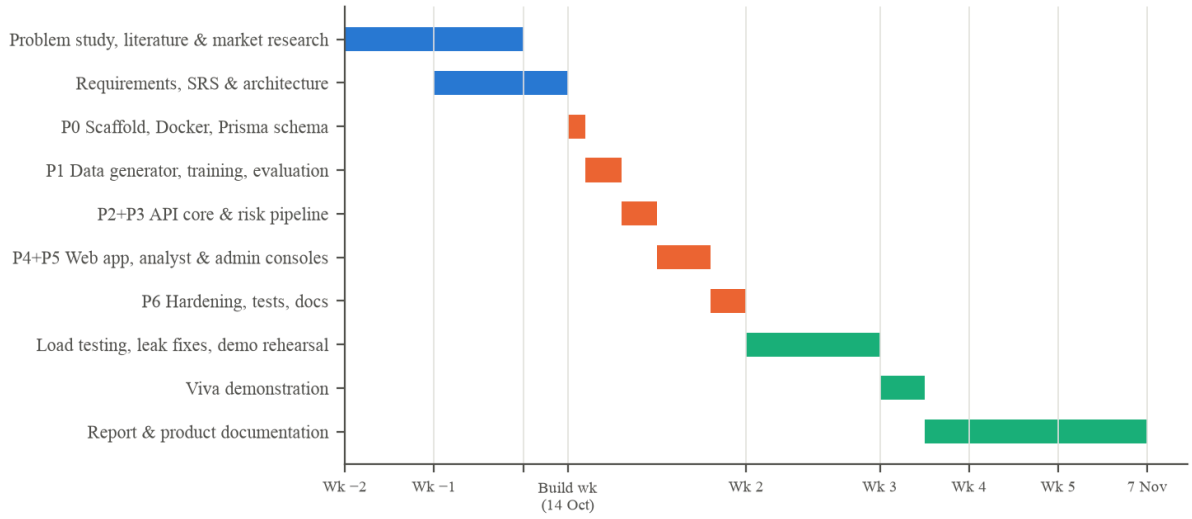


Figure 9.1: Project timeline (Gantt chart)

Daily practice: short stand-ups to agree the day's increment; trunk-based development with one commit per phase; tests written alongside code (integration tests against real PostgreSQL and Redis rather than mocks); every evaluation result checked against feature importances before being accepted (Section 11.1).

9.3 Team Roles and Responsibilities

TEAM TO COMPLETE — The allocation below is a proposal based on the modules of the system; the repository history does not record who did what. Replace it with the actual division of work before submission.

Table 9.3: Team roles and responsibilities

Member	Primary role	Responsibilities
Ansh (23BCS016)	ML and data lead	Synthetic data generator, training and calibration, evaluation harness, leak investigation, ML service
Harshit (23BCS054)	Back-end lead	API, feature builder, risk engine, payments consistency, worker, Redis design
Vijender (23BCS132)	Front-end and integration lead	Web app, interventions UX, analyst and admin consoles, Docker setup, end-to-end tests
Sourabh (23BEC055)	QA, research and compliance	User research and questionnaire, market analysis, legal and compliance review, load testing, test planning, documentation

9.4 Screenshots of the Working Product

All screenshots were captured from the running system with the seeded demo data. They are reproduced from the repository's `docs/screenshots` folder; tall pages are cropped.

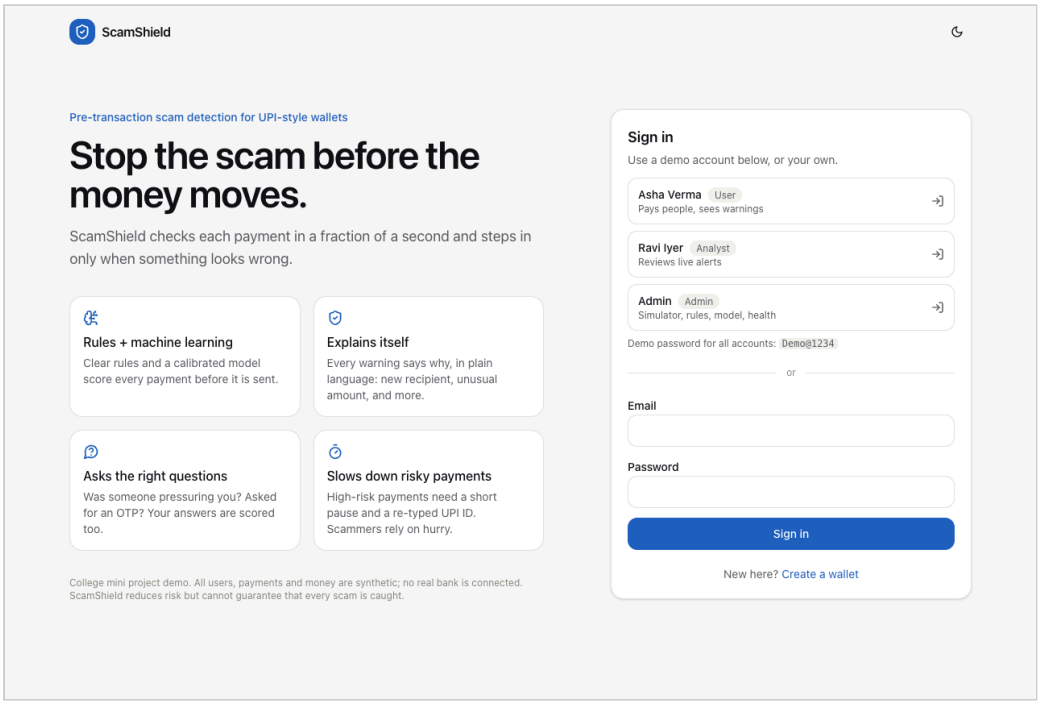


Figure 9.2: Landing page with one-click demo accounts

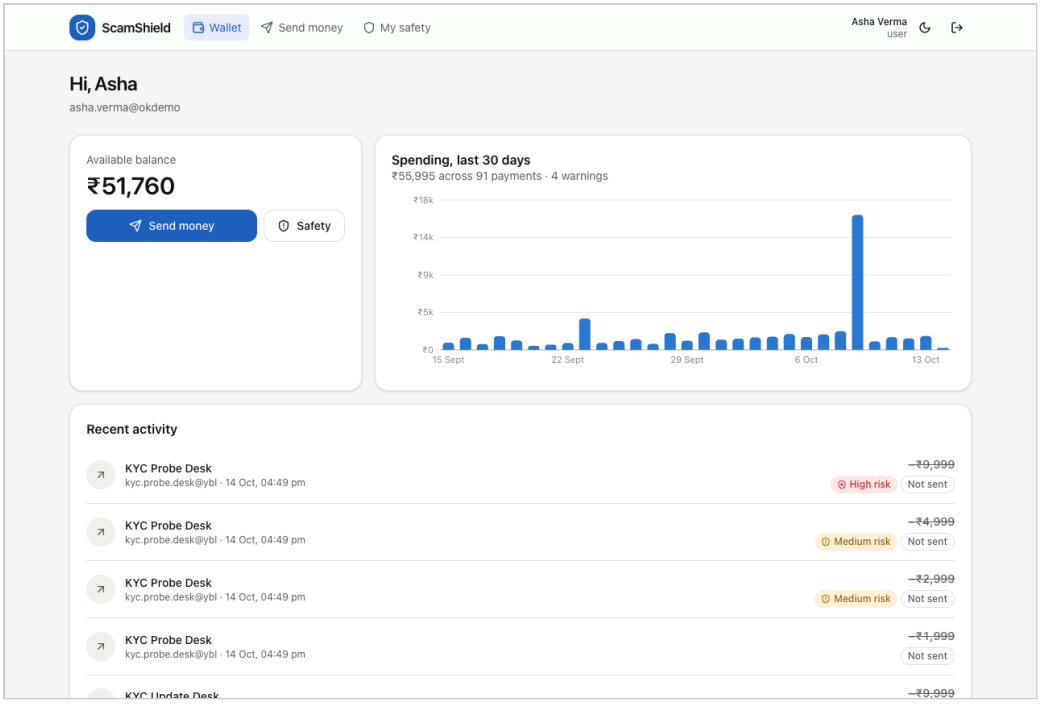


Figure 9.3: Wallet dashboard: balance, 30-day spending and recent transactions

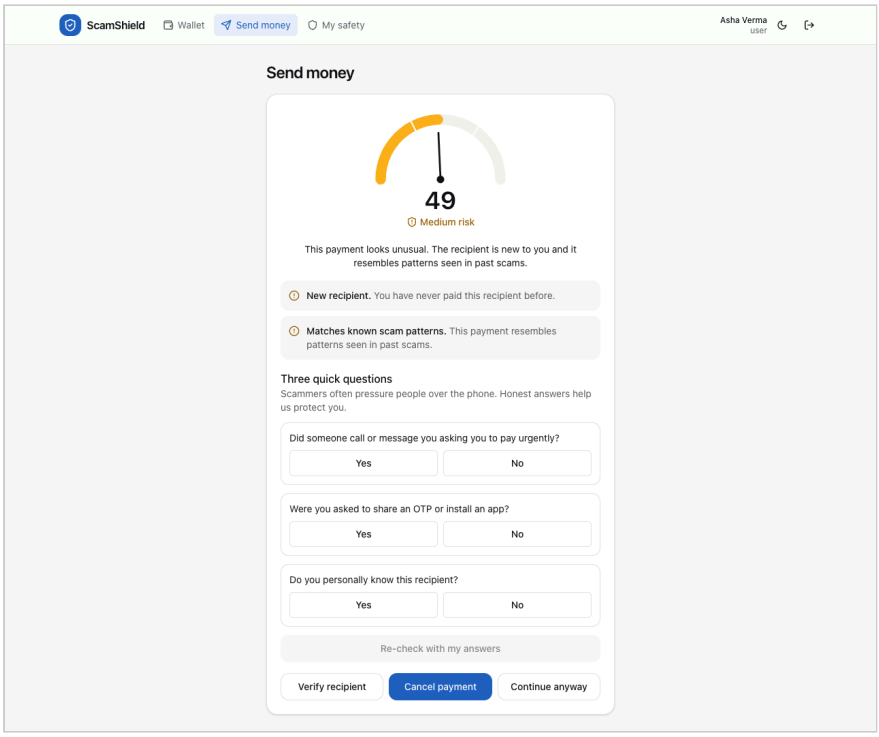


Figure 9.4: MEDIUM risk: score, reasons and the three scam-context questions

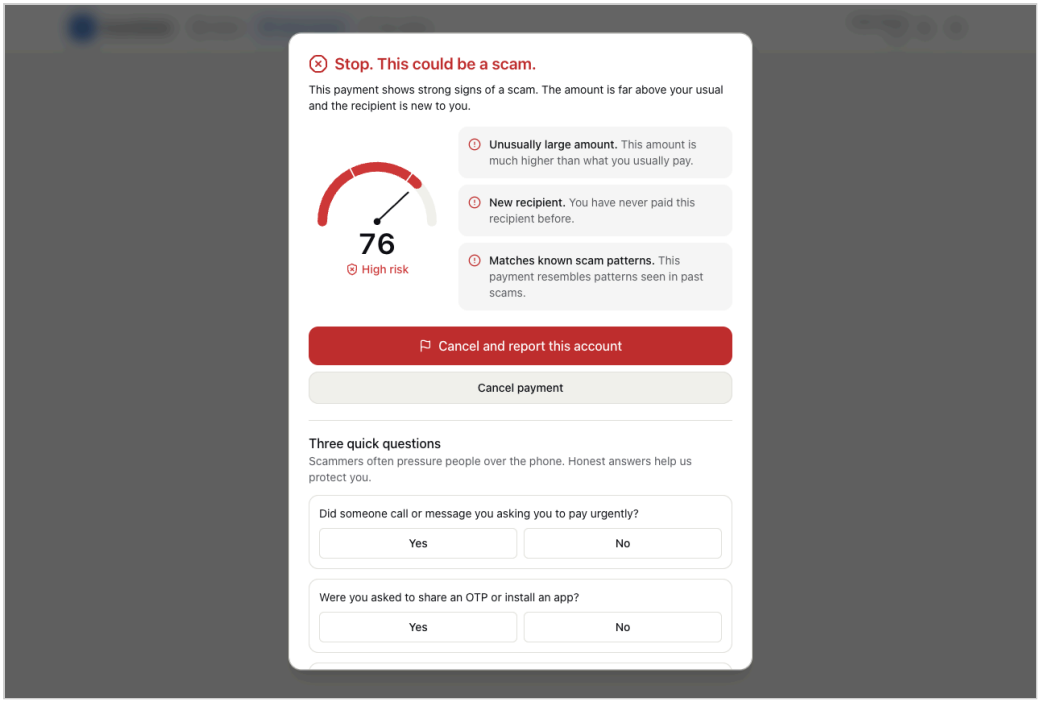


Figure 9.5: HIGH risk: blocking screen with “Cancel and report this account” as the primary action

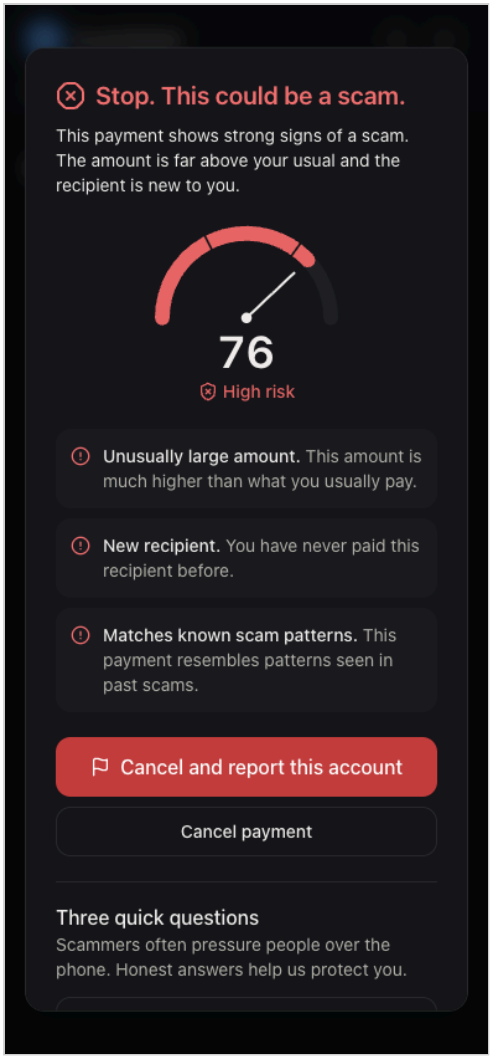


Figure 9.6: HIGH-risk screen on a 390 px mobile viewport in dark mode

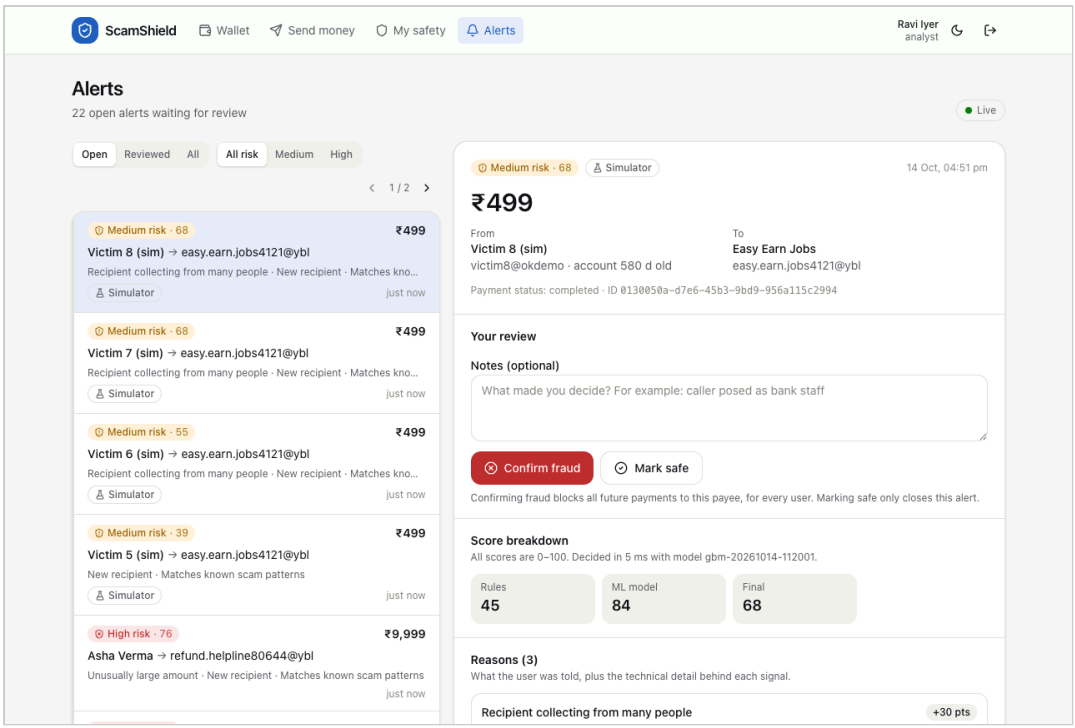


Figure 9.7: Analyst console: live alert queue (left) and alert detail with score breakdown and review form (right)

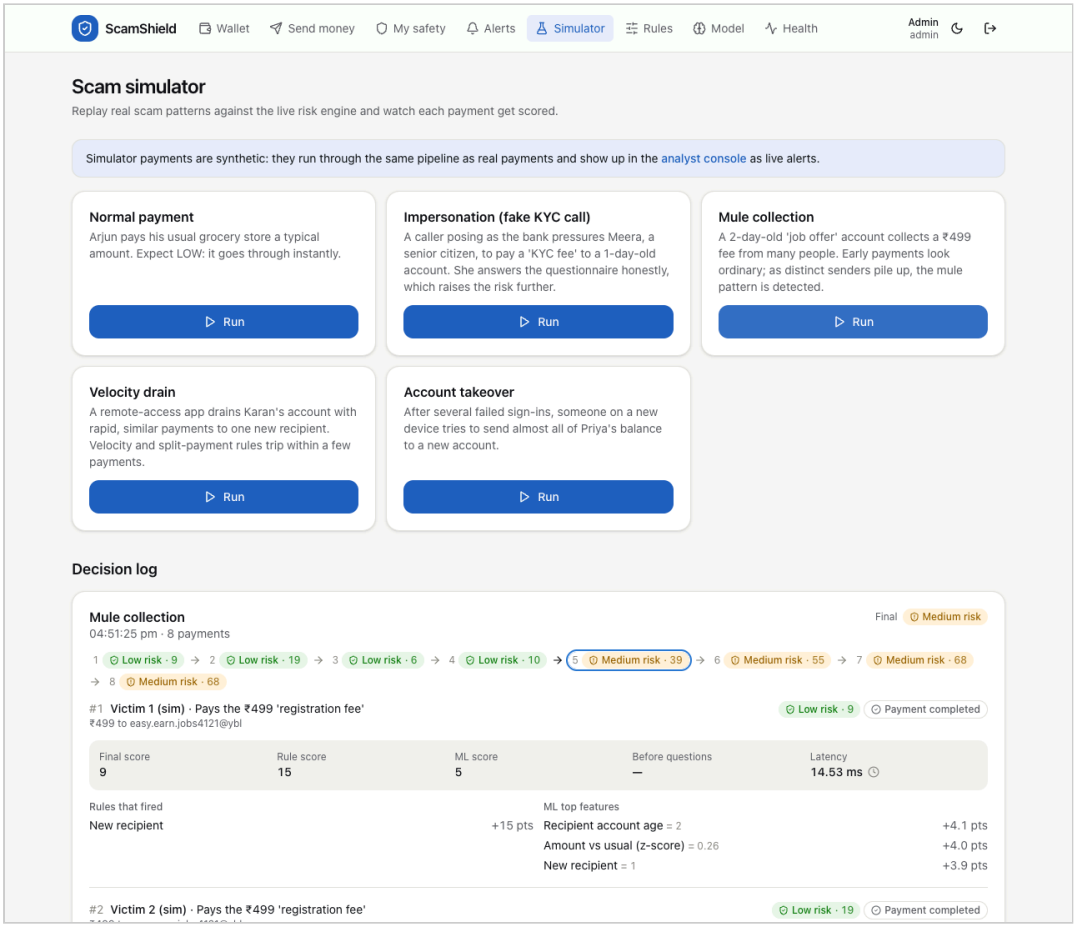


Figure 9.8: Scam Simulator: a mule-collection run escalating from LOW to MEDIUM as distinct senders accumulate

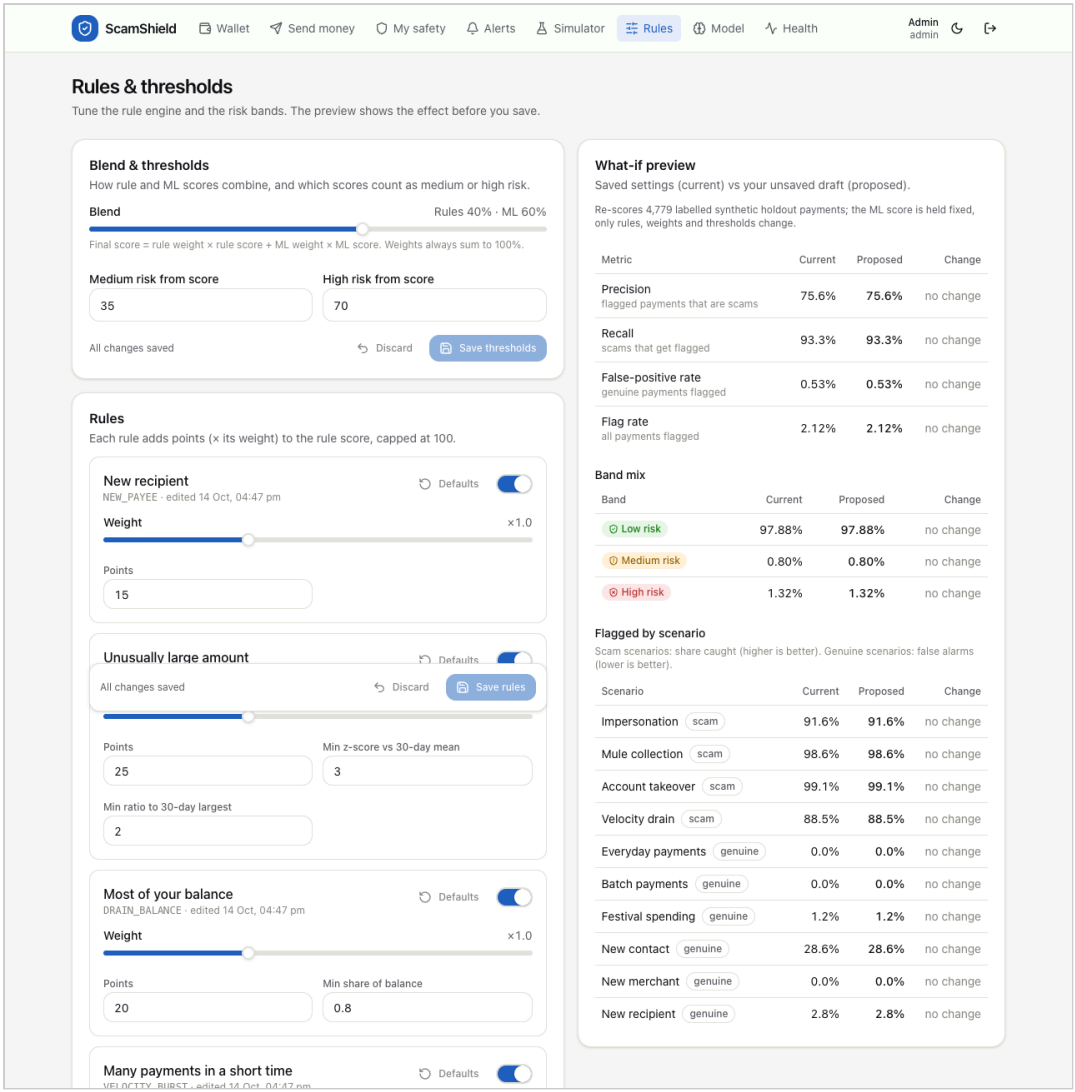


Figure 9.9: Rules and thresholds editor with what-if preview on labelled holdout data

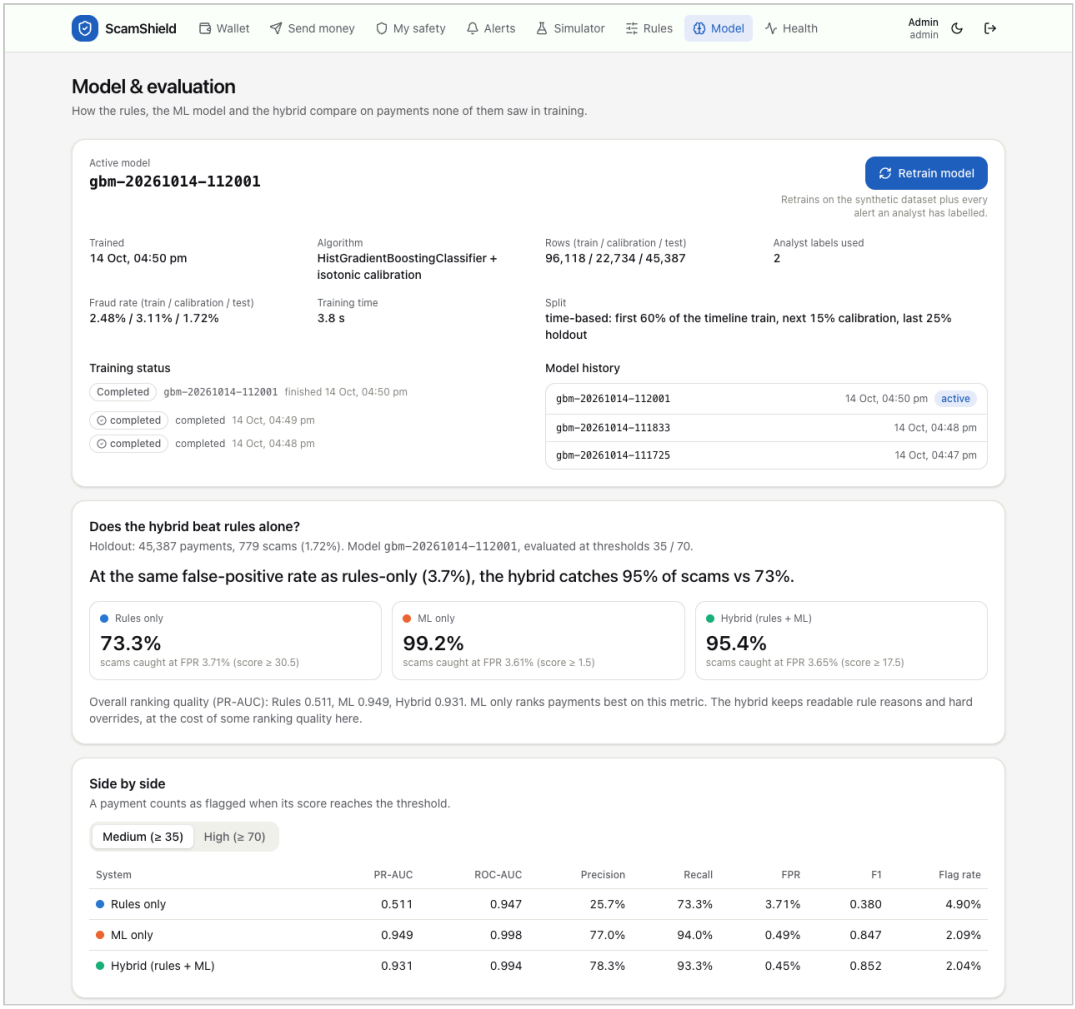


Figure 9.10: Model and evaluation page: active model, headline comparison and side-by-side metrics

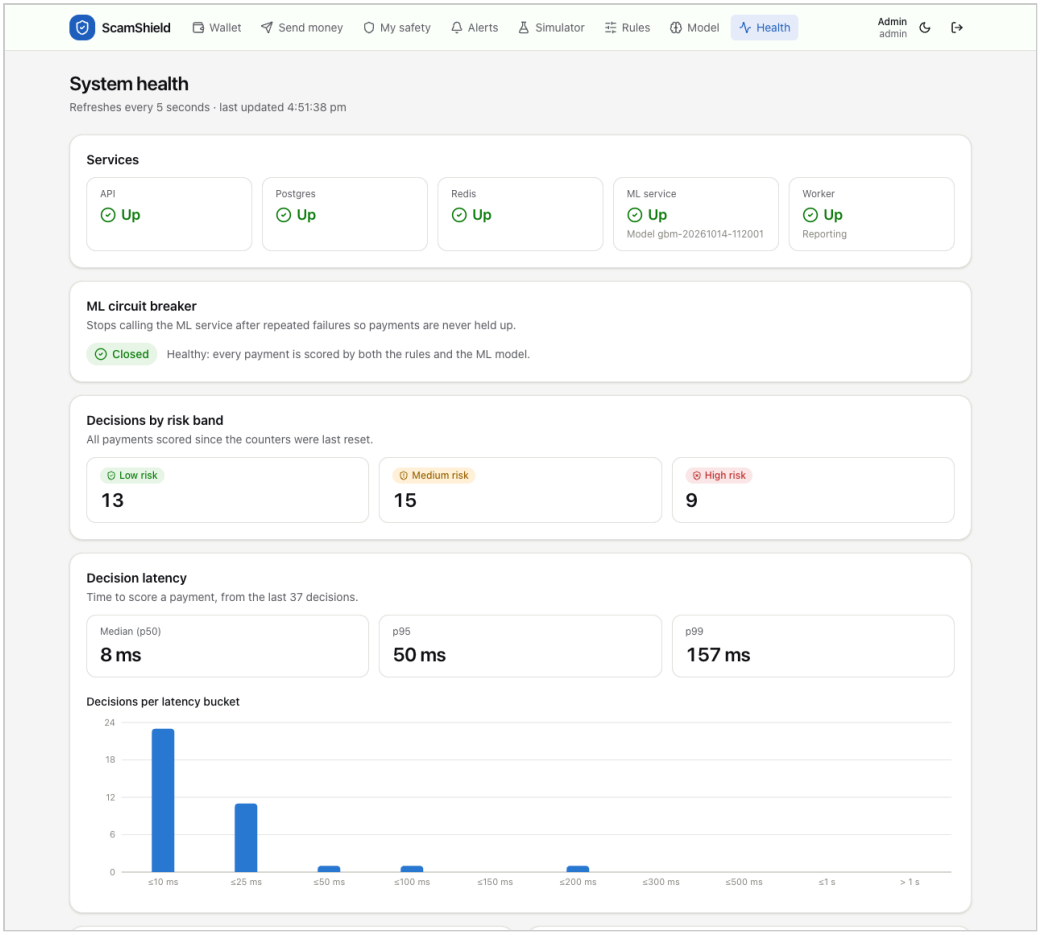


Figure 9.11: System health: service status, circuit breaker, decisions by band and latency

> The screenshots of the Model, Rules and Health pages were captured after an in-app retraining that included two analyst labels (model `gbm-20261014-112001`), so their figures differ slightly from the reference model (`gbm-20261014-074524`) whose evaluation is reported in Chapter 10.

CHAPTER 10

Testing and Results

10.1 Testing Strategy

Testing was organised in five layers, from pure logic to the full running system. Integration tests run against real PostgreSQL and Redis instances with a stubbed ML service, so that concurrency, idempotency and circuit-breaker behaviour are tested as they actually occur.

Table 10.1: Test layers, tools and results

Layer	Tool	Scope	Tests
Unit (risk core)	Vitest	Feature computation, rules, blending, bands, overrides, reason text	9
Parity (Node ↔ Python)	Vitest + pytest	300 random inputs: every feature and rule score identical in both languages	300 + 3
Integration (API)	Vitest + supertest	Auth and RBAC, feature builder, two-phase payments, idempotency, concurrency, tokens, cooldown, questionnaire, blocking, reports, ML fallback, circuit breaker, worker jobs	24
ML service contract	pytest	Health, scoring contract and ordering, input validation, p95 < 50 ms, evaluation shape	5
End-to-end	Playwright	Happy path (LOW goes through); scam path (HIGH, cooldown gate, cancel and report)	2
Performance	autocannon	`POST /payments/check` at 20 req/s for 60 s	1 run

Execution for this report. On 7 November 2026 we re-ran the unit, parity, integration and ML suites on a fresh clone of the repository: **341 of 341 tests passed** (309 in the shared package, 24 API integration tests against temporary PostgreSQL 16 and Redis instances, and 8 ML-service tests). The two Playwright tests require the full Docker stack and were last run during phase P6; the load test was run on 14 October 2026.

10.2 Test Cases

Table 10.2: Representative test cases

ID	Test case	Steps / input	Expected result	Result
TC-01	Normal payment to known shop	Asha pays `kirana.sharma@okaxis` ₹350	LOW; no prompt; completed; ledger debit	Pass

TC-02	Impersonation payment explained	₹9,600 to a 1-day-old new payee; ML score 60	AMOUNT_SPIKE + NEW_PAYEE; final 52, MEDIUM; summary "…8x your largest recent payment and the recipient is new to you."	Pass
TC-03	Questionnaire can only raise score	Answer "no" to all questions on a MEDIUM payment	Final score ≥ pre-context score	Pass
TC-04	OTP override	Answer "asked to share an OTP" = yes	Band at least HIGH	Pass
TC-05	HIGH requires cooldown	Confirm a HIGH payment immediately	425 COOLDOWN_ACTIVE	Pass
TC-06	HIGH requires exact retype	Confirm after cooldown with wrong UPI ID	422 VERIFICATION_FAILED; failed-attempt counter +1	Pass
TC-07	Confirmed scam payee blocked	Analyst marks FRAUD; another user pays same payee	Status BLOCKED; confirm returns 403	Pass
TC-08	Idempotent check	Repeat check with same key and body; then different body	Replayed response; 422; only one row created	Pass
TC-09	No double spending	Five concurrent confirms of one payment	Exactly one 200; balance exact	Pass
TC-10	Tampered check token	Modify token payload / use another user's token	401 INVALID_CHECK_TOKEN	Pass
TC-11	Insufficient balance	Amount above balance	400; no state change	Pass
TC-12	ML failure fallback	ML stub returns error	Rules-only decision, `degraded: true`	Pass
TC-13	ML timeout	ML stub sleeps beyond 150 ms	Decision returned within budget, degraded	Pass
TC-14	Circuit breaker cycle	Five failures, then healthy ML	Opens, skips ML, half-open probe, closes	Pass
TC-15	Baseline excludes fraud	Label a large payment FRAUD; compute baseline	Fraud amount not in mean/max	Pass
TC-16	Distinct-senders count	Several users pay one payee	Counts distinct others in 24 h	Pass
TC-17	Report once per user	Same user reports a payee twice	Report count increments once	Pass
TC-18	RBAC	User calls analyst/admin endpoints	403; analyst cannot call admin	Pass
TC-19	Alert creation	MEDIUM decision job; LOW decision job	One alert + live event; none for LOW	Pass

TC-20	Scoring contract	ML `/score` with missing / unknown feature	422	Pass
TC-21	Scoring latency	Sequential single-row requests	p95 < 50 ms	Pass
TC-22	Feature parity	300 random `FeatureInputs`	TypeScript = Python for all features and rule scores	Pass
TC-23	E2E happy path	Browser: log in as Asha, pay known shop	Payment completes without a prompt	Pass (P6)
TC-24	E2E scam path	Browser: pay fake KYC desk	HIGH screen, cooldown gate, cancel and report	Pass (P6)

10.3 Model Evaluation Results

10.3.1 Evaluation protocol

The reference model (`gbm-20261014-074524`) was trained on 96,118 payments (fraud rate 2.48 %), calibrated on 22,734 (3.11 %), and evaluated on a time-based holdout of the last 25 % of the timeline: **45,387 payments, 779 scams (1.72 %)**, including a festival period never seen in training. Because the classes are highly imbalanced, we report precision, recall and false-positive rate at the operating thresholds and use PR-AUC rather than ROC-AUC as the primary ranking measure [63]. Three systems are compared: rules-only, ML-only and the hybrid blend.

Table 10.3: Holdout results at the MEDIUM threshold (score ≥ 35)

System	PR-AUC	ROC-AUC	Precision	Recall	FPR	F1	Flag rate
Rules only	0.511	0.947	25.7 %	73.3 %	3.71 %	0.380	4.90 %
ML only	**0.946**	0.998	72.7 %	**94.6 %**	0.62 %	0.822	2.23 %
Hybrid (rules + ML)	0.933	0.994	**79.1 %**	93.2 %	**0.43 %**	**0.856**	2.02 %

Table 10.4: Holdout results at the HIGH threshold (score ≥ 70)

System	Precision	Recall	FPR	F1	TP	FP
Rules only	74.5 %	33.0 %	0.197 %	0.457	257	88
ML only	91.2 %	89.9 %	0.152 %	0.905	700	68
Hybrid	96.0 %	76.0 %	0.056 %	0.848	592	25

10.3.2 The headline comparison: recall at matched false-positive rate

The definition of done required the hybrid to beat rules-only in recall at a comparable false-positive rate. At the FPR at which rules-only operates (3.71 %), the hybrid threshold that gives a comparable FPR (3.65 %, score ≥ 17.5) catches **95.6 %** of scams (745 of 779), against **73.3 %** (571 of 779) for rules alone — an absolute gain of 22.3 percentage points. **The criterion is met.**

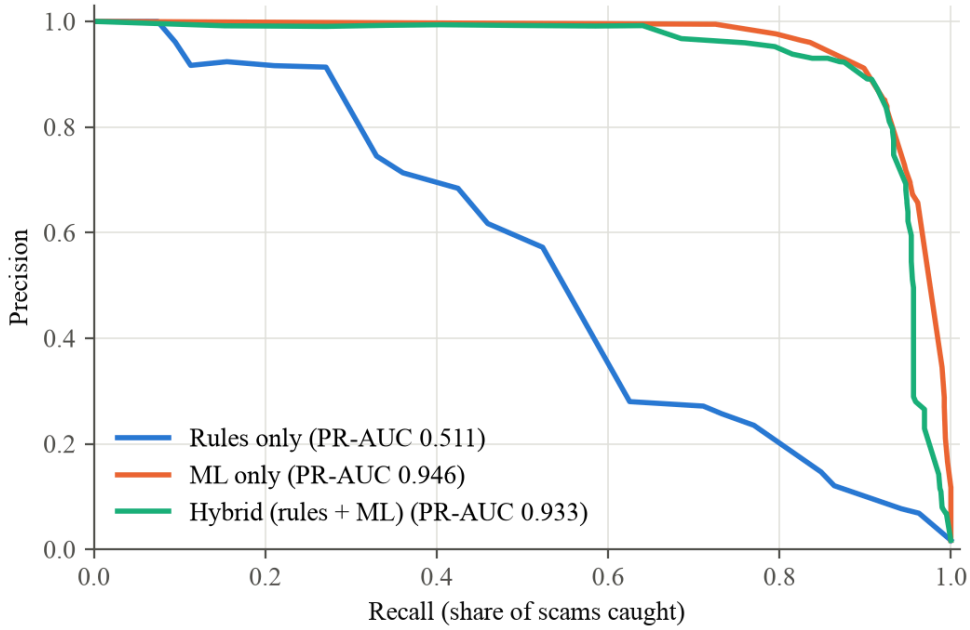


Figure 10.1: Precision–recall curves on the holdout

Honest finding. ML-only ranks payments slightly better than the hybrid (PR-AUC 0.946 vs 0.933) and has higher recall at the MEDIUM threshold. The rule component costs a little ranking quality. In exchange it provides human-readable reasons, carries the hard overrides, keeps working when the ML service is down, and — at our operating thresholds — gives the hybrid the best precision and the lowest false-positive rate (0.43 % at MEDIUM; 0.056 % at HIGH). For a system that interrupts real people, we judge that trade-off worthwhile, and we show it openly on the Model page.

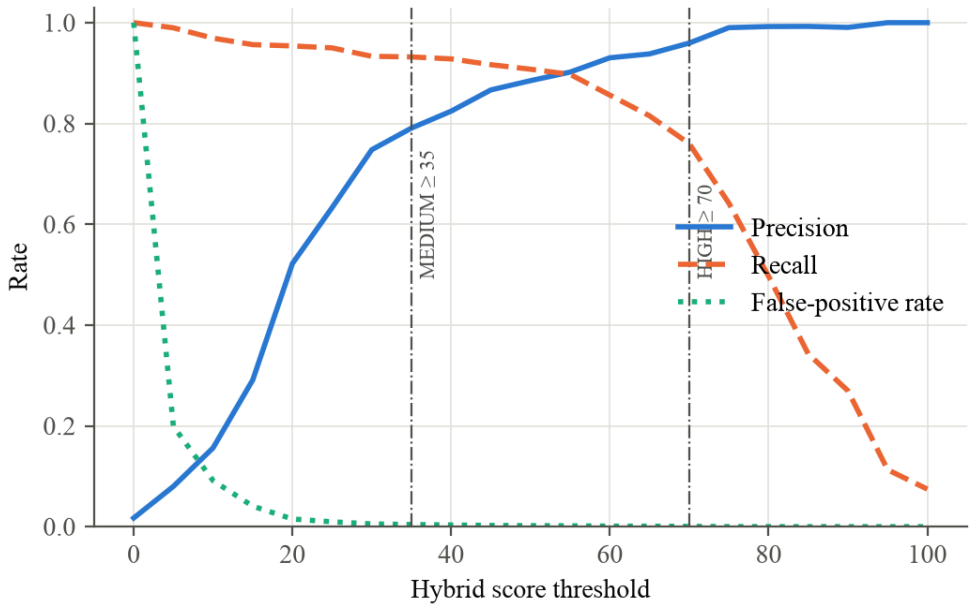


Figure 10.2: Hybrid threshold sweep: precision, recall and false-positive rate against score threshold

Figure 10.2 shows why the bands sit where they do: at 35, recall is still above 93 % while the false-positive rate has fallen below 0.5 %; at 70, precision reaches 96 %, which justifies the stronger HIGH intervention.

10.3.3 Results by scenario

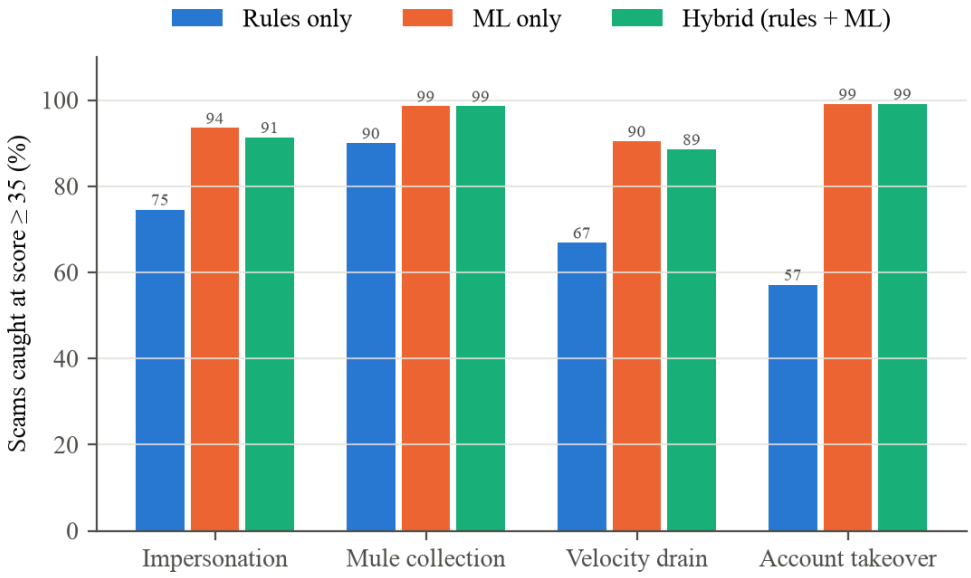


Figure 10.3: Share of scams caught at the MEDIUM threshold, by scenario



Figure 10.4: Share of genuine payments wrongly flagged at the MEDIUM threshold, by legitimate scenario

The hybrid catches 91 % of impersonation, 99 % of mule-collection, 89 % of velocity-drain and 99 % of account-takeover payments. Rules alone are weakest on account takeover (57 %), where the combination of signals matters more than any single threshold. On the false-alarm side, the hybrid flags 0.04 % of everyday payments, 1.0 % of festival payments and none of

the shopkeepers' supplier batches — which rules alone flag at 11.6 %. Its weakest areas are first payments to new contacts (21.7 % flagged, n = 60) and new merchants (31.3 %, n = 16): small groups, but exactly the legitimate behaviour that most resembles a scam, and a known limitation.

10.3.4 Calibration and feature importance

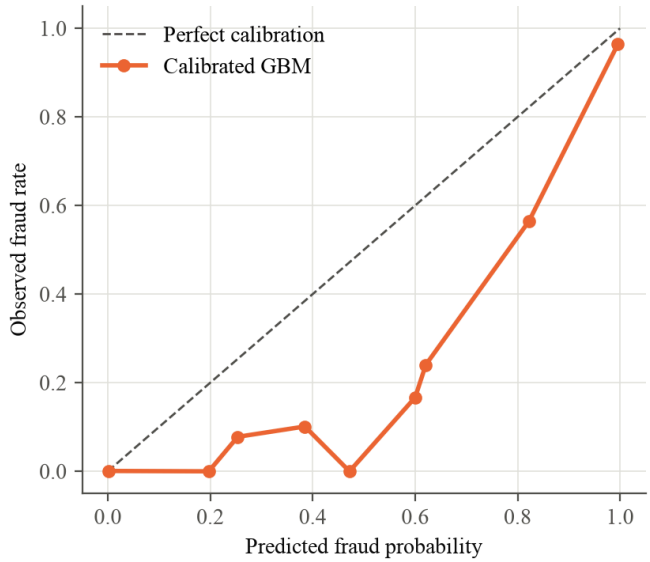


Figure 10.5: Calibration of the ML model on the holdout

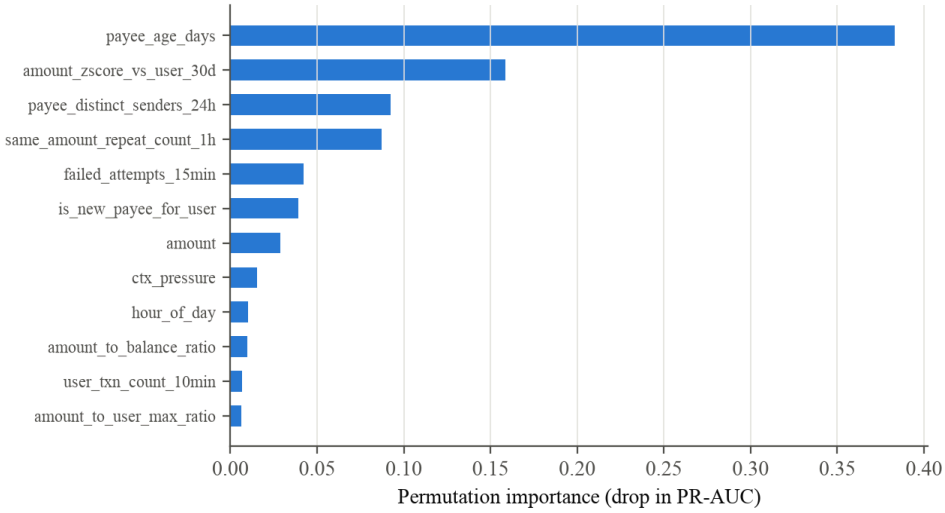


Figure 10.6: Permutation feature importance of the ML model (top 12 features)

Isotonic calibration improved the Brier score from 0.0046 to 0.0035 on the holdout. The calibration curve shows that the model is over-confident in the 30–60 % range on the holdout: the holdout contains a festival period that shifts behaviour, and those bins contain few payments. We show this rather than hide it. The most important features are payee age,

amount z-score, distinct senders to the payee in 24 hours and repeated similar amounts — signals that real fraud teams also use. Feature importances were inspected after every change to the generator; this is how two data leaks were discovered (Section 11.1).

10.4 Performance Results

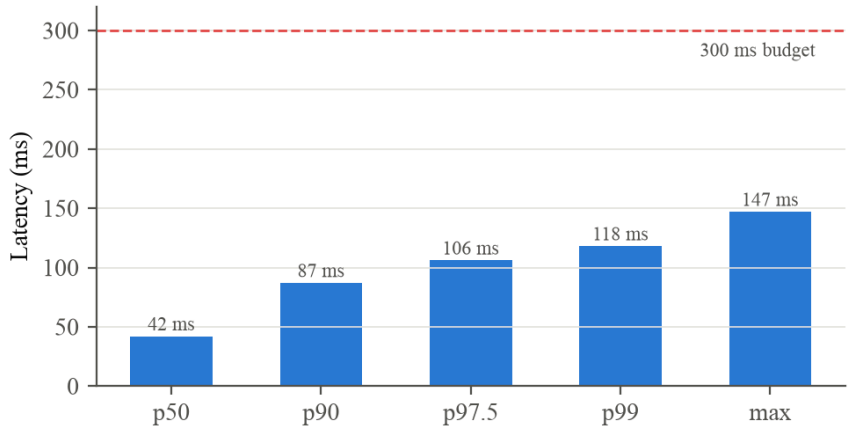


Figure 10.7: Latency of POST /payments/check under load (1,200 requests at 20 req/s, client-side)

Under a 60-second load of 20 requests per second over 10 connections, every request ran the full hybrid pipeline with zero errors and zero ML fallbacks. Client-measured latency was p50 42 ms, p90 87 ms, p99 118 ms, maximum 147 ms; the API's own metrics reported p95 109 ms — well within the 300 ms budget (NFR-1). The first load test exposed a bottleneck: scikit-learn's OpenMP thread pool made single-row predictions slow under concurrency (47 req/s, p50 205 ms), which tripped the circuit breaker. Setting `OMP_NUM_THREADS=1` raised ML throughput to 154 req/s (p50 61 ms) and the breaker stayed closed. The tested rate of 20 req/s reflects the per-user rate limit and the 12 test accounts, not a measured maximum.

10.5 User Feedback and Acceptance Testing

User acceptance testing was planned around the four personas with the following script: (1) make three everyday payments; (2) pay a “KYC desk” while a team member plays a pressuring caller; (3) pay a known contact a large amount (rent); (4) as an analyst, review the alerts created. Acceptance criteria were: everyday payments complete without any prompt; the participant understands, in their own words, why a payment was flagged; the participant cancels at least the coached scam payment; and the participant rates the HIGH friction as acceptable.

TEAM TO COMPLETE — Insert the results of user acceptance sessions and the viva demonstration feedback here: number of participants and their profile, task completion, whether they cancelled the coached scam payment, comments on the wording and friction, and any changes made as a result. Report only sessions that actually took place.

CHAPTER 11

Challenges and Limitations

11.1 Challenges Faced and How We Solved Them

Table 11.1: Development challenges and their resolution

Challenge	What happened	How we solved it
Results that were too good	The first generator produced PR-AUC ≈ 0.99. Feature importances showed the model had memorised exact scam amounts (scams used a fixed menu such as ₹4,999, while legitimate amounts never ended in 99) and that every scammer account was days old while almost every legitimate payee was years old.	We treated this as a bug, not a result. Legitimate and scam amounts now pass through the same price-point logic; legitimate young payees (new shops, friends joining, festival stalls) and older bought scam accounts were added. PR-AUC fell to a believable ≈ 0.95.
Polluted baselines	Each simulated user was hit by many scams, and scam amounts entered the user's baseline, so a normal ₹500 payment later looked extreme.	Fraud-labelled payments are excluded from baselines in both the generator and the API, and drained balances are restored after 1–3 days.
Training/serving skew	Features are computed in Python for training and in TypeScript for serving; tiny differences (rounding, hour boundaries in IST) would silently corrupt the model's inputs.	A shared JSON specification, mirrored JavaScript rounding in Python, and 300 committed parity cases that the TypeScript tests must reproduce exactly.
ML latency under load	Single-row predictions were slow under concurrency because of OpenMP thread start-up, tripping the circuit breaker in the first load test.	`OMP_NUM_THREADS=1`: ML throughput rose from 47 to 154 req/s; the breaker stayed closed. The fallback worked as designed even before the fix.
Double spending and replays	Double taps, network retries and concurrent confirms could debit twice.	Idempotency keys with SET NX; check tokens bound to amount and payee; conditional SQL updates inside one transaction; tested with five concurrent confirms.
Questionnaire abuse	A coached victim could answer “no” to everything to make a warning disappear.	Answers can only raise the score; the OTP answer is a hard override.
Explaining a tree ensemble	SHAP would have added a heavy dependency and latency.	Occlusion explanations: one batched prediction of 21 rows, model-agnostic and easy to explain.
Demo realism without disturbing state	Repeated demo runs changed balances and counters, making later runs behave differently.	Dedicated simulator personas whose Redis state and balances are reset before each run; seeded one-click demo logins bound to a “registered device”.

Time	The core system had to be built in a one-week sprint.	Phased plan with the riskiest work (data, model, evaluation) first; a fixed definition of done; simple, well-understood components.

11.2 Current Limitations

1. **Synthetic data only.** All metrics show that the method works on a realistic simulation; they say nothing about real-world fraud rates. Real scam patterns are more varied, and the simulated fraud rate (≈ 2.4 %) is far higher than real UPI traffic, so precision on real traffic would be lower.
2. **No adversary.** Simulated scam behaviour is stationary; real scammers adapt to whatever gets blocked.
3. **No real payment integration.** There is no connection to NPCI, a bank or real money; the ledger is internal.
4. **The questionnaire depends on honesty.** A coached victim may answer "no" to every question; answers can only raise risk, but they cannot catch a lie.
5. **Weak on legitimate first-time payments.** New contacts and new merchants are flagged at 20–30 % in small samples — the hardest legitimate cases.
6. **Calibration drift.** The model is over-confident in the 30–60 % range on the holdout's festival period.
7. **Overrides not evaluated offline.** The confirmed-fraud-payee override and analyst labels are not exercised in offline evaluation, because the synthetic data has no analyst labels.
8. **Single-instance deployment.** One API, one worker and one ML process; tested to about 20 req/s on a laptop.
9. **Reports can be gamed with many accounts.** Each account can report a payee once, but fake accounts are not prevented; analyst review remains the authority for blocking.
10. **Approximate baselines.** Baselines refresh asynchronously, so a burst of payments may be scored against a baseline a few hundred milliseconds old (velocity counters are synchronous).
11. **Not production-certified.** No penetration test, PCI/RBI review, HA deployment or key rotation; secrets are demo defaults.
12. **No primary user validation yet.** The design is grounded in published research; structured user-acceptance results are pending (Section 10.5).

CHAPTER 12

Future Scope and Roadmap

12.1 Planned Improvements and Features

Table 12.1: Planned improvements

Area	Improvement
Real-data validation	Shadow-mode pilot with a partner institution: score real traffic without intervening, compare with confirmed fraud labels, re-tune thresholds and recalibrate.
Detection	Graph features over the payee–sender network to detect mule rings; device fingerprinting; external signals (DoT FRI for the payee's phone, Suspect Registry hits); drift monitoring with automatic alerts.
Model operations	Model registry (e.g. MLflow) with shadow deployment and canary roll-out: a candidate model scores live traffic without affecting decisions until its metrics are proven.
Intervention design	A/B-tested warning wording; “trusted contact” confirmation for seniors on HIGH payments; varied (polymorphic) warning designs to resist habituation [21]; regional languages and voice read-out.
Analyst tooling	Case grouping by payee, bulk actions, SLA timers, export to the bank's FRM system and to the 1930/CFCFRMS process.
Privacy	Federated learning across institutions to share patterns without sharing customer data; privacy-preserving payee reputation exchange.
Security and compliance	Penetration test, OWASP ASVS L2 verification, DPDP-compliant consent manager integration, CERT-In-compliant logging.

12.2 Scaling Plan

The API and ML services are stateless — all state is in Redis and PostgreSQL, and live alerts fan out via Redis pub/sub — so both scale horizontally behind a load balancer. Table 12.2 sketches the path from the prototype to national scale.

Table 12.2: Scaling stages

Stage	Load	Architecture
Prototype	≈ 20 req/s	One instance of each service on a laptop (current).
Pilot	≈ 200 req/s	3 API + 3 ML replicas behind a load balancer; managed PostgreSQL with a read replica; managed Redis; hosted in an Indian cloud region.
Regional	≈ 2,000 req/s	Redis Cluster with per-user hash tags so a user's keys share a slot; monthly partitioning of Transaction and RiskDecision; autoscaling ML pods (one process per core).
National	≥ 10,000 req/s	Kafka streams for durable, replayable events and streaming features; a feature store (online Redis + offline warehouse) to guarantee training/serving parity; multi-region active-active deployment.

12.3 Product Roadmap

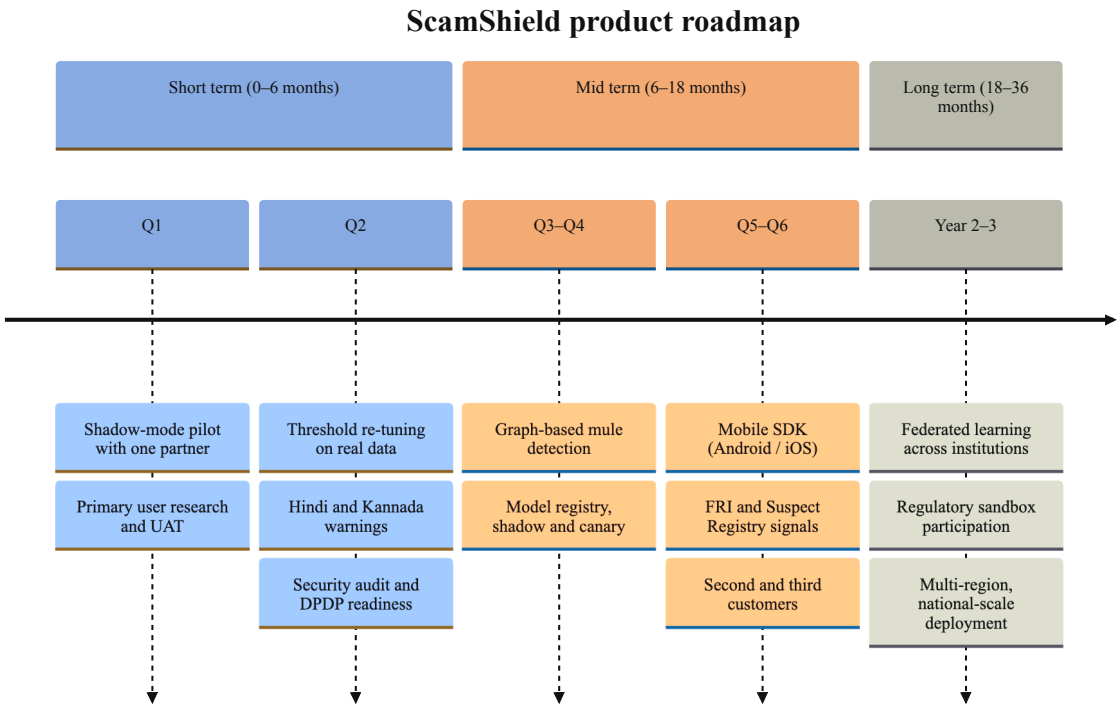


Figure 12.1: Product roadmap

CHAPTER 13

Product Launch Documentation

13.1 Product Requirements Document (Summary)

Table 13.1: PRD summary for ScamShield v1.0 (pilot release)

Item	Content
Product	ScamShield — pre-transaction scam-risk engine with payer-facing interventions and analyst/admin consoles.
Problem	Users are manipulated into authorising payments to scammers; existing controls verify identity, not intent, and act after the money has moved.
Target customers	Cooperative banks, small finance banks and smaller TPAPs originating UPI payments.
Target users	Payers (personas: senior citizen, student, shopkeeper, salaried); fraud analysts; risk administrators.
Goals	(1) Reduce successful scam payments among intervened users; (2) keep friction on genuine payments negligible; (3) shorten time from first victim to payee block.
Non-goals (v1)	Telecom-level blocking; card or net-banking fraud; credit decisions; automatic blocking without analyst review (except confirmed scam payees).
Key features	Hybrid score + band + reasons; proportional interventions; questionnaire; payee reputation and reports; live analyst console; rules editor with what-if; evaluation and retraining; simulator; health dashboard.
Success metrics	See Table 13.4 (KPIs).
Release criteria	Shadow-mode precision ≥ 50 % at HIGH and FPR ≤ 0.5 % at MEDIUM on partner data; p95 latency < 300 ms at pilot load; zero critical findings in security audit; DPDP notice and consent flow approved by partner's compliance team.
Dependencies	Partner data access and labels; integration with partner's UPI app back end; hosting in India.
Open questions	Who bears liability for a missed scam? How are blocked-payee appeals handled? Which languages first?

13.2 Lean Canvas

Figure 13.1 summarises the business model in the Lean Canvas format [64].

<table>
<tr>
<td>PROBLEM
Victims are talked into authorising scam payments
Controls check identity, not intent
Recovery after payment is slow and rare
EXISTING ALTERNATIVES
Bank FRM tools, 1930 helpline, generic SMS warnings, caller-ID apps</td>
<td>UNIQUE VALUE PROPOSITION
Stop the scam before the money moves — explain the risk in one sentence, ask what an analyst would ask, and add friction only when it matters.
HIGH-LEVEL CONCEPT
“A smoke alarm for payments.”</td>
<td>CUSTOMER SEGMENTS
Cooperative and small finance banks
Smaller UPI TPAPs and PSPs
EARLY ADOPTERS
Institutions with rising APP-fraud complaints and no in-house data-science team</td>
<td>SOLUTION
Pre-payment hybrid risk score
Proportional, explained interventions
Analyst feedback loop and payee blocking
KEY METRICS
Scams prevented
False-positive rate
Time to payee block</td>
<td>UNFAIR ADVANTAGE
Payer-facing intervention design + honest, reproducible evaluation harness
Lightweight stack affordable for small banks
CHANNELS
Direct sales to risk teams
Core-banking / UPI-switch vendors
Sandboxes and fintech events</td>
</tr>
<tr>
<td colspan="2">COST STRUCTURE
Cloud hosting in India; security audits; compliance and legal
Team salaries; customer onboarding</td>
<td colspan="3">REVENUE STREAMS
Per-check API fee (₹0.01–0.02); platform subscription; onboarding and tuning fee; on-premise licence</td>
</tr>
</table>

Figure 13.1: Lean Canvas for ScamShield

13.3 Cost Estimation and Budget

Cost of the academic project. The prototype was built entirely with open-source software on the team members' own laptops; the direct monetary cost was nil. **Pilot budget.** Table 13.2 estimates the first year of a pilot with one partner institution. All figures are planning estimates based on typical Indian cloud and services pricing, not quotations.

Table 13.2: Estimated first-year budget for a pilot (₹)

Item	Basis	Annual cost
Compute (API, worker, ML)	2 × 4 vCPU / 16 GB VMs, India region	2,90,000
Managed PostgreSQL (with replica, backups)	≈ ₹15,000 / month	1,80,000
Managed Redis	≈ ₹8,000 / month	96,000
Monitoring, logging (180-day retention), backups	≈ ₹7,000 / month	84,000
Security audit and penetration test	One external engagement	4,00,000
Legal and compliance (DPDP, contracts)	Counsel review, policies	3,00,000
Team	4 members × ₹30,000 / month stipend	14,40,000
Domain, certificates, tools, travel to partner	Miscellaneous	60,000
Contingency	10 %	2,85,000
Total		**≈ 31,35,000**

At the assumed price of ₹0.02 per check, the pilot's non-team costs (≈ ₹14 lakh) are covered at about 7 crore scored payments a year — roughly 2 lakh a day, which a mid-sized cooperative bank or TPAP could reach. Funding routes for the pilot include institute incubation, startup-India seed schemes and fintech grand challenges.

13.4 Risk Analysis and Mitigation

Table 13.3: Risk register (likelihood and impact: L = low, M = medium, H = high)

Risk	L	I	Mitigation
Model performs worse on real data than on synthetic data	H	H	Shadow-mode pilot before any intervention; recalibrate and re-tune thresholds on real labels; rules provide a floor.
High false-positive rate annoys users	M	H	LOW band sees nothing; monitor FPR by segment; what-if preview before rule changes; kill-switch per rule.
Scammers coach victims through warnings	H	M	Answers only raise risk; OTP override; vary warning designs; payee-side signals (mule, reports) independent of victim.
Data breach or misuse	L	H	Data minimisation; encryption; access control; audit; CERT-In / DPDP incident plan; India-only hosting.
ML service outage	M	M	Timeout + circuit breaker + rules-only degraded mode (implemented and tested).
Regulatory change	M	M	Track RBI / NPCI / MeitY notifications; configurable rules and thresholds; modular consent.
Competition from large apps / vendors	H	M	Focus on underserved small institutions; integrate as a complement to existing FRM.
Malicious mass reporting of a payee	M	L	One report per account; reports never block alone; analyst verdict required.
Key-person dependency in a small team	M	M	Documentation (architecture, API, data, demo script); tests; shared code ownership.

13.5 Key Metrics and KPIs After Launch

Table 13.4: Post-launch KPIs and targets for the pilot

KPI	Definition	Pilot target
Scam interception rate	Confirmed scam payments flagged MEDIUM/HIGH ÷ all confirmed scam payments	≥ 70 %
Prevented loss	Value of flagged scam payments the user cancelled	Track and report monthly
False-positive rate	Genuine payments flagged ÷ genuine payments (MEDIUM)	≤ 0.5 %
HIGH-band precision	Confirmed scams ÷ HIGH decisions	≥ 50 %
Cancellation rate on HIGH	HIGH payments cancelled ÷ HIGH payments	≥ 60 %
Time to payee block	First victim payment → analyst FRAUD verdict	Median ≤ 2 h
Decision latency	p95 of check endpoint	< 300 ms
Degraded-mode share	Decisions scored rules-only ÷ all decisions	≤ 0.1 %
Alert handling	Median time from alert to verdict	≤ 30 min (business hours)
User sentiment	Post-warning survey: “the warning was helpful”	≥ 70 % agree

CHAPTER 14

Conclusion

This project set out to address a problem that the authentication layer of digital payments cannot solve on its own: people being persuaded to authorise payments to scammers. Our answer, ScamShield, moves the defence to the one moment when it can still prevent the loss — after the user presses Pay and before the money moves — and treats that moment as both a detection problem and a human-design problem.

On the detection side, we built a hybrid engine that combines eleven transparent, runtime-configurable rules with a calibrated gradient-boosting model over twenty behavioural features, computed identically in TypeScript and Python. Evaluated on a time-based synthetic holdout of 45,387 payments that the model never saw, the hybrid catches 95.6 % of scams at the false-positive rate at which rules alone catch 73.3 %, and at the operating thresholds it achieves the best precision (79.1 % at MEDIUM, 96.0 % at HIGH) and the lowest false-positive rate (0.43 % and 0.056 %) of the three systems compared. We also report what did not favour us: the ML model alone ranks payments slightly better than the hybrid, the model is over-confident in part of the probability range during the festival period, and the first version of our data contained two leaks that made results look near-perfect until we found and fixed them.

On the human side, the system intervenes in proportion to risk: it stays silent for the great majority of payments, explains a medium-risk payment in plain language and asks three questions that capture the context transaction data cannot see, and imposes deliberate friction — a pause and re-typing of the recipient's UPI ID — on high-risk payments, with “Cancel and report” as the obvious choice. Analysts receive live alerts with full explanations, and their verdicts protect every other user by blocking confirmed scam accounts and feeding retraining.

On the engineering side, the system meets its non-functional requirements: p95 latency of about 110 ms against a 300 ms budget, graceful degradation to rules-only scoring when the model is unavailable, idempotent and double-spend-safe payments, and 341 passing automated tests.

The most important limitation is also the clearest next step: every result in this report comes from synthetic data. The method — leak-free features, time-based splits, calibration, fair comparison against a baseline, and honest reporting — is what we are confident in; the numbers themselves must be re-established on real data in a shadow-mode pilot before the system is trusted with real payments. The product analysis suggests that such a pilot is both

feasible and worthwhile: India's fraud-prevention market is growing at about 20 % a year, regulation now expects risk-based checks on digital payments, and smaller banks and payment apps lack an explainable, payer-facing layer of protection.

ScamShield lowers risk; it does not and cannot guarantee that every scam will be caught. But by giving people the right information, at the right moment, with the right amount of friction, it shows how a payment system can become an active ally of its users against those who would deceive them.

References

References are listed in IEEE style in order of first citation. Government statistics from parliamentary replies are cited through the published reports that carried them; readers requiring the primary record should consult the corresponding Lok Sabha / Rajya Sabha question on the Digital Library of Parliament (eparlib.sansad.in).

1. Free Press Journal, "UPI transaction volume rises nearly 13,000-fold in a decade to over 24,162 crore: Govt" (reporting a Lok Sabha written reply citing NPCI data). [Online]. Available: https://www.freepressjournal.in/business/upi-transaction-volume-rises-nearly-13000-fold-in-a-decade-to-over-24162-crore-govt. Accessed: Nov. 7, 2026.
2. Outlook Money, "UPI transaction value reaches Rs 314 lakh crore in FY26, Govt says in Lok Sabha." [Online]. Available: https://www.outlookmoney.com/banking/upi-transaction-value-reaches-rs-314-lakh-crore-in-fy26-govt-says-in-lok-sabha. Accessed: Nov. 7, 2026.
3. National Payments Corporation of India, "UPI product statistics." [Online]. Available: https://www.npci.org.in/product/upi/product-statistics. Accessed: Nov. 7, 2026.
4. EaseMyPrep News, "Nearly 55.49 crore users onboarded on UPI as in June 2026" (reporting a Lok Sabha written reply, Ministry of Finance), Jul. 2026. [Online]. Available: https://easemyprep.in/news/3248-nearly-5549-crore-users-onboarded-on-upi-as-in-june-2026. Accessed: Nov. 7, 2026.
5. Republic World, "UPI turns 10: 703 banks live, 60 crore daily transactions; IMF calls it world's largest real-time payment system." [Online]. Available: https://www.republicworld.com/india/upi-turns-10-703-banks-live-60-crore-daily-transactions-imf-calls-it-worlds-largest-real-time-payment-system. Accessed: Nov. 7, 2026.
6. YourStory, "UPI frauds peak in FY24, show signs of decline: Parliament data," Dec. 2025 (reporting Lok Sabha Unstarred Question 2481, Ministry of Finance). [Online]. Available: https://yourstory.com/2025/12/upi-frauds-peak-in-fy24-show-signs-of-decline-parliament-data. Accessed: Nov. 7, 2026.
7. YourStory, "Citizens lost over Rs 22,845 crore to cyber criminals in 2024," Jul. 2025 (reporting a Lok Sabha written reply, Ministry of Home Affairs). [Online]. Available: https://yourstory.com/2025/07/citizens-lost-over-rs-22845-cr-to-cyber-criminals-in-2024. Accessed: Nov. 7, 2026.
8. Business Standard, "One in five UPI users' families hit by transaction fraud since 2022: LocalCircles survey," Jun. 26, 2025. [Online]. Available: https://www.business-standard.com/finance/news/upi-transaction-fraud-india-survey-one-in-five-users-hit-localcircles-125062601141_1.html. Accessed: Nov. 7, 2026.
9. Free Press Journal, "Centre blocks 3,718 fraudulent apps, saves Rs 11,158 crore from cyber fraud," Aug. 12, 2026 (reporting a Lok Sabha written reply, Ministry of Home Affairs). [Online]. Available: https://www.freepressjournal.in/india/centre-blocks-3718-fraudulent-apps-saves-11158-crore-from-cyber-fraud. Accessed: Nov. 7, 2026.
10. R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002, doi: 10.1214/ss/1042727940.
11. A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot, and G. Bontempi, "Learned lessons in credit card fraud detection from a practitioner perspective," *Expert Systems with Applications*, vol. 41, no. 10, pp. 4915–4928, 2014, doi: 10.1016/j.eswa.2014.02.026.

12. IANS, "UPI frauds worth Rs 805 crore witnessed this fiscal so far: Minister," Dec. 15, 2025. [Online]. Available: https://ianslive.in/upi-frauds-worth-rs-805-crore-witnessed-this-fiscal-so-far-minister--20251215175703. Accessed: Nov. 7, 2026.

13. Inc42, "Indians lost INR 1,935 Cr to digital arrest scams in 2024: Govt," Mar. 2025 (reporting a Rajya Sabha reply). [Online]. Available: https://inc42.com/buzz/indians-lost-inr-1935-cr-to-digital-arrest-scams-in-2024-govt. Accessed: Nov. 7, 2026.

14. NewsMeter, "51 percent of UPI fraud victims never filed a complaint: LocalCircles survey." [Online]. Available: https://newsmeter.in/data-stories/51-percent-of-upi-fraud-victims-never-filed-a-complaint-localcircles-survey-750856. Accessed: Nov. 7, 2026.

15. MediaNama, "Cards and internet frauds made up 66% of bank fraud cases in FY25: RBI," Dec. 30, 2025 (reporting RBI, *Report on Trend and Progress of Banking in India 2024-25*). [Online]. Available: https://www.medianama.com/2025/12/223-cards-internet-fraud-66-in-fy25-decline-banking-rbi/. Accessed: Nov. 7, 2026.

16. F. Stajano and P. Wilson, "Understanding scam victims: Seven principles for systems security," *Communications of the ACM*, vol. 54, no. 3, pp. 70–75, 2011, doi: 10.1145/1897852.1897872.

17. D. Akhawe and A. P. Felt, "Alice in Warningland: A large-scale field study of browser security warning effectiveness," in *Proc. 22nd USENIX Security Symposium*, 2013, pp. 257–272.

18. A. Vance, B. B. Anderson, C. B. Kirwan, and D. Eargle, "Tuning out security warnings: A longitudinal examination of habituation through fMRI, eye tracking, and field experiments," *MIS Quarterly*, vol. 42, no. 2, pp. 355–380, 2018, doi: 10.25300/MISQ/2018/14124.

19. A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, 2018, doi: 10.1109/TNNLS.2017.2736643.

20. The Paypers, "Lenders reimburse GBP 173 million in APP fraud claims under new regime." [Online]. Available: https://thepaypers.com/fraud-and-fincrime/news/lenders-reimburse-gbp-173-million-in-app-fraud-claims-under-new-regime. Accessed: Nov. 7, 2026.

21. B. B. Anderson, A. Vance, C. B. Kirwan, D. Eargle, and J. L. Jenkins, "How polymorphic warnings reduce habituation in the brain: Insights from an fMRI study," in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, 2015, pp. 2883–2892, doi: 10.1145/2702123.2702322.

22. A. Cooper, *The Inmates Are Running the Asylum*. Indianapolis, IN, USA: Sams Publishing, 2004.

23. MarketsandMarkets, "India fraud detection and prevention market." [Online]. Available: https://www.marketsandmarkets.com/Market-Reports/geography/fraud-detection-prevention-market/India. Accessed: Nov. 7, 2026.

24. IMARC Group, "India fraud detection and prevention market size, share, trends and forecast 2025-2033." [Online]. Available: https://www.imarcgroup.com/india-fraud-detection-prevention-market. Accessed: Nov. 7, 2026.

25. KPMG India, "Reserve Bank of India (Authentication Mechanisms for Digital Payment Transactions) Directions, 2025," Dec. 2025. [Online]. Available: https://kpmg.com/in/en/insights/2025/12/reserve-bank-of-india-rbi-authentication-mechanisms-for-digital-payment-transactions-directions-2025.html. Accessed: Nov. 7, 2026.

26. MediaNama, "23 banks using RBI's MuleHunter.AI to detect mule accounts: RTI," Dec. 2025. [Online]. Available: https://www.medianama.com/2025/12/223-rti-23-banks-mulehunter-mule-accounts/. Accessed: Nov. 7, 2026.

27. Business Today, "Indian govt, RBI Innovation Hub sign AI pact to tackle financial frauds," May 12, 2026. [Online]. Available: https://www.businesstoday.in/technology/story/indian-govt-rbi-innovation-signs-ai-pact-to-tackle-financial-frauds-531090-2026-05-12. Accessed: Nov. 7, 2026.

28. Business Standard, "DoT launches Financial Fraud Risk Indicator to aid cybercrime detection," May 21, 2025. [Online]. Available: https://www.business-standard.com/industry/news/dot-launches-financial-fraud-risk-indicator-to-aid-cybercrime-detection-125052101912_1.html. Accessed: Nov. 7, 2026.

29. Free Press Journal, "Government's anti-cyber fraud drive disconnects 39.4 lakh mobile connections, blacklists 2.27 lakh handsets," Feb. 2026. [Online]. Available: https://www.freepressjournal.in/tech/governments-anti-cyber-fraud-drive-disconnects-394-lakh-mobile-connections-blacklists-227-lakh-handsets. Accessed: Nov. 7, 2026.

30. *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998, doi: 10.1109/IEEESTD.1998.88286.

31. Government of India, *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023), Gazette of India, Aug. 11, 2023.

32. Storyboard18, "Govt notifies final DPDP Rules, sets staggered rollout with key obligations for data fiduciaries," Nov. 2025. [Online]. Available: https://www.storyboard18.com/digital/breaking-govt-notifies-final-dpdp-rules-sets-staggered-rollout-with-key-obligations-for-data-fiduciaries-84201.htm. Accessed: Nov. 7, 2026.

33. Government of India, *The Information Technology Act, 2000* (Act No. 21 of 2000), as amended by the Information Technology (Amendment) Act, 2008.

34. Ministry of Communications and Information Technology, *Information Technology (Reasonable Security Practices and Procedures and Sensitive Personal Data or Information) Rules, 2011*, G.S.R. 313(E), Apr. 11, 2011.

35. Indian Computer Emergency Response Team (CERT-In), "Directions under sub-section (6) of section 70B of the Information Technology Act, 2000 relating to information security practices, procedure, prevention, response and reporting of cyber incidents for safe and trusted Internet," No. 20(3)/2022-CERT-In, Apr. 28, 2022.

36. Reserve Bank of India, "Storage of Payment System Data," RBI/2017-18/153, DPSS.CO.OD No. 2785/06.08.005/2017-2018, Apr. 6, 2018.

37. European Parliament and Council, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, L 119, pp. 1–88, May 4, 2016.

38. OWASP Foundation, "Application Security Verification Standard 4.0.3," Oct. 2021. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

39. W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 5, 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/

40. Government of India, *The Rights of Persons with Disabilities Act, 2016* (Act No. 49 of 2016).

41. Reserve Bank of India, "Master Direction on Digital Payment Security Controls," RBI/2020-21/74, Feb. 18, 2021. [Online]. Available: https://www.rbi.org.in/Scripts/BS_ViewMasDirections.aspx?id=12032. Accessed: Nov. 7, 2026.

42. Reserve Bank of India, "Master Directions on Fraud Risk Management in Commercial Banks (including Regional Rural Banks) and All India Financial Institutions," Jul. 15, 2024. Summary: Grant Thornton Bharat. [Online]. Available: https://www.grantthornton.in/globalassets/1.-member-

firms/india/assets/pdfs/services/rbi-master-directions-on-fraud-risk-management.pdf. Accessed: Nov. 7, 2026.

43. The Week, "Major UPI changes from August 1: balance enquiry limits, pending payment status and more from NPCI," Aug. 1, 2025. [Online]. Available: https://www.theweek.in/news/biz-tech/2025/08/01/major-upi-changes-from-august-1-here-is-the-full-list-of-balance-enquiry-limits-pending-payment-status-upi-security-and-more-from-npci.html. Accessed: Nov. 7, 2026.
44. Docker, Inc., "Docker Compose documentation." [Online]. Available: https://docs.docker.com/compose/. Accessed: Nov. 7, 2026.
45. Vercel, "Next.js documentation." [Online]. Available: https://nextjs.org/docs. Accessed: Nov. 7, 2026.
46. OpenJS Foundation, "Express — Node.js web application framework." [Online]. Available: https://expressjs.com/. Accessed: Nov. 7, 2026.
47. Prisma Data, Inc., "Prisma ORM documentation." [Online]. Available: https://www.prisma.io/docs. Accessed: Nov. 7, 2026.
48. The PostgreSQL Global Development Group, "PostgreSQL 16 documentation." [Online]. Available: https://www.postgresql.org/docs/16/. Accessed: Nov. 7, 2026.
49. Redis Ltd., "Redis documentation." [Online]. Available: https://redis.io/docs/. Accessed: Nov. 7, 2026.
50. Taskforce.sh, "BullMQ documentation." [Online]. Available: https://docs.bullmq.io/. Accessed: Nov. 7, 2026.
51. S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/. Accessed: Nov. 7, 2026.
52. F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
53. T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
54. G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154.
55. S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.
56. Microsoft, "Playwright documentation." [Online]. Available: https://playwright.dev/. Accessed: Nov. 7, 2026.
57. J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.
58. B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proc. 8th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2002, pp. 694–699, doi: 10.1145/775047.775151.
59. A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Machine Learning (ICML)*, 2005, pp. 625–632, doi: 10.1145/1102351.1102430.
60. M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.
61. M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

62. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015, doi: 10.17487/RFC7519.

63. T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015, doi: 10.1371/journal.pone.0118432.

64. A. Maurya, *Running Lean: Iterate from Plan A to a Plan That Works*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2012.

APPENDIX A

Questionnaire

Title: "Digital payments and scams — a 5-minute survey" (IIIT Dharwad final-year project). **Consent statement shown first:** "This survey is anonymous and voluntary. We do not collect your name, phone number or account details. Responses will be used only for our academic project. You may stop at any time." Respondents under 18 were asked not to continue.

A.1 Section A — About you and your payments

1. Your age group: Under 18 (end survey) / 18–24 / 25–40 / 41–59 / 60 and above
2. Your occupation: Student / Salaried / Self-employed or shop owner / Homemaker / Retired / Other
3. How often do you make UPI payments? Several times a day / Once a day / A few times a week / Rarely / Never
4. Which payment app do you use most? (open text)
5. Who set up your UPI app? Myself / A family member / Bank staff / Other

A.2 Section B — Exposure to scams

6. In the last 12 months, have you received a call or message you believe was a scam? Yes / No / Not sure
7. If yes, what kind? (tick all) Fake KYC/bank / Fake police/customs ("digital arrest") / Job or task offer / Investment / Fake buyer or refund / Lottery or prize / Request for OTP / Request to install an app / Other
8. Have you or a family member ever lost money through a UPI scam? Yes, me / Yes, family member / No / Prefer not to say
9. If yes, how did the payment happen? I paid myself after being convinced / I shared an OTP or PIN / An app was installed on my phone / I don't know / Other
10. How was the scammer in contact at the time of payment? On a phone/video call / By chat or SMS / Through a link or QR code / Not in contact / Don't know

A.3 Section C — Losses and recovery

11. Approximate amount lost: Under ₹1,000 / ₹1,000–10,000 / ₹10,000–1,00,000 / Above ₹1,00,000 / Prefer not to say
12. Was any money recovered? All / Some / None / Still pending

13. Did you report it? Bank / 1930 helpline or cybercrime.gov.in / Police / Did not report — why? (open text)

A.4 Section D — Warnings and friction

14. Have you ever seen a warning in your payment app before a payment? Yes / No / Don't remember
15. If your app believed a payment was very likely a scam, would you accept a 10-second pause before you could continue? Yes / Only for large amounts / No
16. Would you accept having to retype the recipient's UPI ID for such a payment? Yes / Maybe / No
17. Would you answer three quick yes/no questions (e.g. "Did someone call you asking you to pay urgently?") before a risky payment? Yes / Maybe / No
18. Which message would make you stop and think most? (a) "Be careful while paying." (b) "You have never paid this person. Their account is 2 days old and 3 people have reported it." (c) "This payment is risky."

A.5 Section E — Trust and privacy

19. Would you be comfortable with your payment app analysing your payment patterns (amounts, times, recipients) only to detect scams? Yes / Yes, if I can see what is used / No
20. Any suggestions or experiences you would like to share? (open text)

A.6 Interview guide (semi-structured, 10–15 minutes)

- Tell us about the last suspicious call or message you received. What made you suspicious — or not?
- If you (or someone you know) paid a scammer: walk us through the moments before pressing Pay. What would have made you stop?
- [Show HIGH-risk screen] What do you understand from this screen? What would you do?
- [Show MEDIUM screen] Are the questions clear? Would you answer honestly if someone was on the phone with you?
- For shopkeepers: which of your routine payments might look unusual to a computer?

APPENDIX B

Survey Data

TEAM TO COMPLETE — Paste the summary of the collected responses here (or attach the exported spreadsheet), using the coding sheet below. Report n, dates and channel; exclude incomplete responses and state how many were excluded.

Table B.1: Coding sheet for survey responses

Q	Variable	Coding	n	%
Q1	age_group	1 = 18–24, 2 = 25–40, 3 = 41–59, 4 = 60+		
Q3	upi_frequency	1 = several/day … 5 = never		
Q6	scam_contact_12m	1 = yes, 0 = no, 9 = not sure		
Q8	loss_self_or_family	1 = self, 2 = family, 0 = no, 9 = prefer not		
Q9	loss_mechanism	1 = authorised, 2 = OTP/PIN, 3 = app, 9 = unknown		
Q12	recovered	1 = all, 2 = some, 3 = none, 4 = pending		
Q15	accept_pause	1 = yes, 2 = large only, 0 = no		
Q17	accept_questions	1 = yes, 2 = maybe, 0 = no		
Q18	message_choice	a / b / c		
Q19	privacy_comfort	1 = yes, 2 = yes if transparent, 0 = no		

APPENDIX C

Additional Diagrams and Tables

Table C.1: Redis keys, types, time-to-live and purpose

Key	Type	TTL	Purpose
`vel:{user}`	ZSET	2 h	Completed payments for 10-min / 1-h velocity
`vel:{user}:{upi}`	ZSET	2 h	Similar amounts to the same payee (split payments)
`payee:{upi}:senders`	ZSET	25 h	Distinct senders in 24 h (mule signal)
`payee:{upi}:info`	STRING (JSON)	10 min	Payee age, reports, confirmed fraud; invalidated on change
`user:{id}:payees / :devices`	SET	7 d	Known payees / devices; rebuilt from PostgreSQL on miss
`user:{id}:profile`	HASH	24 h	30-day baseline (count, mean, std, max, hour sin/cos)
`fail:{id}`	ZSET	20 min	Failed logins / failed verifications
`idem:{user}:{key}`	STRING (JSON)	24 h	Idempotent responses
`rl:auth:*, rl:check:*`	counter	1 min	Rate limits
`cb:ml:*`	STRING	5 s – 1 h	Circuit breaker: open flag, failures, tripped, probe lock
`metrics:*`	HASH / LIST	—	Decision counts, latency samples, ML outcomes, worker summary
`scamshield:events`	pub/sub	—	Live alert events → every API instance → SSE clients

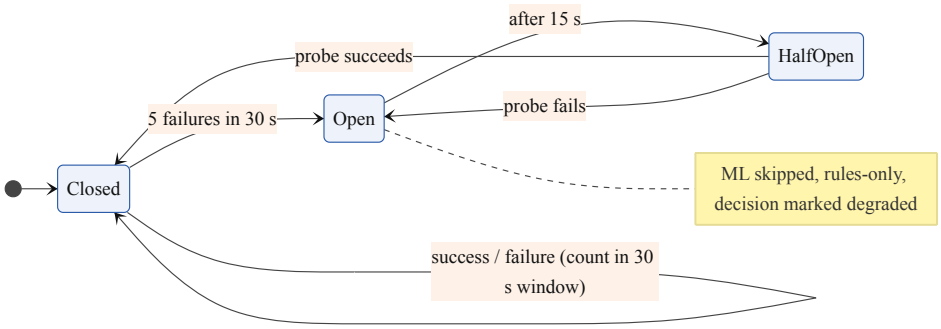


Figure C.1: Circuit-breaker state machine for the ML call

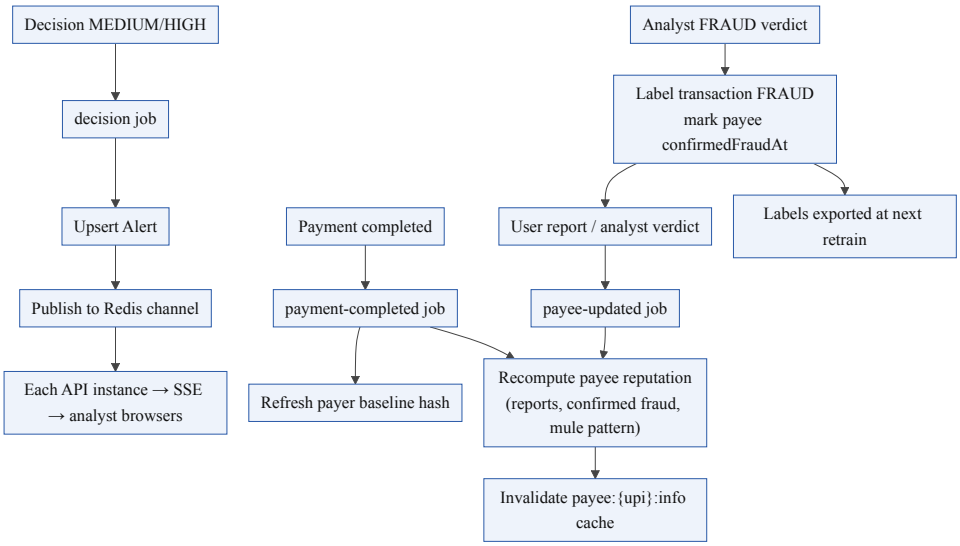


Figure C.2: Asynchronous pipeline: alerts, reputation and the feedback loop

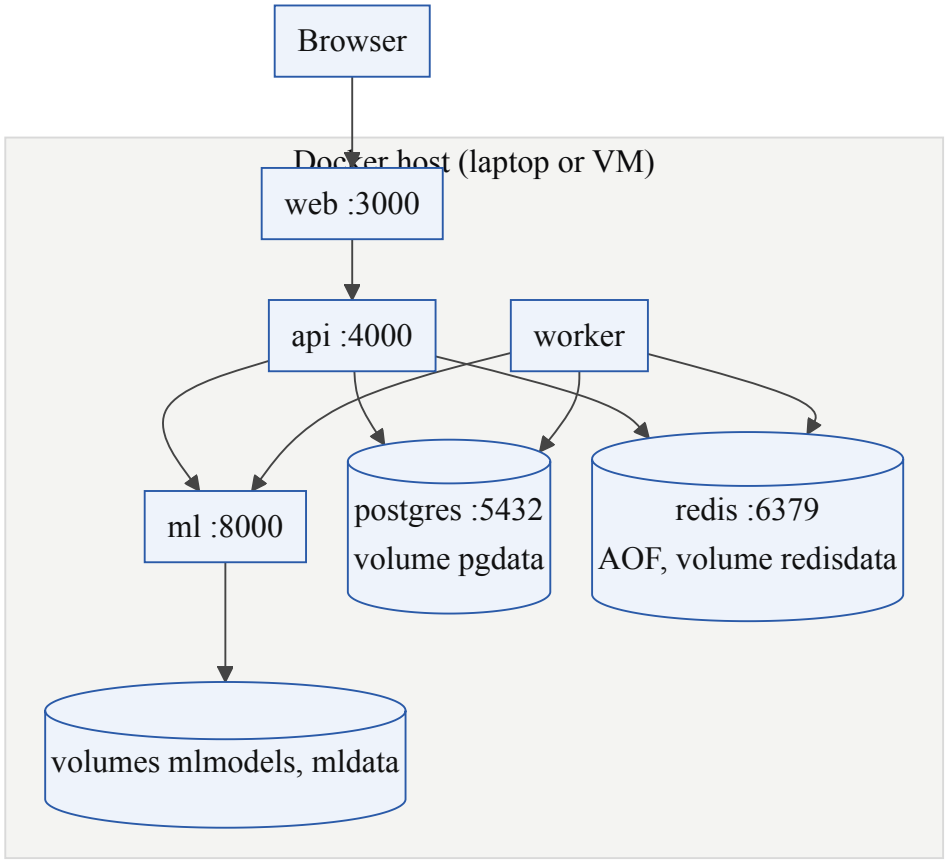


Figure C.3: Deployment diagram (Docker Compose)

APPENDIX D

Code Excerpts

D.1 Decision function (packages/shared/src/blend.ts)

```
export function blendScores(ruleScore: number, mlScore: number | null, t: Thresholds):
number {
  const raw = mlScore === null ? ruleScore : t.ruleWeight * ruleScore + t.mlWeight *
mlScore;
  return Math.round(Math.min(100, Math.max(0, raw)) * 10) / 10;
}

export function decide(input: DecisionInput): DecisionOutcome {
  const { thresholds: t } = input;
  let finalScore = blendScores(input.ruleScore, input.mlScore, t);
  const overrides: OverrideCode[] = [];
  if (input.payeeConfirmedFraud) {
    overrides.push("CONFIRMED_FRAUD_PAYEE");
    finalScore = Math.max(finalScore, CONFIRMED_FRAUD_FLOOR);      // 95
  }
  if (input.features.ctx_otp_or_app === 1) {
    overrides.push("OTP_REQUESTED");
    finalScore = Math.max(finalScore, t.highThreshold);             // at least HIGH
  }
  const reasons: Reason[] = [
    ...overrides.map((code) => ({ code, points: 100, vars: {} })),
    ...input.hits.map((h) => ({ code: h.code, points: h.points, vars: h.vars })),
  ];
  if (input.mlScore !== null && input.mlScore >= ML_REASON_MIN_SCORE) {
    reasons.push({ code: "ML_PATTERN", points: Math.round(input.mlScore * t.mlWeight),
                   vars: { mlScore: Math.round(input.mlScore) } });
  }
  return { finalScore, band: bandFor(finalScore, t), blocked:
input.payeeConfirmedFraud,
           degraded: input.mlScore === null, overrides, reasons };
}
```

D.2 Two example rules (packages/shared/src/rules.ts)

```
MULE_PATTERN: (f, p) =>
  f.payee_distinct_senders_24h >= p.minDistinctSenders && f.payee_age_days <=
p.maxPayeeAgeDays
    ? { points: p.points, vars: { senders: f.payee_distinct_senders_24h, ageDays:
one(f.payee_age_days) } }
    : null,
REPORTED_PAYEE: (f, p) =>
  f.payee_report_count >= 1
    ? { points: Math.min(p.maxPoints, p.points + (f.payee_report_count - 1) *
p.pointsPerExtraReport),
        vars: { reports: f.payee_report_count } }
    : null,
```

D.3 Occlusion explanation (services/ml/scamshield_ml/explain.py)

```
def top_contributions(artifact: dict, x: np.ndarray, k: int = 3) -> list[dict]:
    names = artifact["feature_names"]
    reference = artifact["reference"]          # a typical legitimate payment
    batch = np.repeat(x[None, :], len(names) + 1, axis=0)
    for i in range(len(names)):
        batch[i + 1, i] = reference[i]         # reset one feature per row
    p = artifact["raw"].predict_proba(batch)[:, 1]
    contributions = p[0] - p[1:]               # drop in fraud probability
    order = np.argsort(-contributions)[:k]
    return [{"feature": names[i], "value": float(x[i]),
             "contribution": round(float(contributions[i]), 4)}
            for i in order if contributions[i] > 0.005]
```

D.4 Atomic confirm (conceptual SQL inside one transaction)

```
BEGIN;
UPDATE "Transaction" SET status = 'COMPLETED', "completedAt" = now()
 WHERE id = $txn AND status = 'PENDING';                -- 0 rows → 409 (already
processed)
UPDATE "User" SET balance = balance - $amt
 WHERE id = $user AND balance >= $amt;                  -- 0 rows → 400 (insufficient
balance)
INSERT INTO "LedgerEntry" ("userId", "transactionId", amount, "balanceAfter") VALUES
(...);
-- credit the payee if it is a platform user; upsert Payee
COMMIT;                                                 -- any failure rolls back
everything
```

APPENDIX E

User Manual

E.1 Installation and start-up

1. Install Docker Desktop (with Compose v2) and `make`.
2. Clone the repository and run `make demo`. This copies `.env.example` to `.env`, builds and starts all six containers, and waits until the stack is ready. The first start takes 3–6 minutes because the ML container generates about 165,000 synthetic transactions and trains the model.
3. Open `http://localhost:3000`. API documentation is at `http://localhost:4000/docs`.
4. Useful commands: `make ps` (status), `make logs`, `make test`, `make e2e`, `make loadtest`, `make train`, `make reset` (deletes all data).

E.2 Demo accounts

Table E.1: Demo accounts (password Demo@1234; one-click buttons on the landing page)

Role	E-mail	Use for
User	`asha@demo.scamshield.in`	Wallet, sending money, warnings, My Safety
Analyst	`ravi@demo.scamshield.in`	Live alert queue, reviewing alerts
Admin	`admin@demo.scamshield.in`	Simulator, rules, model, health (and analyst pages)

E.3 For payers

1. **Wallet** shows your balance, spending over 30 days and recent payments with their risk band.
2. **Send money:** enter the recipient's UPI ID — the recipient card shows whether you have paid them before and how long they have been on the platform — then the amount, and press *Check & pay*.
3. **Low risk:** the payment completes immediately.
4. **Medium risk:** read the reasons and answer the three questions honestly; press *Re-check with my answers*. You may then verify the recipient, cancel, or continue.
5. **High risk:** the screen cannot be dismissed. The safest choice is *Cancel and report this account*. If you are sure you know the person, wait for the 10-second pause and retype their UPI ID exactly.
6. **Payment stopped:** the recipient has been confirmed as a scam account; the payment cannot proceed.
7. **My Safety** lists the warnings you received and the accounts you reported.

E.4 For analysts

1. Open **Alerts**. New alerts arrive live (green *Live* indicator). Filter by status and band.
2. Select an alert to see the amount, payer and payee, score breakdown (rules, ML, final), all reasons with technical detail, the model's top features and the payee's history.
3. Add notes and choose **Confirm fraud** (labels the payment and blocks all future payments to the payee for every user) or **Mark safe** (closes the alert).

E.5 For administrators

1. **Simulator:** run a scenario (normal, impersonation, mule collection, velocity drain, account takeover) and inspect each step's scores and reasons.
2. **Rules:** enable/disable rules, change weights, points and parameters, blend weights and thresholds; the what-if panel previews precision, recall, false-positive rate, band mix and per-scenario effects before you save.
3. **Model:** view the active model and its evaluation; press *Retrain model* to train on the synthetic data plus analyst labels.
4. **Health:** monitor service status, circuit breaker, decisions by band, latency percentiles, ML outcomes, queues and the worker summary.

E.6 Troubleshooting

- *"Checked with safety rules only"* on a decision means the ML service was unavailable; check `make ps` and the Health page.
- Signing in with typed credentials in a new browser counts as a new device; use the one-click demo buttons for the registered device.
- To start completely fresh, run `make reset` and then `make demo`.